

Analisi e Progettazione del Software

Diagrammi delle classi

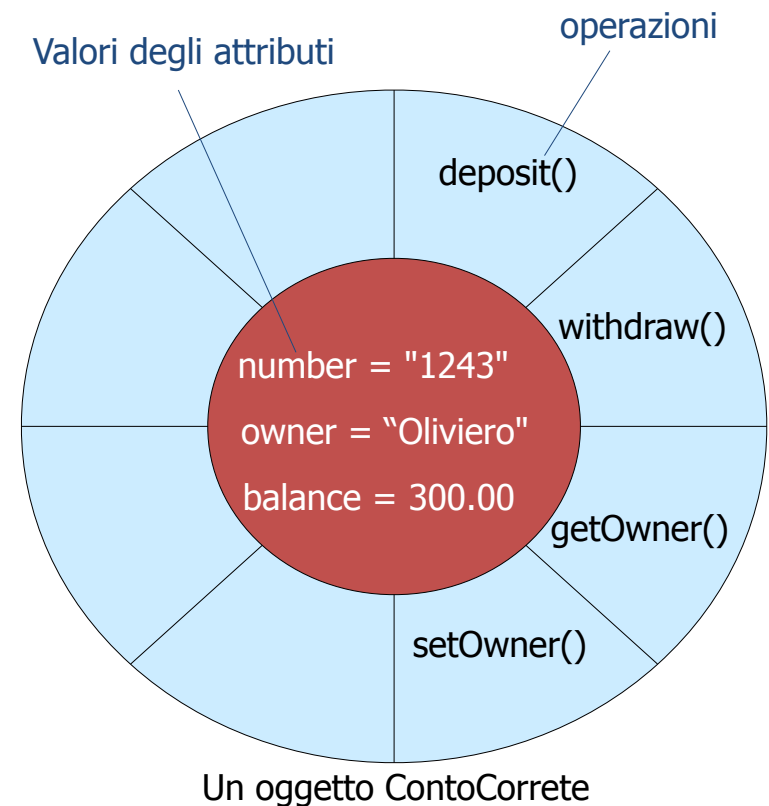
Oliviero Riganelli

Che cosa sono gli oggetti

- Un oggetto è un pacchetto coeso di dati e funzioni incapsulate in una unità riusabile. Oggetti incapsulare i dati.
- Ogni oggetto è un'istanza di una classe che definisce l'insieme comune di caratteristiche (attributi e operazioni) condivisi da tutte le sue istanze. Gli oggetti hanno:
 - *I valori degli attributi - la parte dei dati*
 - *Le operazioni - la parte comportamento*
- Tutti gli oggetti hanno:
 - *Identità: ogni oggetto ha il suo identificativo univoco*
 - *Stato: viene stabilito dai valori effettivi dei dati memorizzati in un oggetto ad un dato istante*
 - *Comportamento: l'insieme delle operazioni che l'oggetto può eseguire*

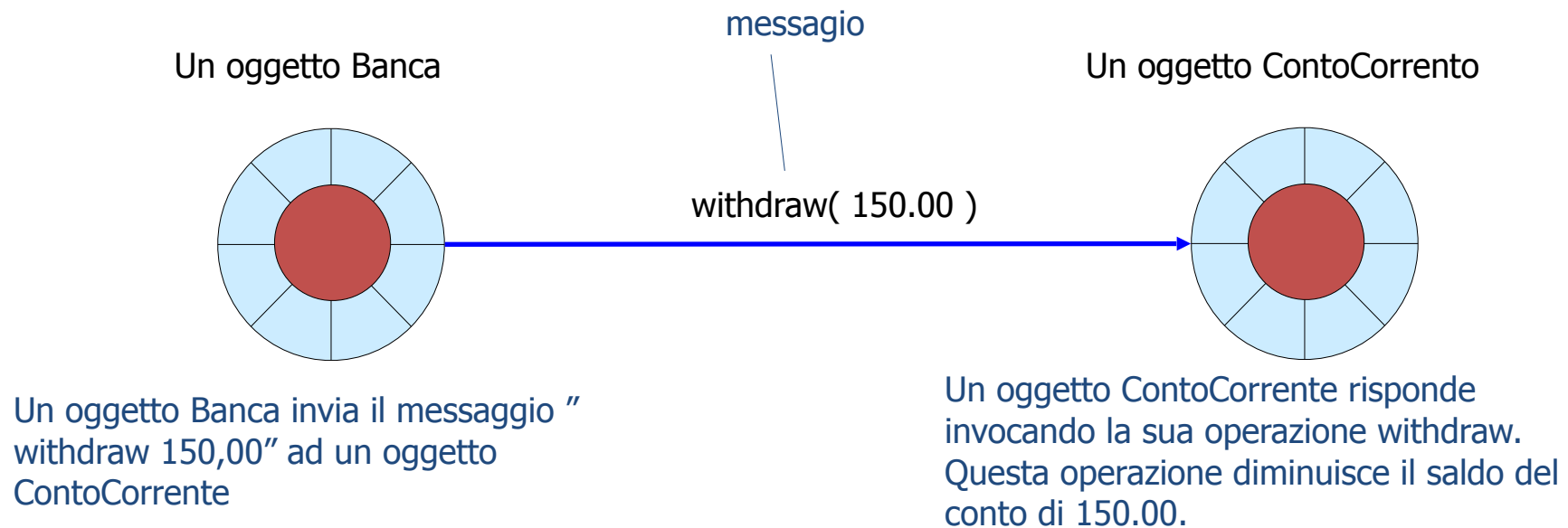
Incapsulamento

- I dati sono nascosti all'interno dell'oggetto, l'unico modo per accedere ai dati è attraverso le operazioni
- **incapsulamento** o **data hiding**:
 - *Software più robusto*
 - *Codice riusabile*

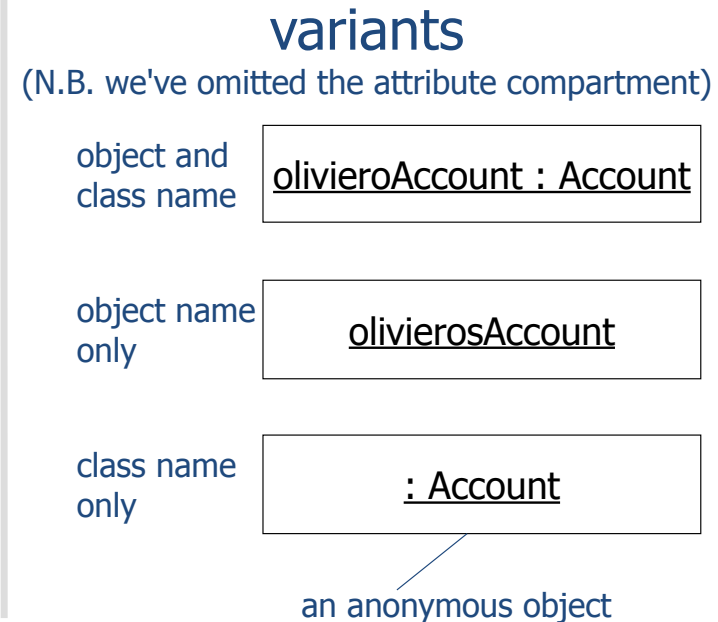
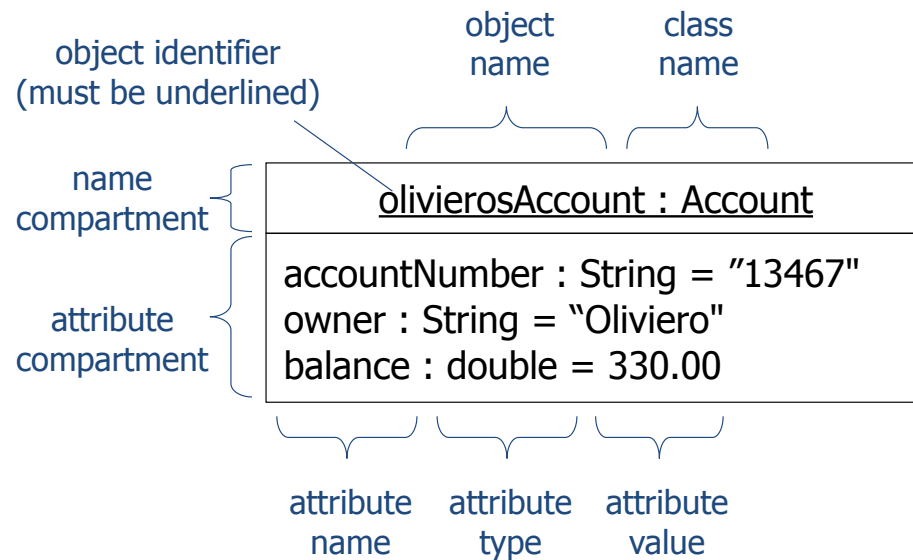


Messaggi

- Gli oggetti collaborano per eseguire le funzioni del sistema stabilendo dei collegamenti tra loro e scambiandosi messaggi
- I messaggi causano l'invocazione delle operazioni da parte dell'oggetto



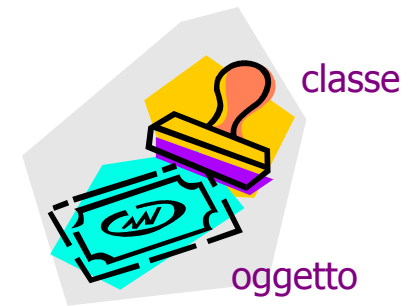
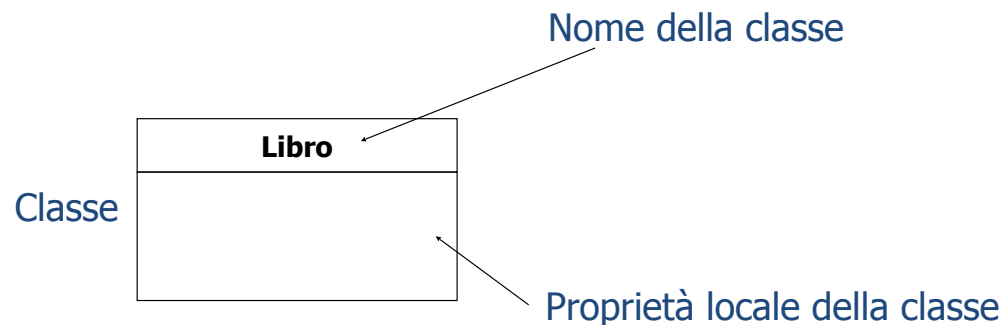
Sintassi degli oggetti UML



- Tutti gli oggetti di una particolare classe hanno lo stesso insieme di proprietà
- I nomi degli oggetti sono scritti con le lettere maiuscole e minuscole e iniziano con una lettera minuscola
- Il nome della classe inizia con la maiuscola

Che cosa sono le classi

- Una classe modella un insieme di oggetti omogenei (le istanze della classe) ai quali sono associate proprietà statiche (attributi) e dinamiche (operazioni)
- La classificazione è uno dei più importanti modi che noi abbiamo per organizzare la nostra vista del mondo



Quante classi ci sono?

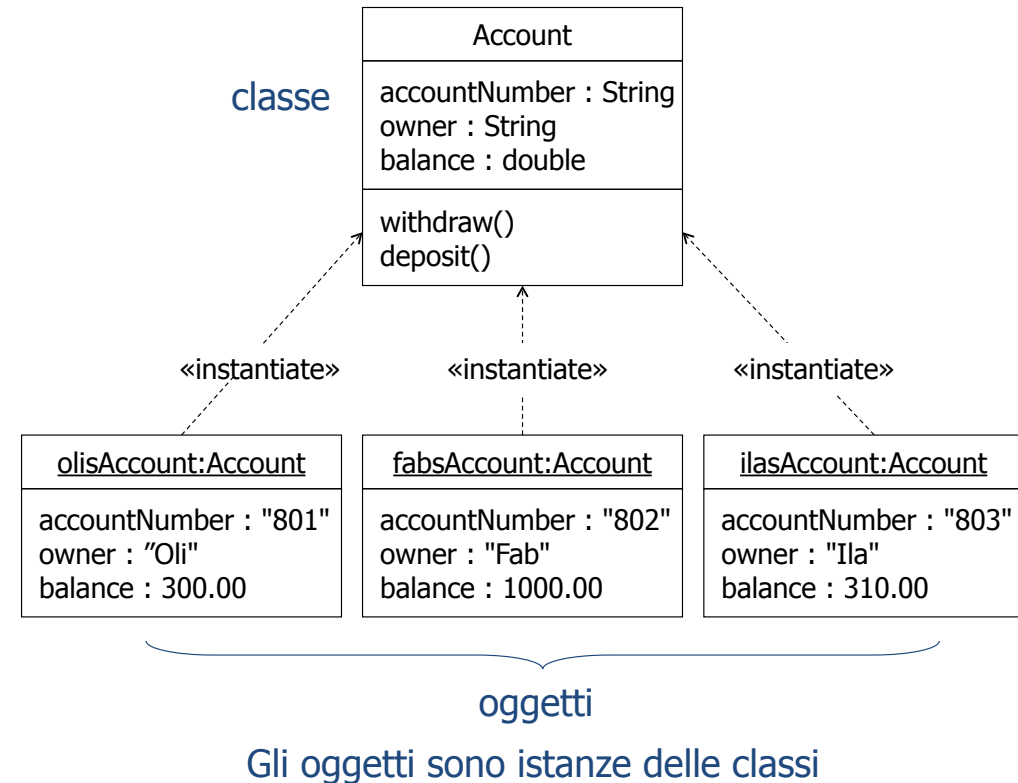


Quante classi ci sono?

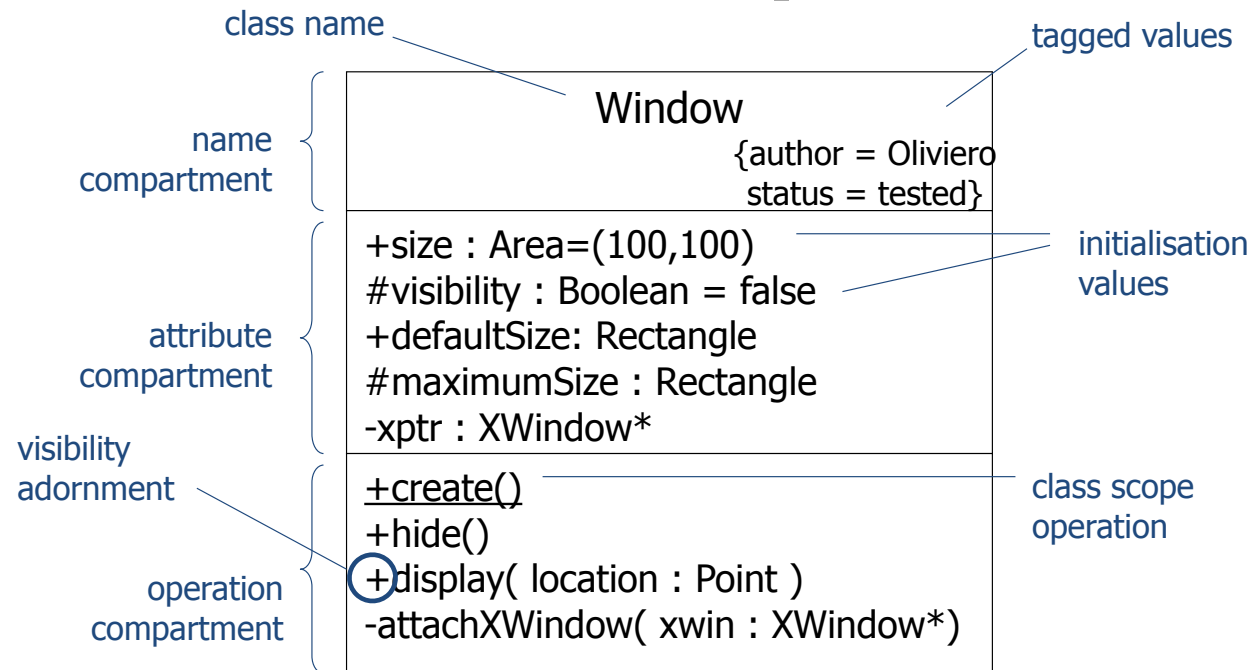
- A questa domanda non c'è risposta!
- Ci sono un numero quasi infinito di modi per classificare gli oggetti del mondo reale:
 - *La classe dei gatti*
 - *La classe dei gatti golosi*
 - *La classe degli alberi*
 - *La classe delle foglie*
 - *Ecc.*

Classi e Oggetti

- Tra un oggetto che è istanza di una classe C e la classe C si traccia un arco Instantiated
- Ricordiamo che gli oggetti formano il livello estensionale, mentre le classi a livello intensionale



Notazione UML per le classi



- Utilizzare nomi descrittivi
- Evitare abbreviazioni

Diagrammi delle classi

- I diagrammi delle classi illustrano le classi, le interfacce e le relative associazioni
- Sono utilizzati per la modellazione statica degli oggetti

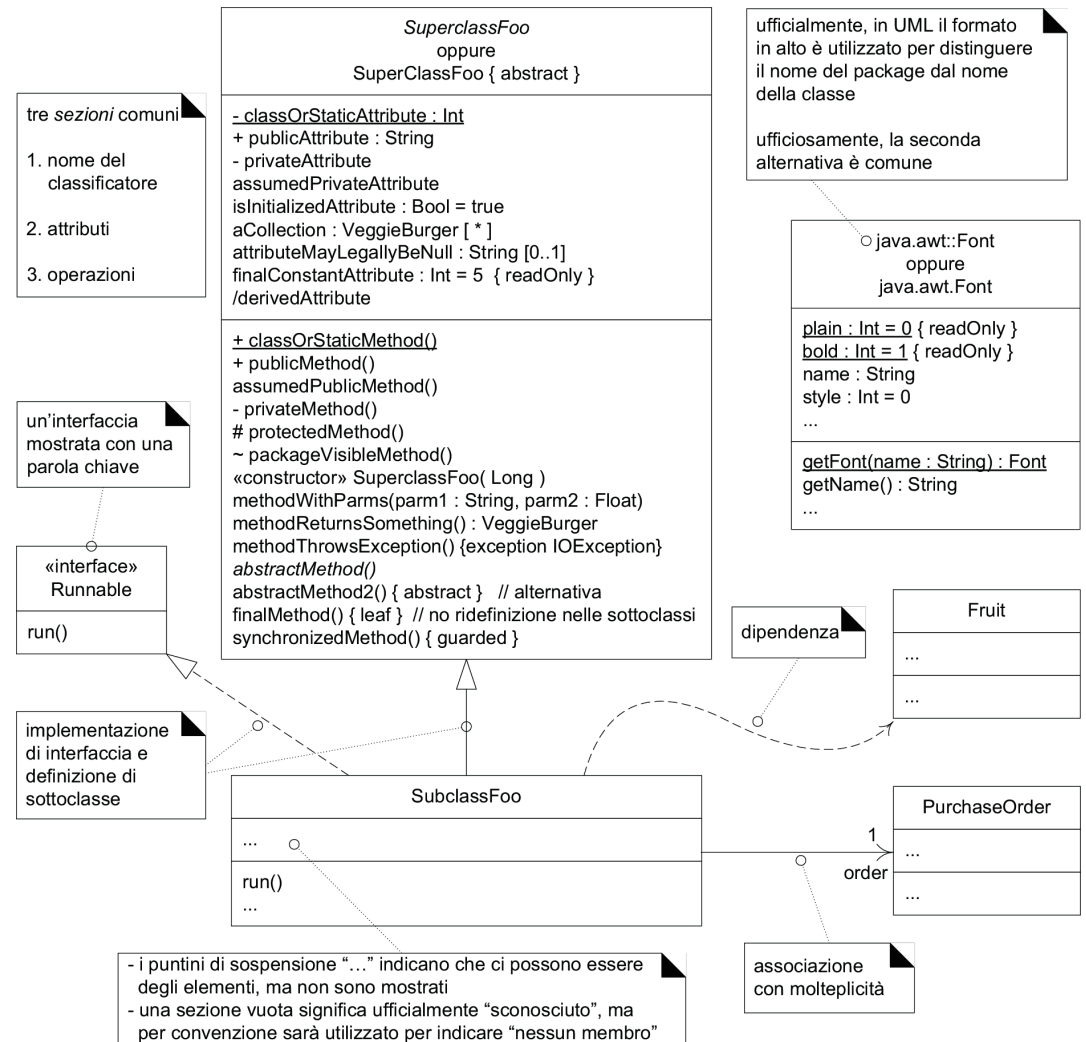
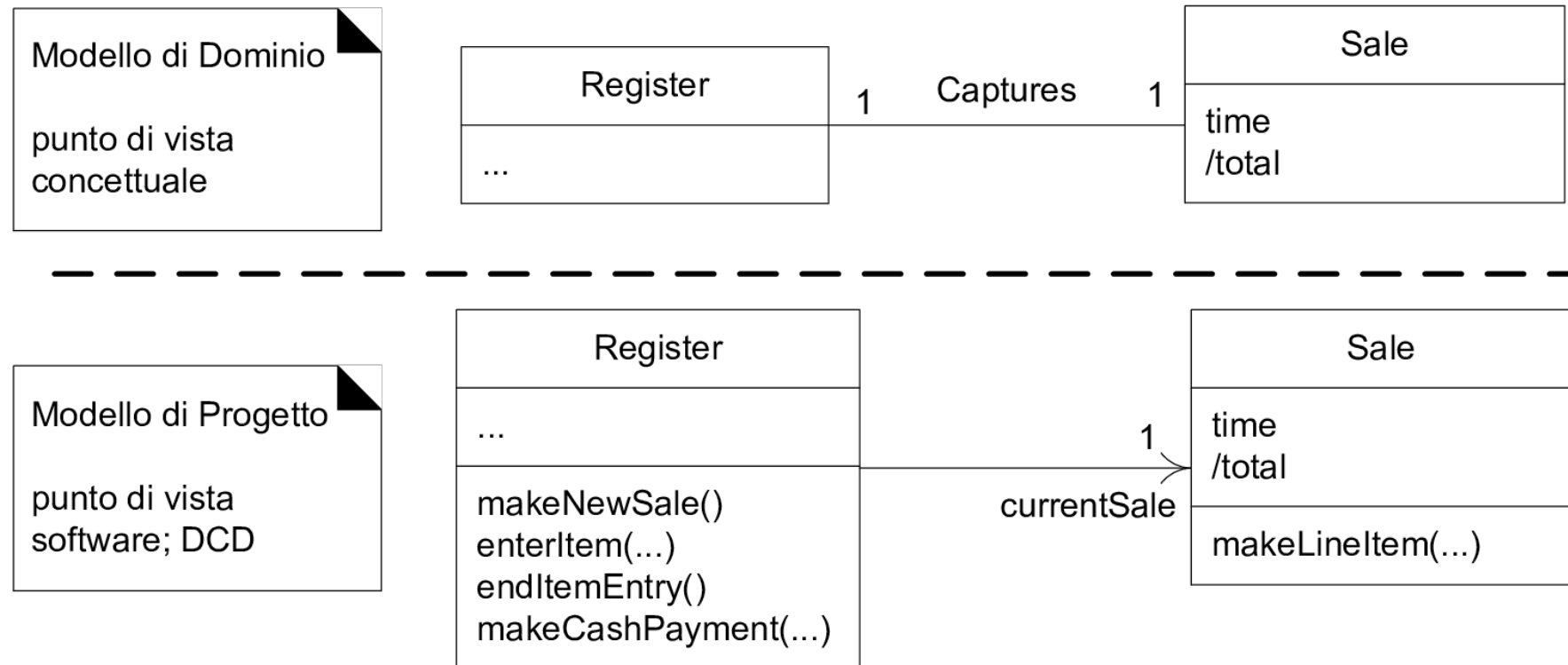


Diagramma delle classi di progetto

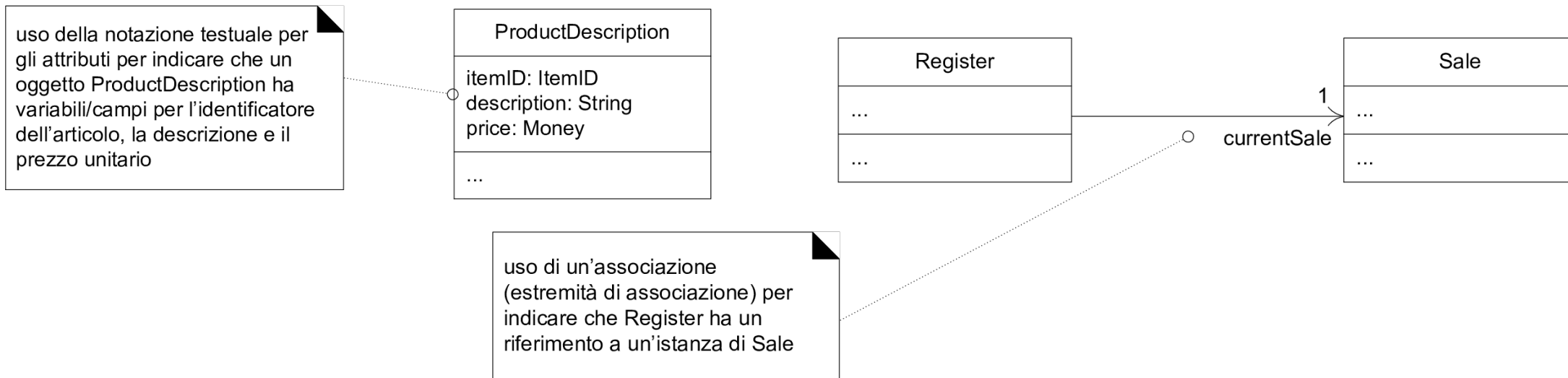


Definizione: classificatore

- In UML, un classificatore è un “elemento di modello che descrive caratteristiche comportamentali e strutturali”
- I classificatori sono una generalizzazione di molti degli elementi di UML, come classi, interfacce, casi d’uso e attori
- Nei diagrammi delle classi, i due classificatori più comuni sono le classi e le interfacce

Proprietà strutturali

- In UML le **proprietà strutturali** di un classificatore comprendo
 - *Gli attributi*
 - *Le estremità di associazioni*



Attributi

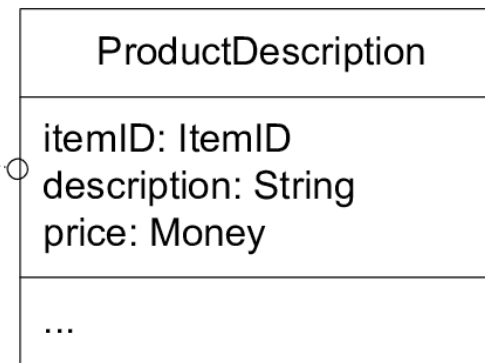
- Un attributo modella una **proprietà locale** della classe ed è caratterizzato da un nome e dal tipo dei valori associati
- Ogni attributo di una classe stabilisce una proprietà locale **valida per tutte le istanze** della classe. Il fatto che la proprietà sia locale significa che è una proprietà indipendente da altri oggetti
- Formalmente, un attributo A della classe C si può considerare una **funzione** che associa un valore di tipo T ad ogni oggetto che è istanza di C

Attributi in UML

visibilità nome : tipo [molteplicità] = valoreIniziale

obbligatorio

uso della notazione testuale per gli attributi per indicare che un oggetto ProductDescription ha variabili/campi per l'identificatore dell'articolo, la descrizione e il prezzo unitario

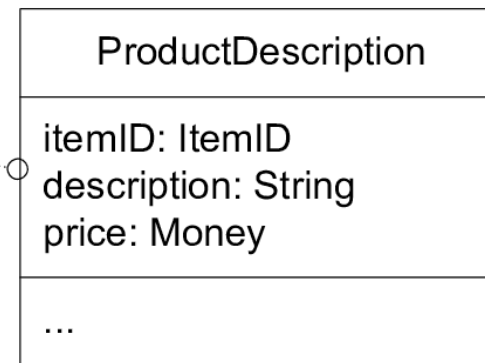


- Solo il nome è obbligatorio
- valoreIniziale è il valore che l'attributo prende quando gli oggetti della classe sono creati

Attributi in UML

Se non viene indicata alcuna visibilità, solitamente si ipotizza che gli attributi siano privati

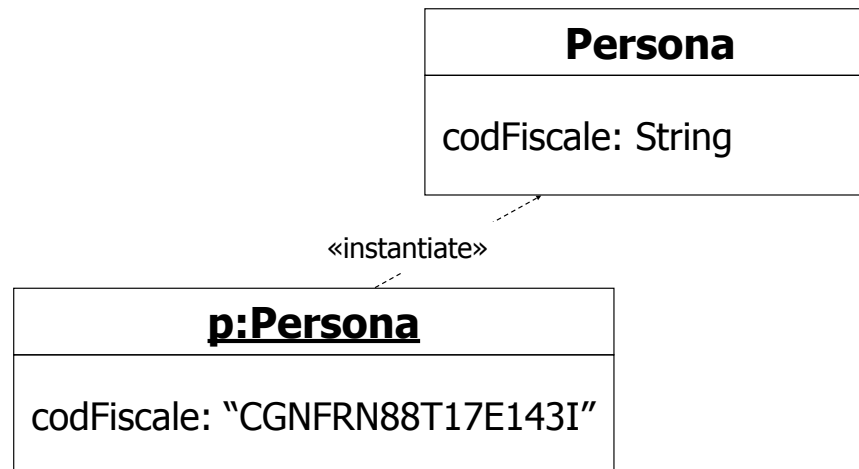
uso della notazione testuale per gli attributi per indicare che un oggetto ProductDescription ha variabili/campi per l'identificatore dell'articolo, la descrizione e il prezzo unitario



```
public class ProductDescription {  
    private ItemID itemID;  
    private String description;  
    private Money price;  
    ...  
}
```

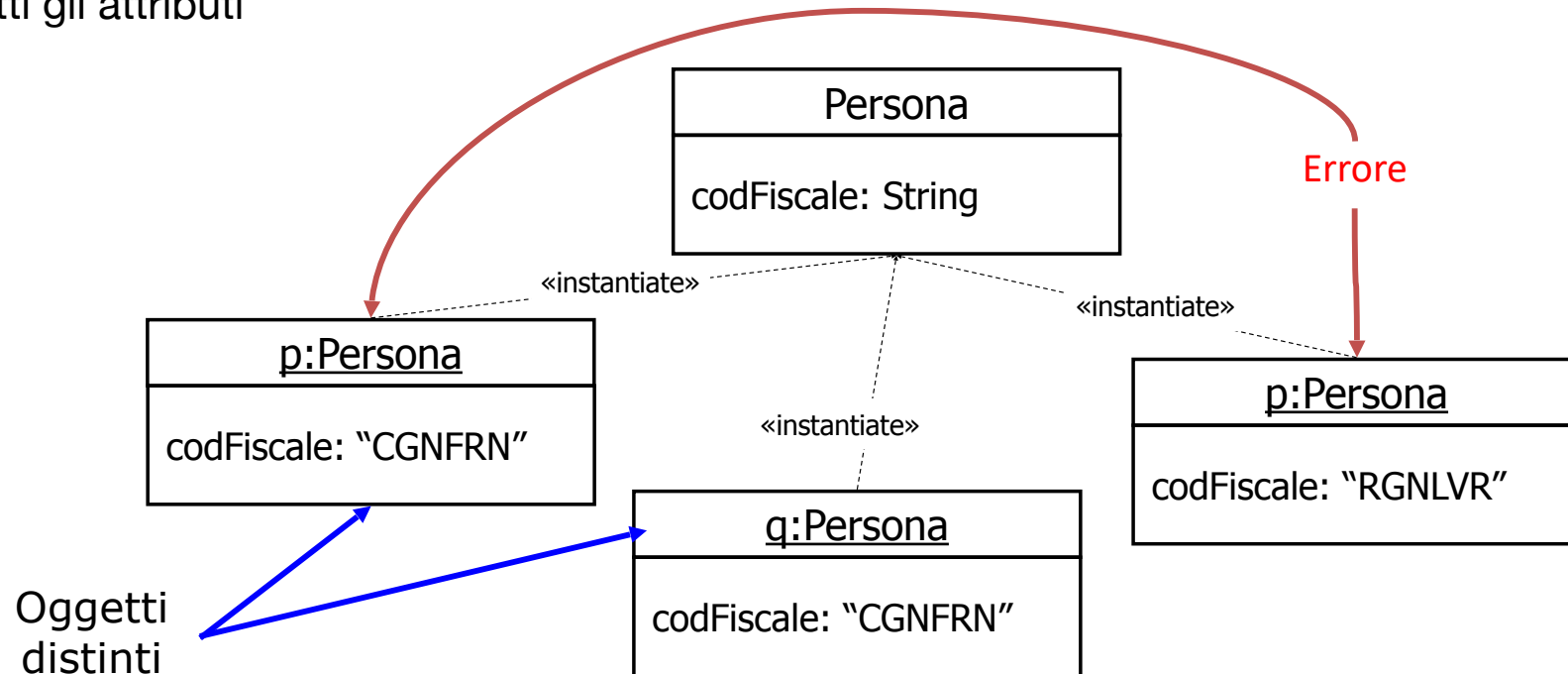
Attributi di oggetti

- Gli attributi di una classe determinano gli attributi delle sue istanze
- **Regola importante:** se una classe C ha un attributo A di tipo T, ogni oggetto che è istanza di C ha l'attributo A, con un valore associato di tipo T
- **Regola importante:** un oggetto X non può avere un valore per un attributo che non è definito nella classe di cui X è istanza

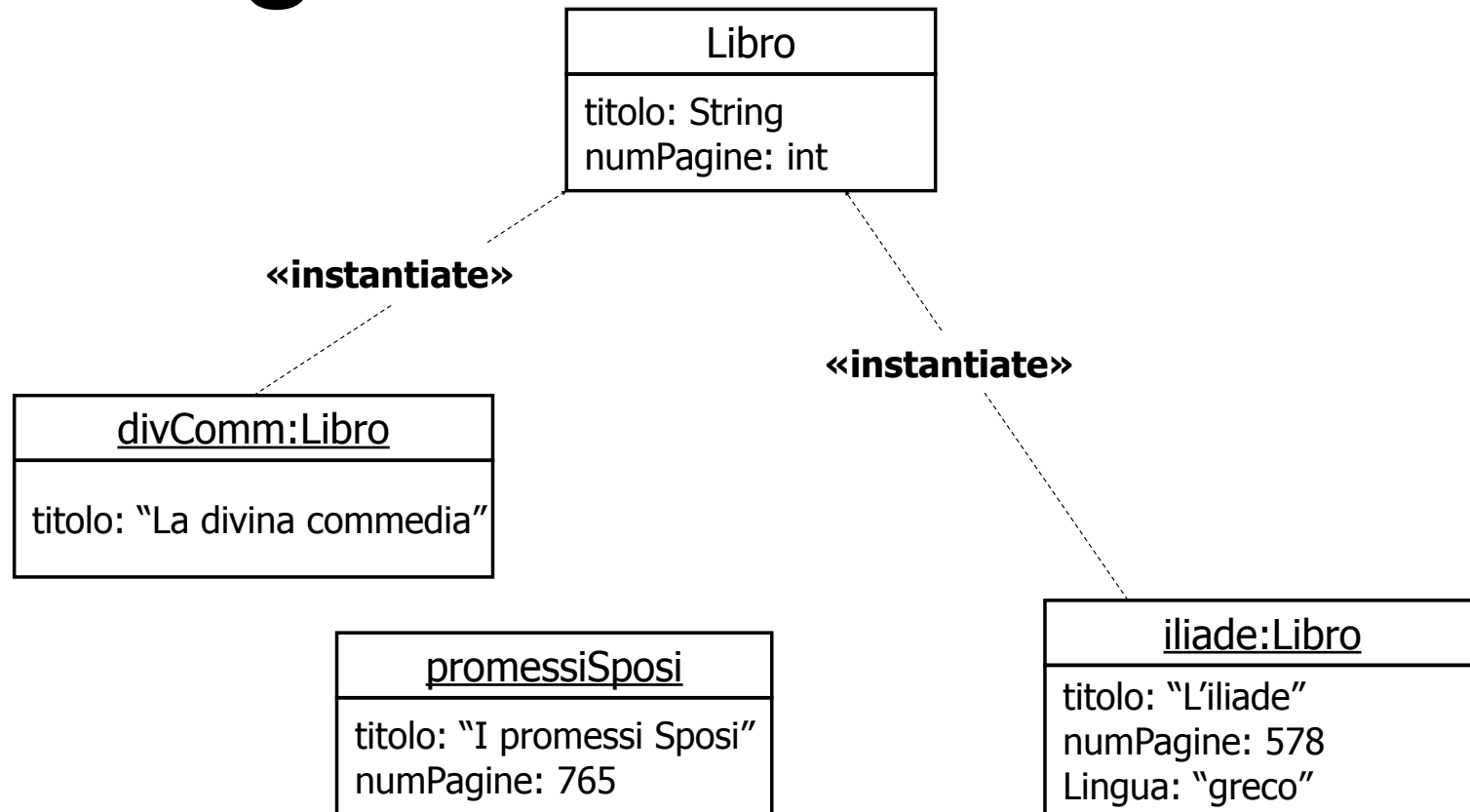


Identificatore di oggetti

- Due oggetti con identificatori distinti sono comunque distinti, anche se hanno i valori di tutti gli attributi uguali
- Due oggetti diversi devono avere identificatori diversi, anche se possono avere gli stessi valori per tutti gli attributi



Il diagramma è corretto?



Il diagramma è corretto?

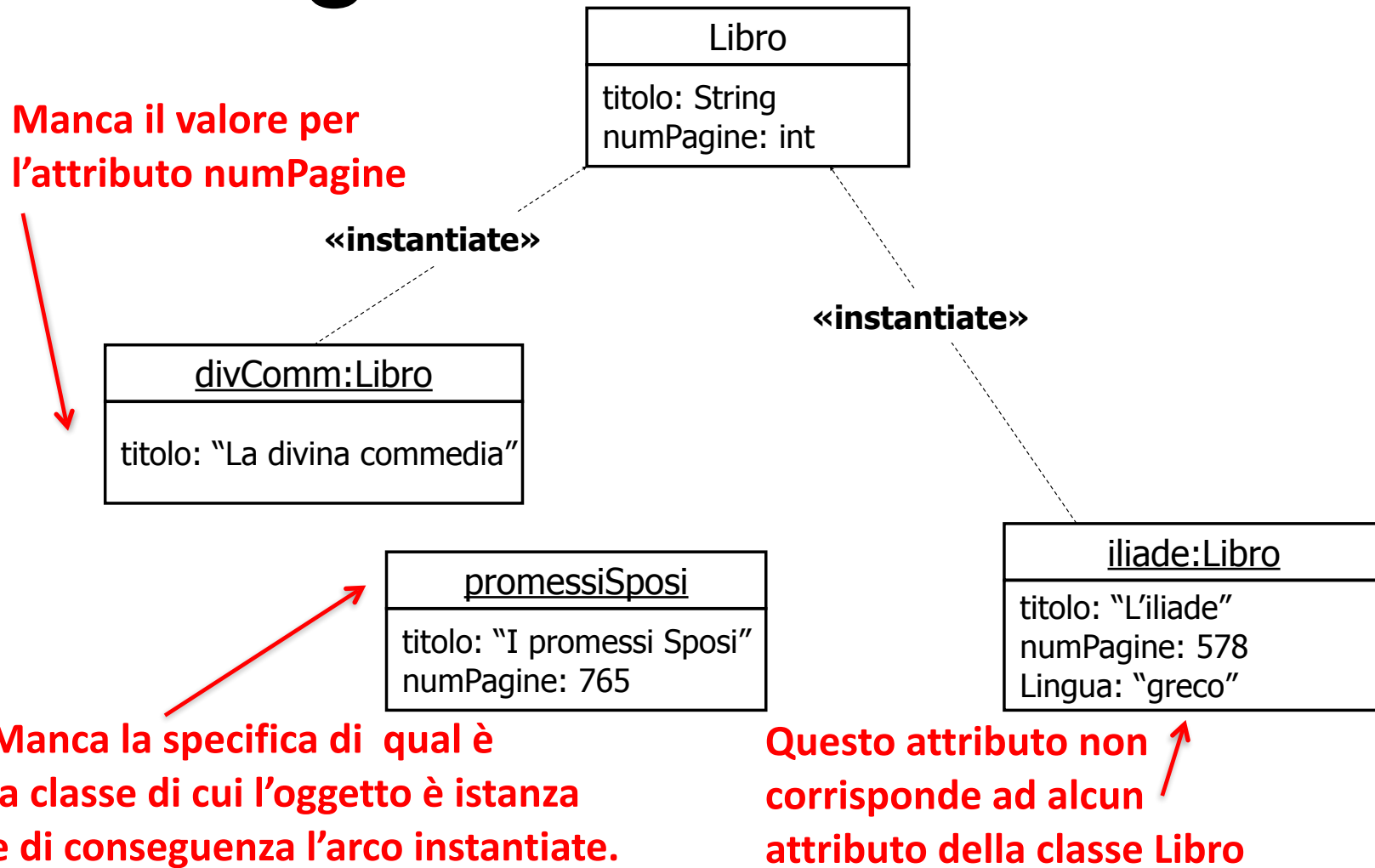
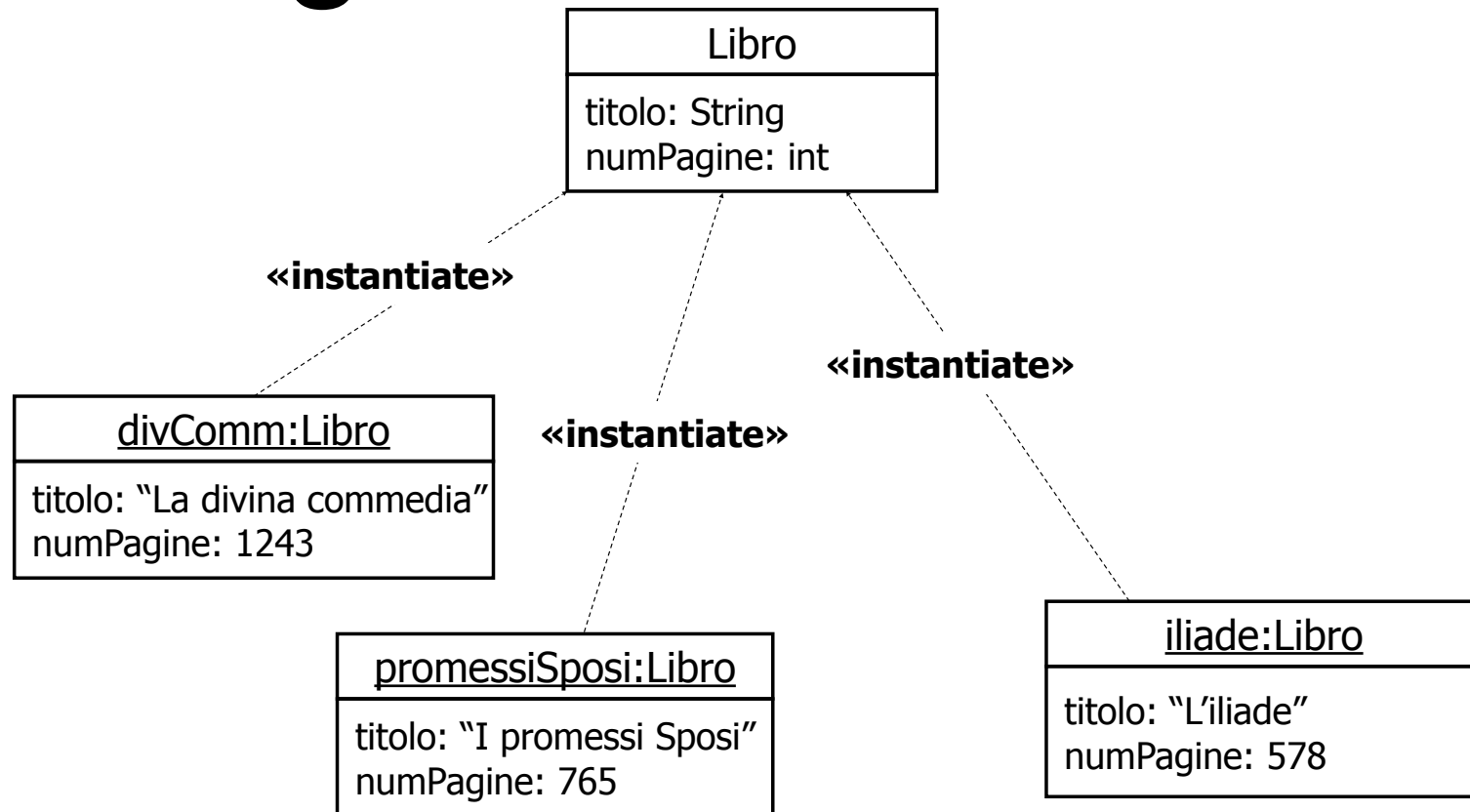


Diagramma corretto



Visibilità

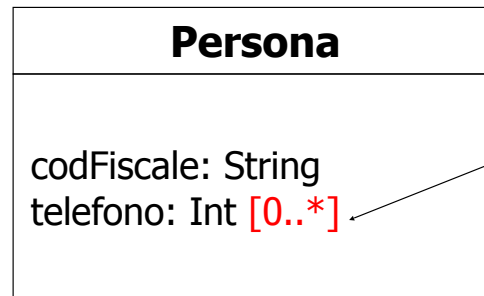
Simbolo	Nome	Semantica
+	pubblica	Ogni elemento che può accedere alla classe può anche accedere a ogni suo membro con visibilità pubblica
-	privata	Solo le operazioni della classe possono accedere ai membri con visibilità privata
#	protetta	Solo operazioni appartenenti alla classe o ai suoi discendenti possono accedere ai membri con visibilità protetta
~	package	Ogni elemento nello stesso package della classe può accedere ai membri della classe con visibilità package

PersonDetails
-name : String [2..*] -address : String [3] -emailAddress : String [0..1]

- Nella progettazione, gli attributi hanno di solito visibilità privata (incapsulamento)

Molteplicità

- Se le molteplicità di un attributo B di tipo T di una classe C non vengono indicate, vuol dire che B associa ad ogni istanza di C esattamente un valore di T (come detto prima), che è equivalente a dire che la molteplicità è **1..1**
- Al contrario, se un attributo B di tipo T di una classe C ha molteplicità **x..y**, allora B associa ad ogni istanza di C al minimo x e al massimo y valori di tipo T

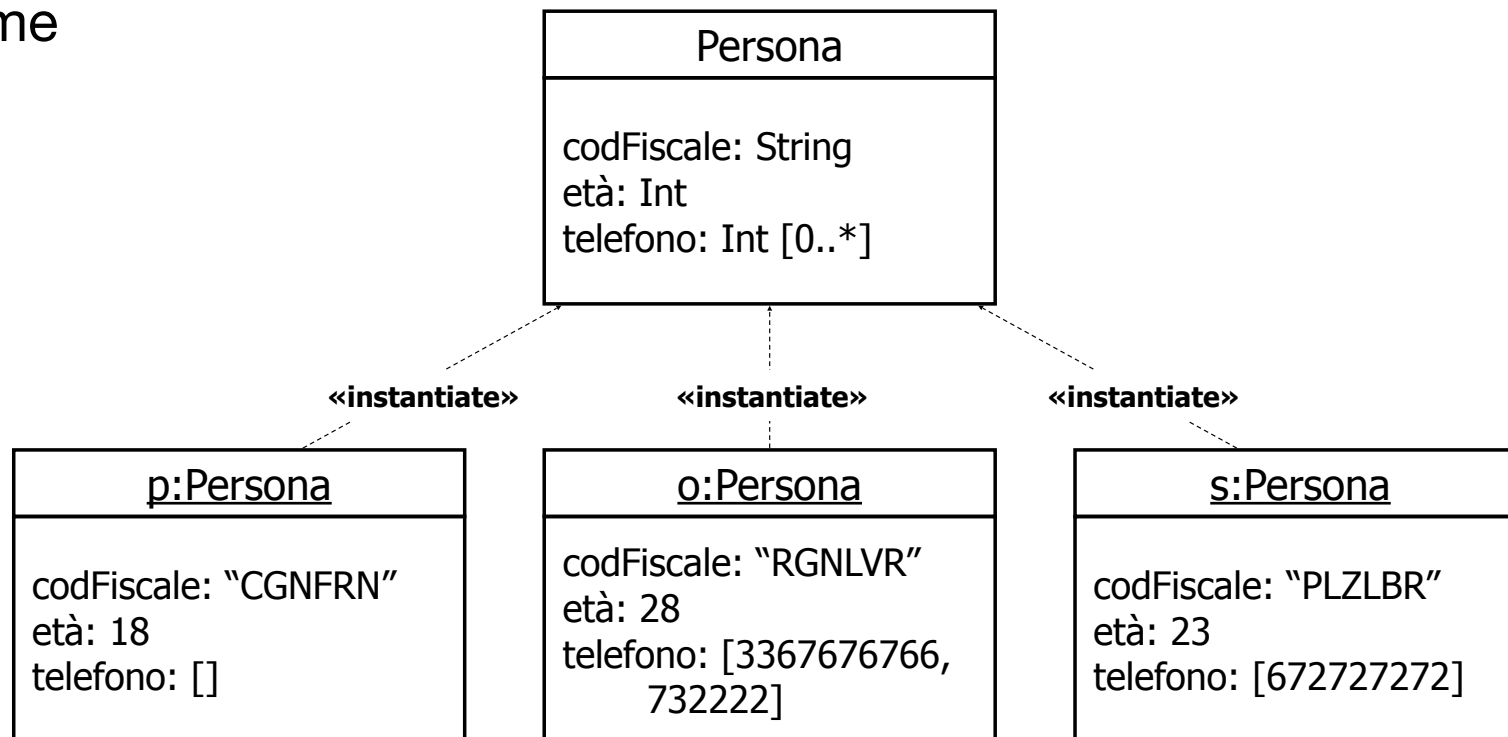


Molteplicità di attributo:
Una persona può avere un
numero qualsiasi di numeri di
telefono

Un attributo di tipo *T* della classe *C* con molteplicità diversa da **1..1** si dice **multivalore**, e formalmente non è una funzione totale, ma una relazione tra la classe *C* ed il tipo *T*

Attributi multivalore nelle istanze

- Nelle istanze, il valore di un attributo multivalore si indica mediante un insieme



Intermezzo: relazione matematica

- Se S_1 e S_2 sono due insiemi, una relazione R tra S_1 e S_2 è un sottoinsieme del prodotto cartesiano tra S_1 e S_2 :

$$R \subseteq S_1 \times S_2$$

- Il prodotto cartesiano $S_1 \times S_2$ è l'insieme di tutte le coppie $\langle x, y \rangle$ tali che $x \in S_1, y \in S_2$

- Ovviamente, la nozione si estende banalmente a relazioni tra n insiemi:

$$R \subseteq S_1 \times S_2 \times \dots \times S_n$$

Intermezzo: relazione matematica

- Consideriamo gli insiemi $S1=\{1,2,3\}$ e $S2=\{a,b\}$
- Prodotto cartesiano: $S1 \times S2 = \{<1,a>, <1,b>, <2,a>, <2,b>, <3,a>, <3,b>\}$
- Esempio di relazione tra $S1$ e $S2$: $R=\{<1,a>, <2,a>, <3,a>\}$

Si noti come una relazione R selezioni un sottoinsieme degli elementi del prodotto cartesiano, quelli che sono significativi rispetto al significato della relazione R

Intermezzo: relazione matematica

- Consideriamo gli insiemi

studente = {Paolo, Anna, Lucia}

esame = {Analisi, Geometria}

- Esempio di relazione “sostenuto” tra “studente” ed “esame”:

sostenuto = {<Paolo,Analisi>, <Anna,Analisi>, <Anna,Geometria>}

Si noti come, tra tutte le coppie del prodotto cartesiano tra “studente” ed “esame”, la relazione “sostenuto” seleziona un insieme di coppie, e cioè solo le coppie <x,y> tali che lo studente x ha sostenuto l'esame y

Associazioni in UML

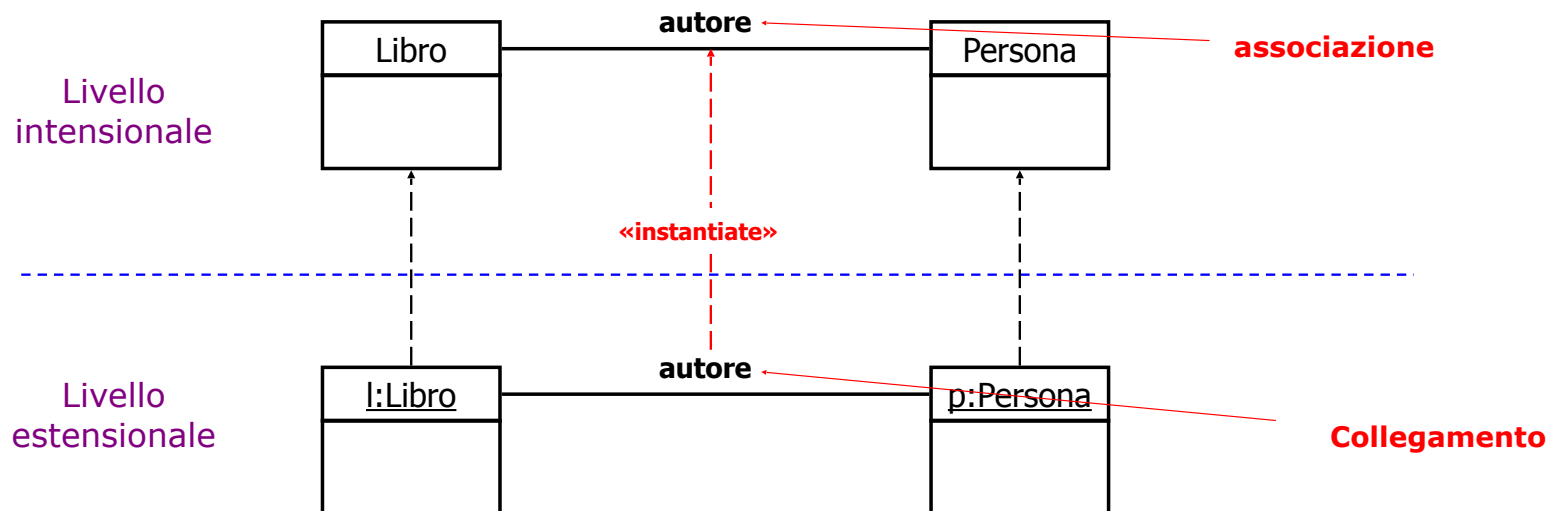
- Per il momento, ci limitiamo a discutere associazioni tra due classi (ma le associazioni possono coinvolgere N classi)
- Una associazione tra una classe C1 ed una classe C2 modella una relazione matematica tra l'insieme delle istanze di C1 e l'insieme delle istanze di C2
- Gli attributi modellano proprietà locali di una classe, le associazioni modellano proprietà che coinvolgono altre classi.



Collegamenti: Istanze delle associazioni

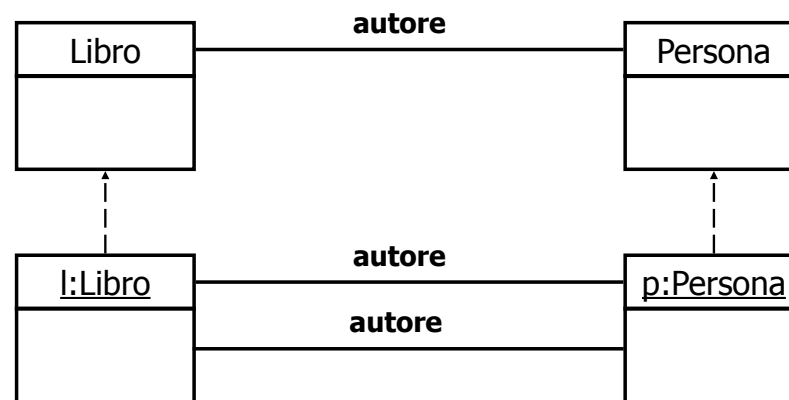
Le istanze di associazioni si chiamano collegamenti:

- se A è una associazione tra le classi C1 e C2, una istanza di A è un collegamento tra due oggetti (in altre parole, una coppia), uno della classe C1 e l'altro della classe C2



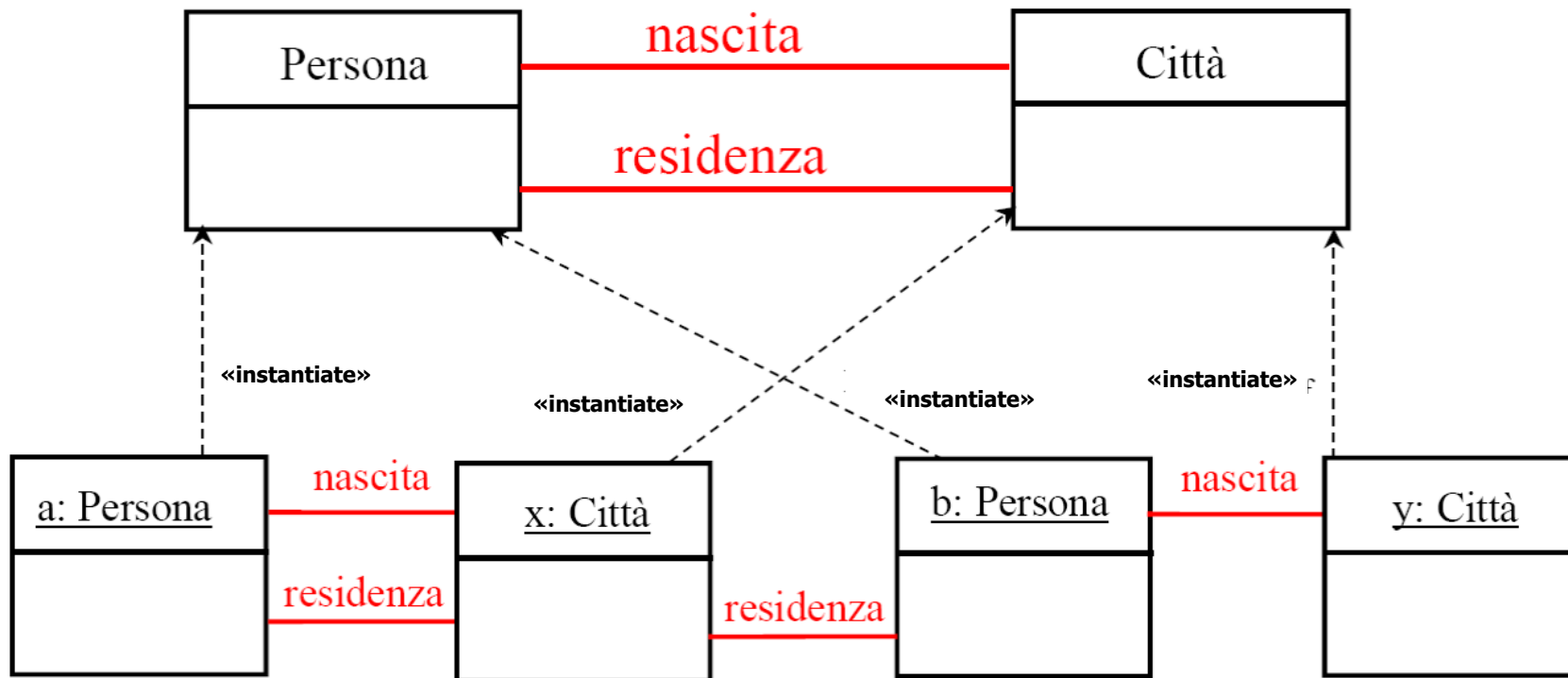
Collegamenti: Istanze delle associazioni

- Come gli oggetti sono istanze delle classi, così i collegamenti sono istanze delle associazioni (gli archi istantiate non sono necessari)
- Al contrario degli oggetti, però, i collegamenti non hanno identificatori espliciti: **un collegamento è implicitamente identificato dalla coppia (o in generale dalla ennupla) di oggetti che esso rappresenta**
- Ciò implica, ad esempio, che il seguente diagramma è errato:



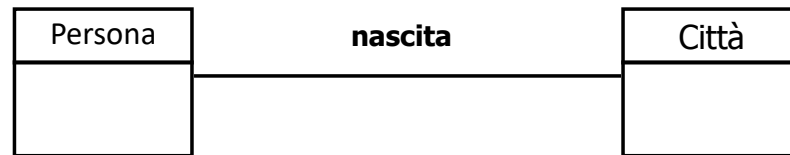
Più associazioni tra due classi

Ovviamente, tra le stesse due classi possono essere definite più associazioni



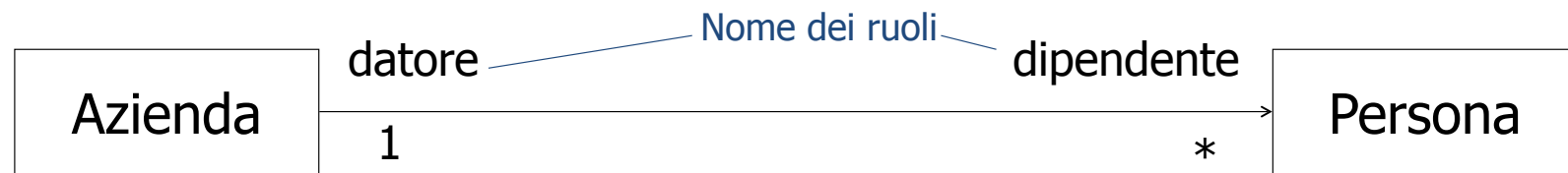
Osservazioni sull'associazioni

- Attenzione: una relazione R tra $C1$ e $C2$ non dice nulla sul numero di link di R che coinvolgono due istanze delle classi $C1$ e $C2$. Ad esempio, dato questo diagramma:



- Una istanza di Persona può essere legata a zero, una, o più istanze di Città da link di tipo “nascita”

Sintassi dell'associazione



- Le associazioni possono essere indicate o con il nome dell'associazione, oppure con il nome dei ruoli
- L'associazione dell'esempio si legge: "un'Azienda impiega molte persone"

Nome dell'associazione

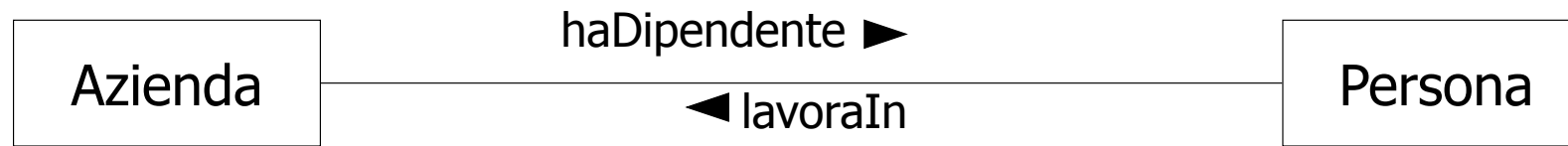
- Alcune volte è interessante specificare un **verso** per il nome della associazione
- Attenzione: la notazione riportata sopra **non significa che l'associazione ha un verso**



- In altre parole, il verso **non è una caratteristica del significato della associazione**, ma dice semplicemente che il nome scelto per la associazione evoca un verso (nell'esempio, il verso è dalla Azienda alla Persona)

Nome dell'associazione

- È anche possibile assegnare due nomi con i relativi versi alla stessa associazione



- Osserviamo ancora che le frecce che simboleggiano il verso non aggiungono nulla al significato della associazione (che formalmente si può considerare sempre una relazione matematica), ma aiutano ad interpretare il senso dei nomi scelti per l'associazione
- Le frecce che simboleggiano il verso si indicano anche nel link che sono istanze delle associazioni

Ruoli nell'associazione

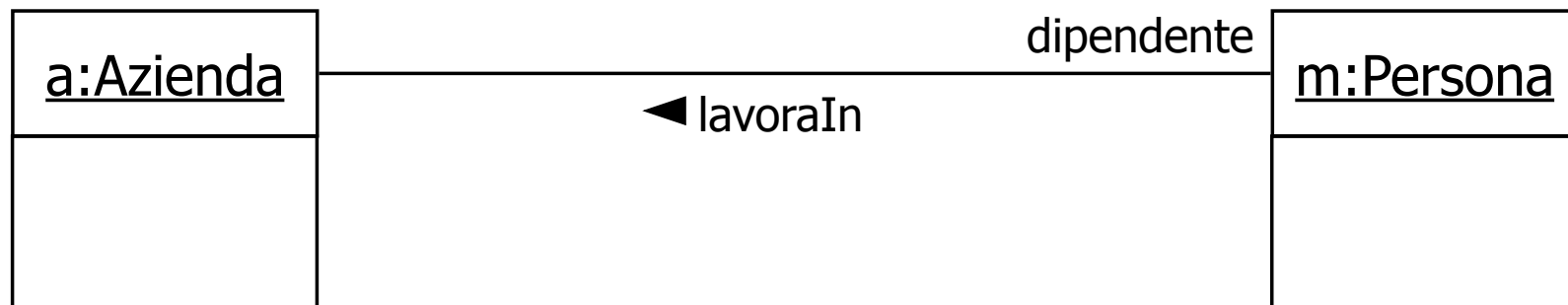
- È possibile aggiungere alla associazione una informazione che specifica il ruolo che una classe gioca nella associazione



- Il ruolo si indica con un nome posizionato lungo la linea che rappresenta l'associazione, vicino alla classe alla quale si riferisce
- Nell'esempio, dipendente è il ruolo che la persona gioca nell'associazione "lavoraIn" con Azienda

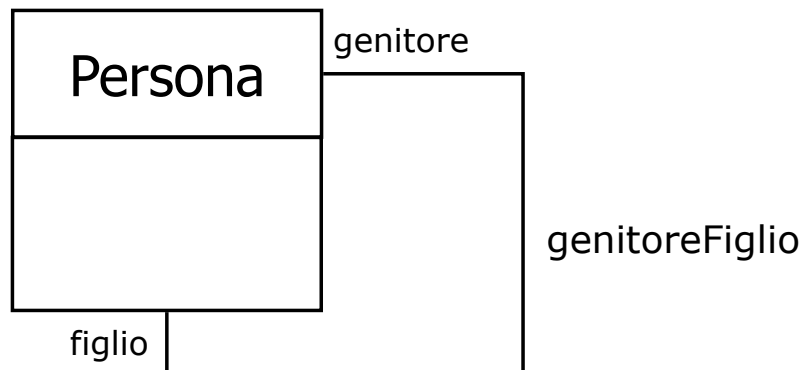
Ruoli nei collegamenti

- Se nella associazione A è indicato il ruolo giocato dalla classe C, tale ruolo sarà indicato (vicino alla corrispondente istanza di C) in ogni link che è istanza di A
- Esempio:



Osservazione sui ruoli

- Analogamente al verso, il ruolo non aggiunge nulla al significato dell'associazione
- L'unico caso in cui il ruolo è obbligatorio è quello in cui l'associazione insiste più volte sulla stessa classe, e rappresenta una relazione non simmetrica
- Esempio:



Importanza dei ruoli

- Senza ruoli non è chiaro chi è il genitore e chi il figlio. In altre parole, dalla coppia $\langle x, y \rangle$ in `genitoreFiglio` non si evincono i ruoli di x e y



- Con i ruoli, non c'è più ambiguità. Formalmente, il link è ora una coppia etichettata:

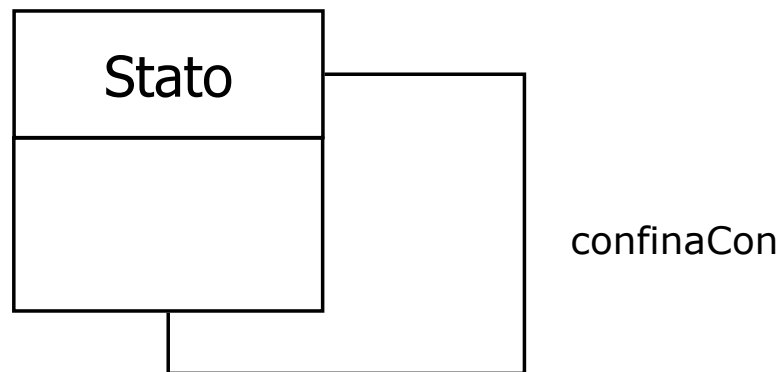
$\langle \text{genitore}:x, \text{figlio}:y \rangle$

dalla quale si evincono i ruoli di x e y



Casi in cui i ruoli non sono significativi

- Ci sono casi in cui l'associazione insiste più volte sulla stessa classe, ma il ruolo non è significativo, in particolare quando l'associazione rappresenta una relazione simmetrica



Navigabilità

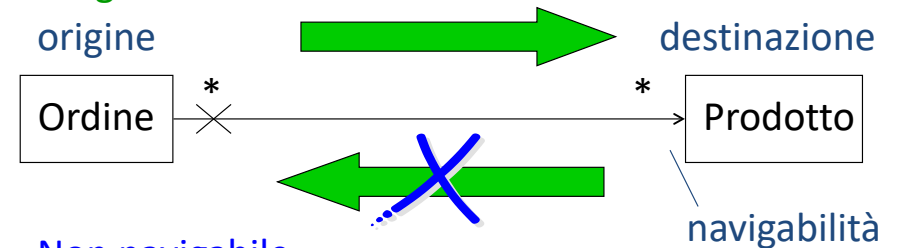
La navigabilità indica che è possibile spostarsi da un qualsiasi oggetto della classe origine a uno o più oggetti della classe destinazione

- Gli oggetti della classe di origine possono fare riferimento a oggetti della classe di destinazione utilizzando il nome del ruolo

Pratica Standard

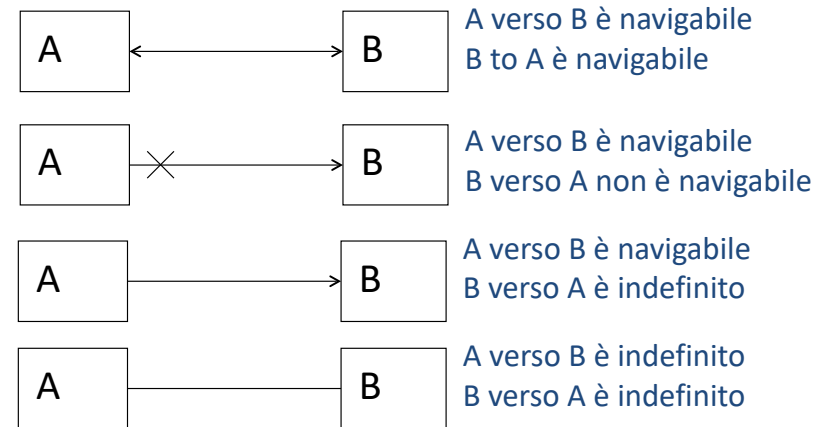
Un oggetto Ordine memorizza una lista di Prodotti

Navigabile

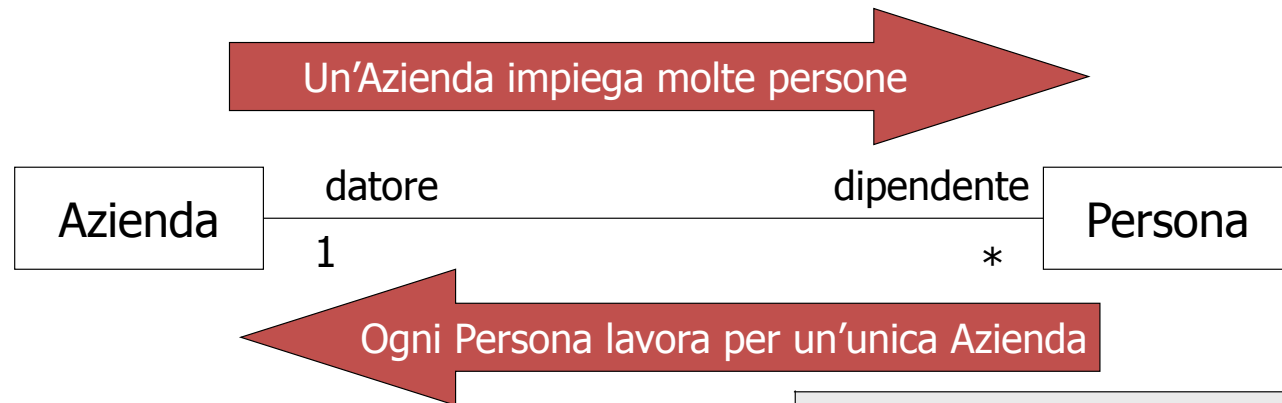


Non navigabile

Un oggetto Prodotto non memorizza una lista di Ordini



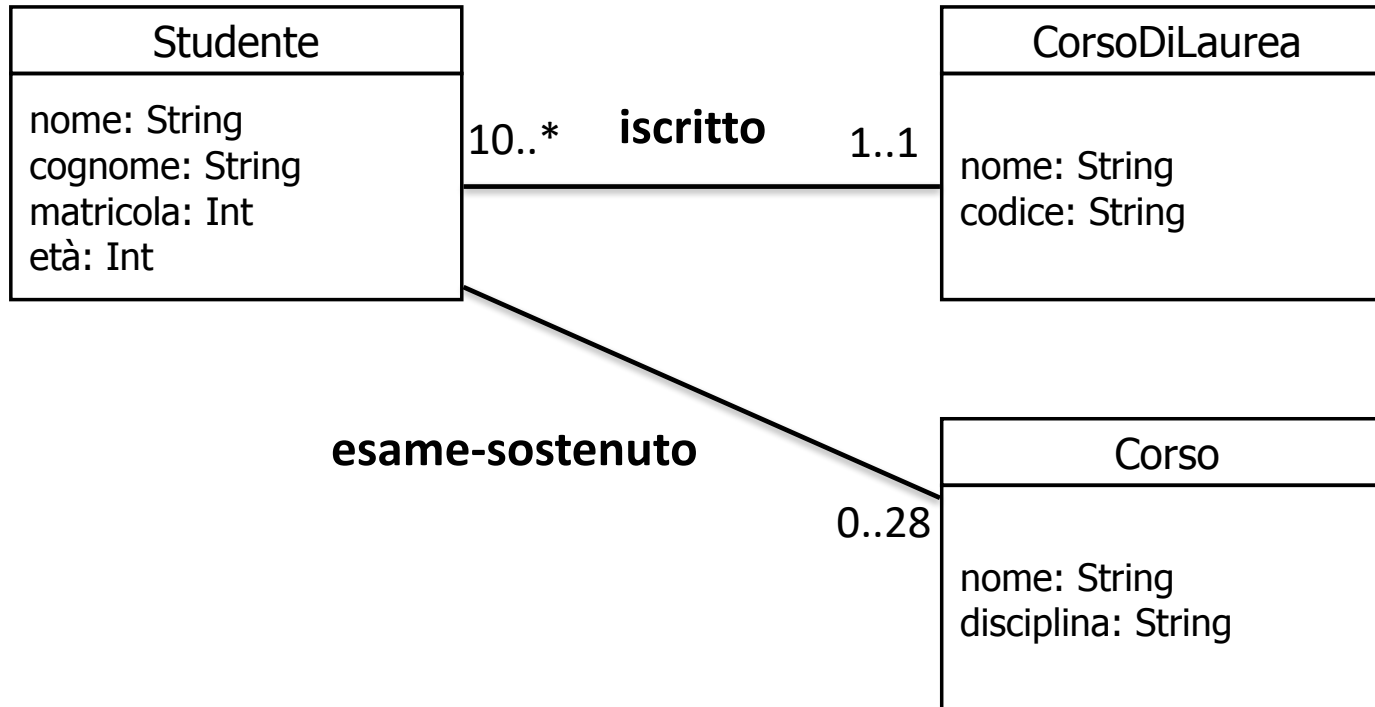
Molteplicità



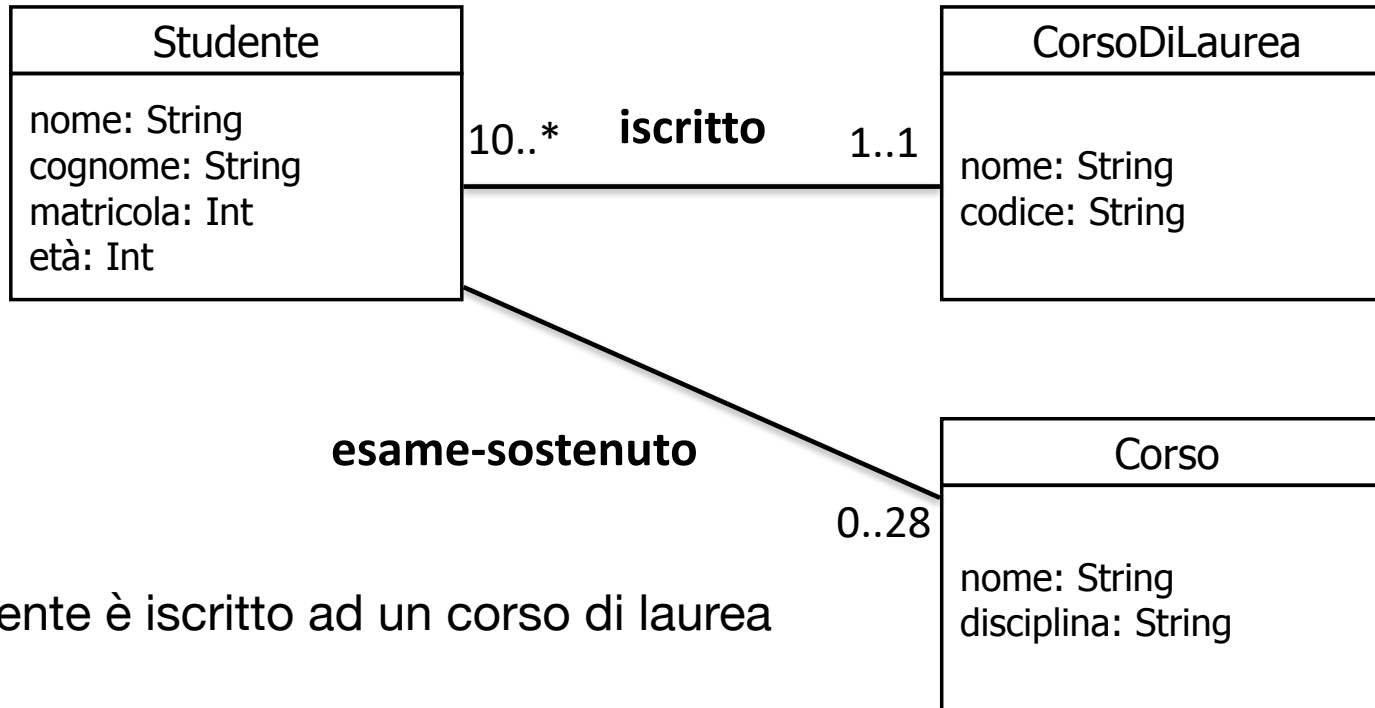
- La molteplicità limita il numero di oggetti di una classe che possono partecipare in una relazione in un dato istante
- Se la molteplicità non viene indicata esplicitamente, allora è indefinita

sintassi: minimo..massimo	
0..1	Zero o 1
1	esattamente 1
0..*	zero o più
*	zero o più
1..*	1 o più
1..6	Da 1 a 6
1..3,7..10,15	Da 1 a 3, o da 7 a 10, o esattamente 15

Leggere il diagramma

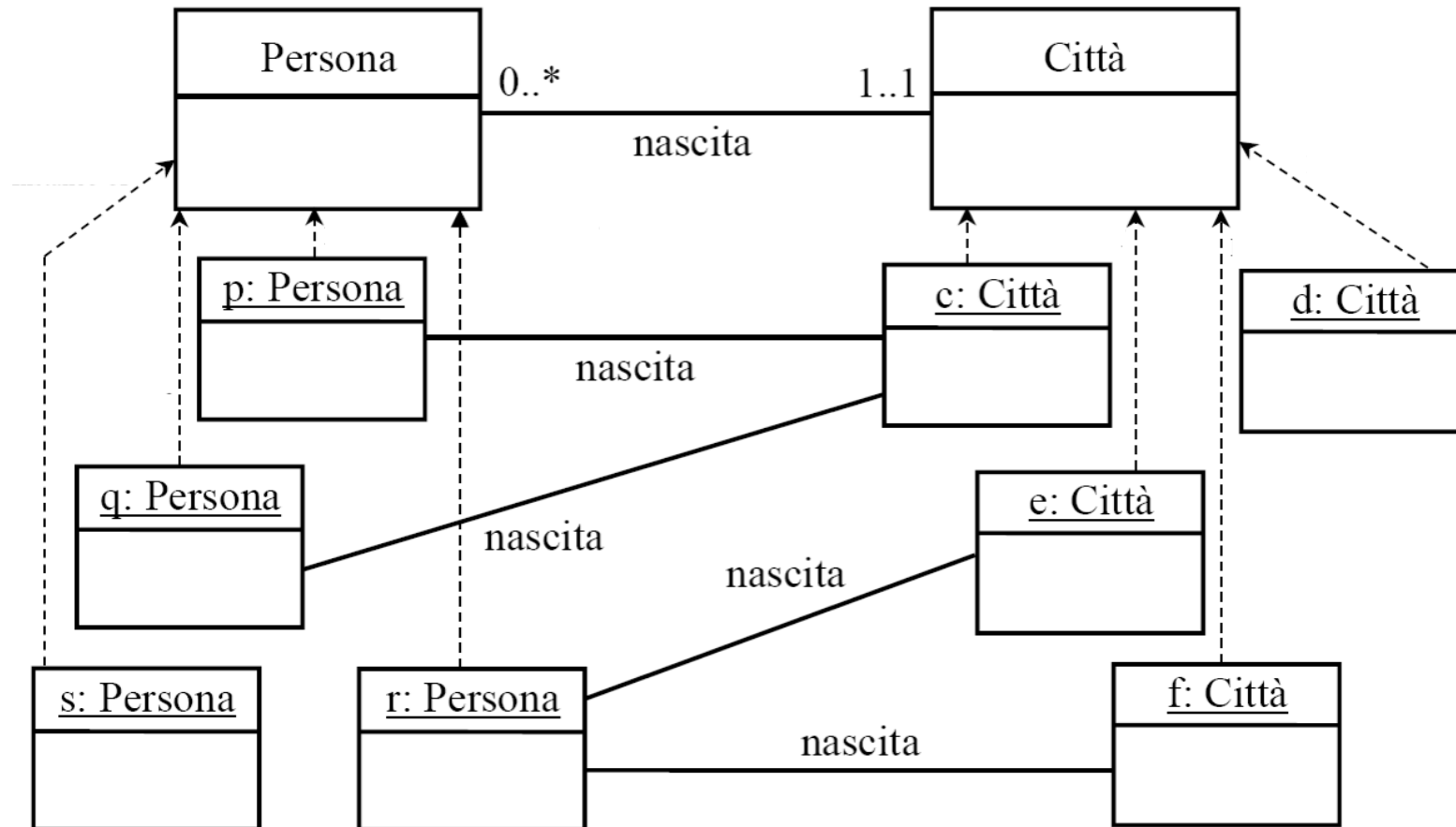


Leggere il diagramma

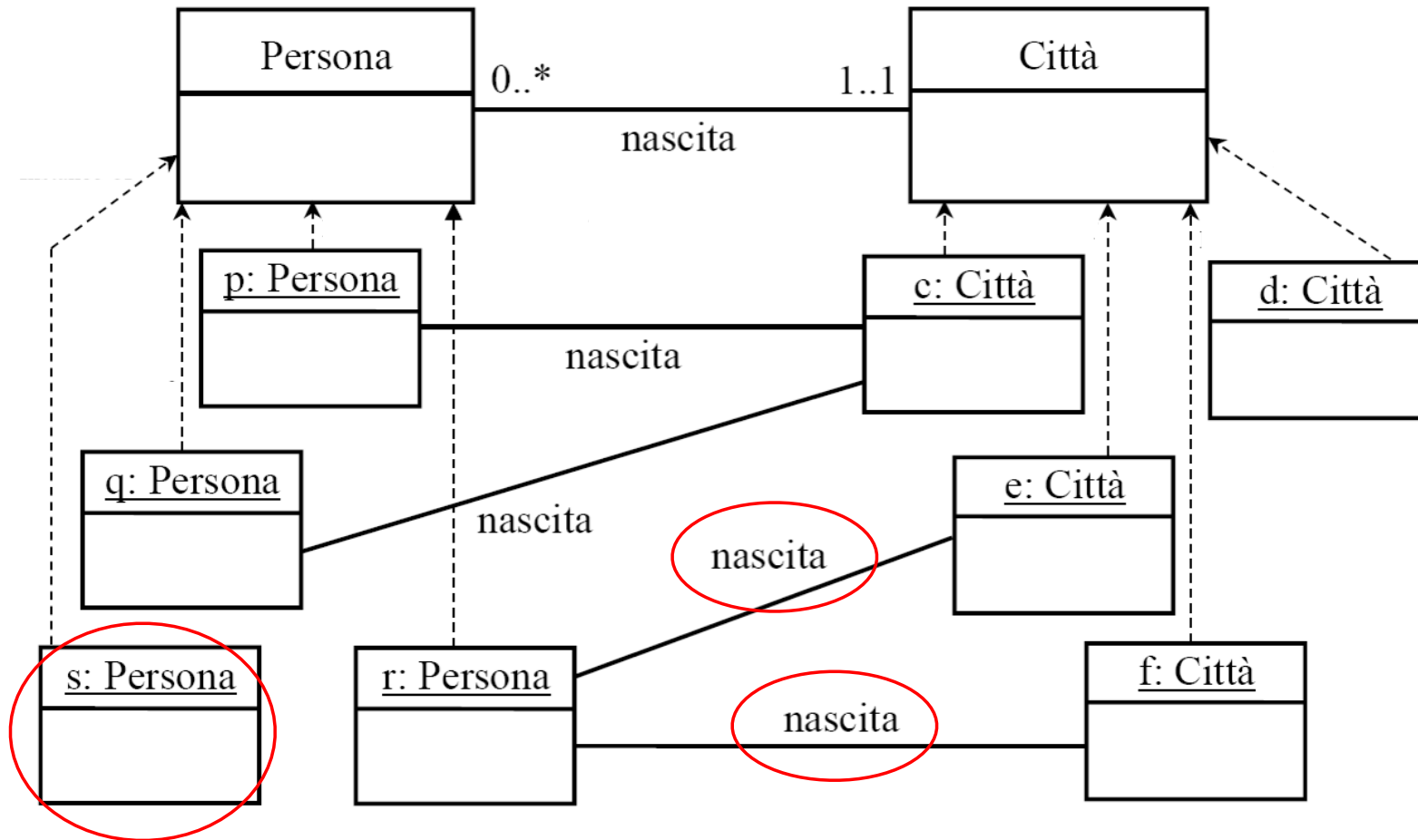


- Ogni studente è iscritto ad un corso di laurea
- Ogni corso di laurea ha almeno 10 iscritti
- Ogni studente può sostenere al massimo 28 esami

Individuare gli errori

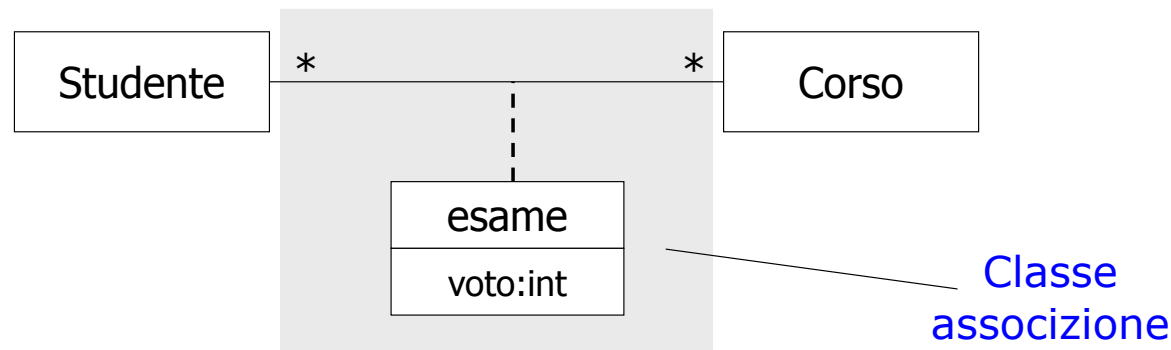


Individuare gli errori



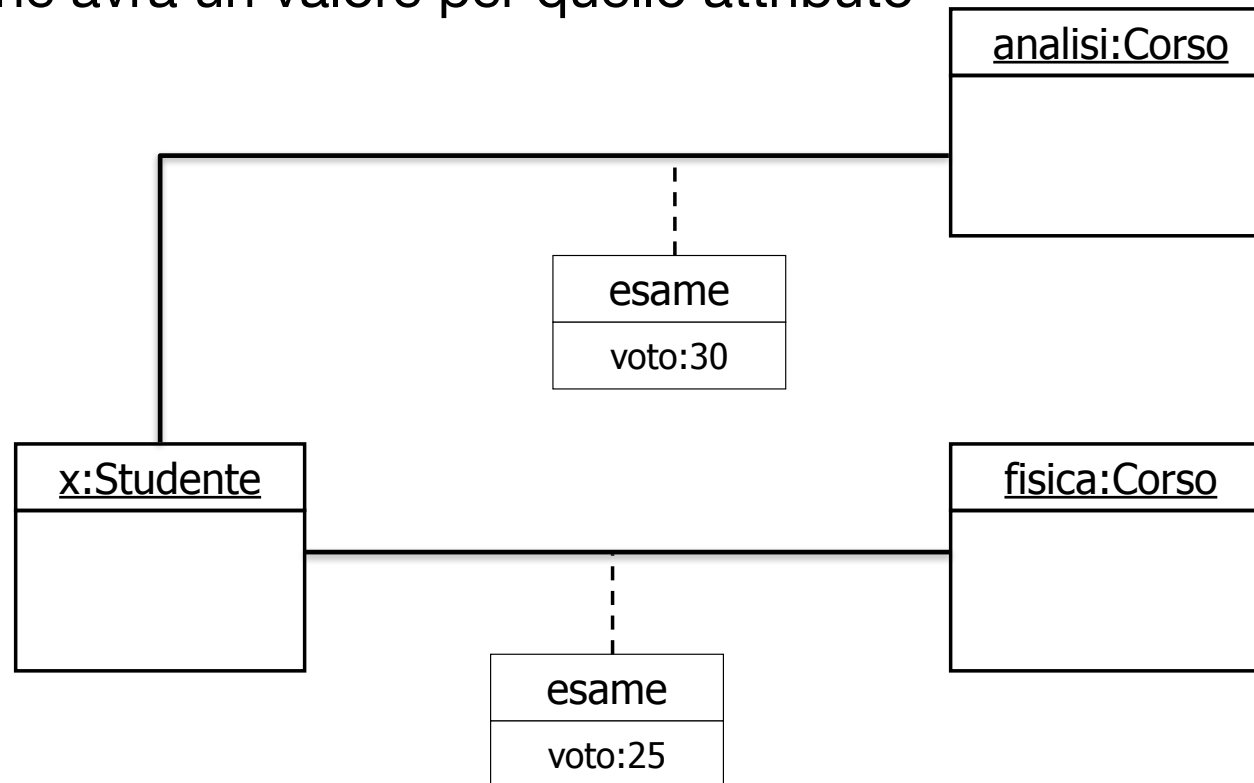
Attributi di associazioni

- Analogamente alle classi, anche le associazioni possono avere attributi. Formalmente, un attributo di una associazione è una funzione che associa ad ogni link che è istanza dell'associazione un valore di un determinato tipo
- Voto non è una proprietà né di Studente, né di Corso, ma della associazione Esame tra Studente e Corso



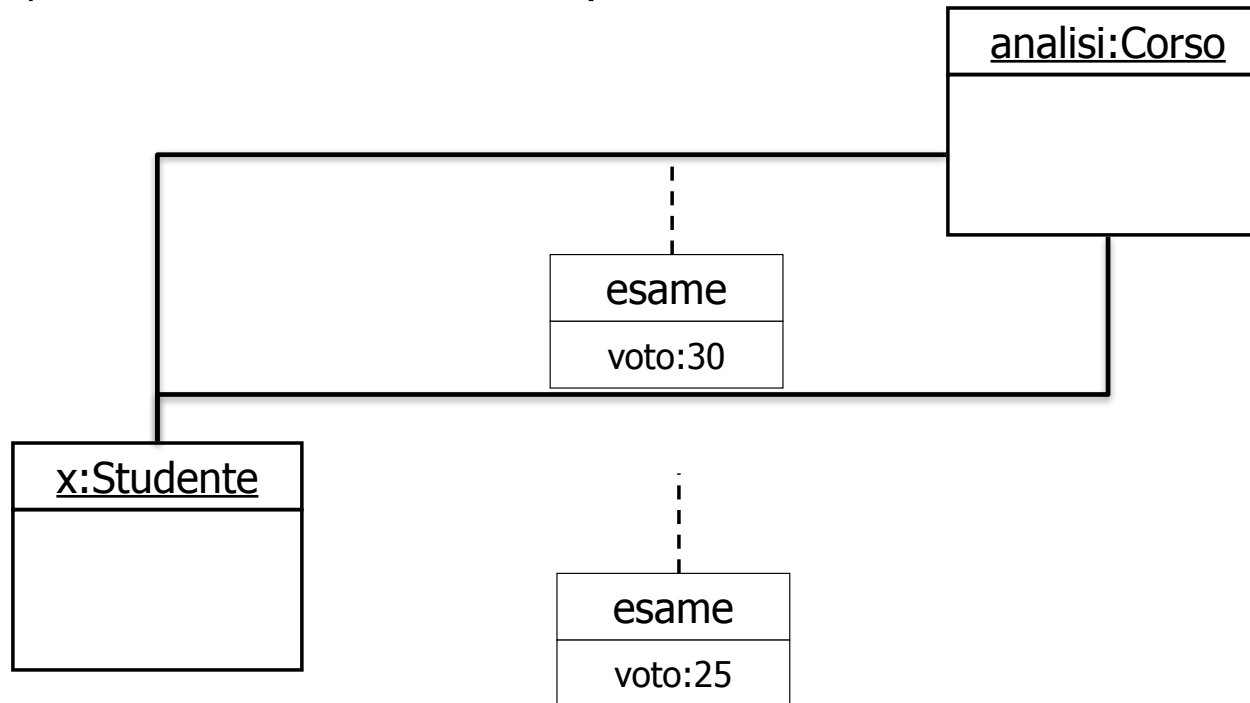
Attributi nei collegamenti

- Ovviamente, se una associazione ha un attributo, ogni link che è istanza dell'associazione avrà un valore per quello attributo



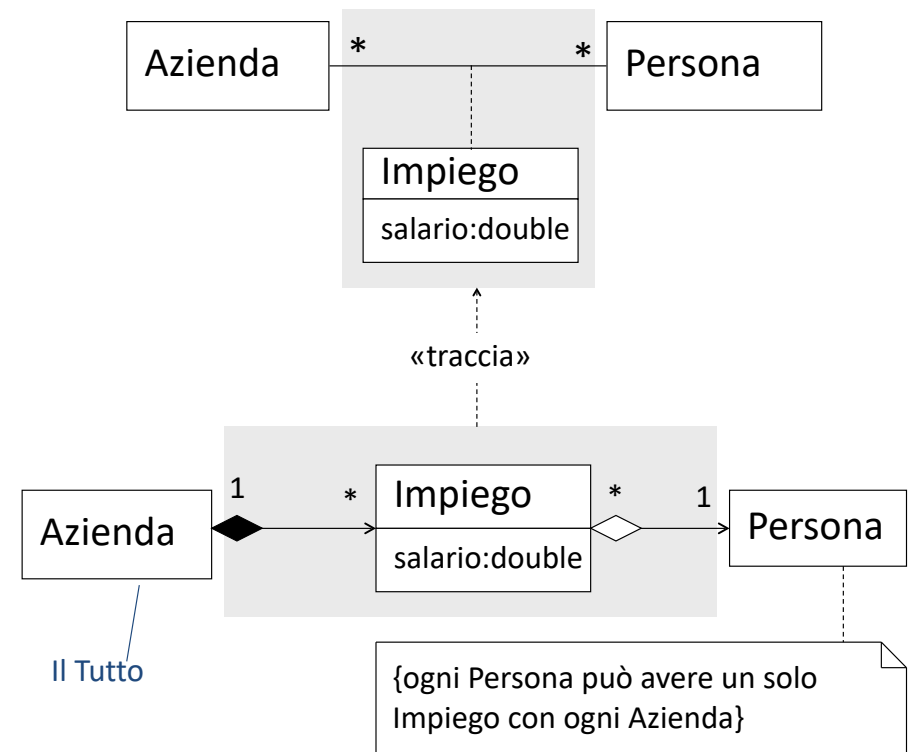
Attributi nei collegamenti

- Con la presenza degli attributi, il significato dell'associazione (relazione matematica) non cambia. Quindi questo è un errore:



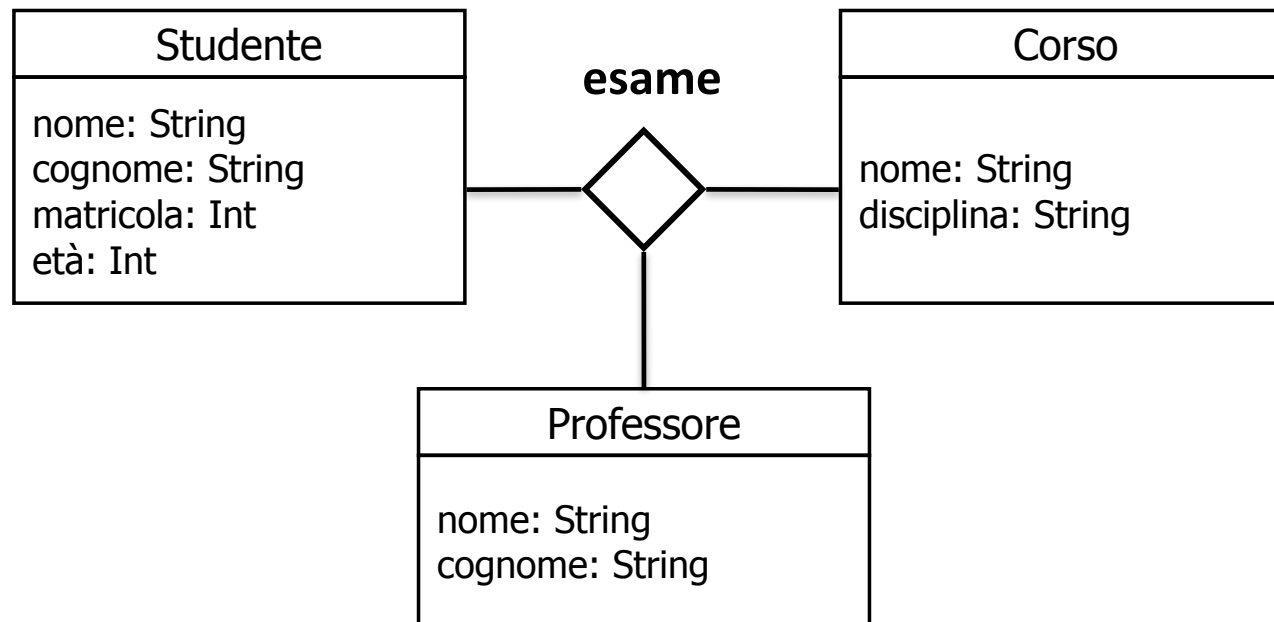
Classi associazioni nella progettazione

- Nessuno dei linguaggi OO comunemente utilizzati supporta le classi associazione
- Le classi associazioni vengono reificate tramite l'uso di classi di progettazione
- E' necessario innanzitutto decidere quali delle due classi coinvolte sia il Tutto e quale sia la Parte, quindi si dovrà utilizzare l'aggregazione e la composizione in modo opportuno



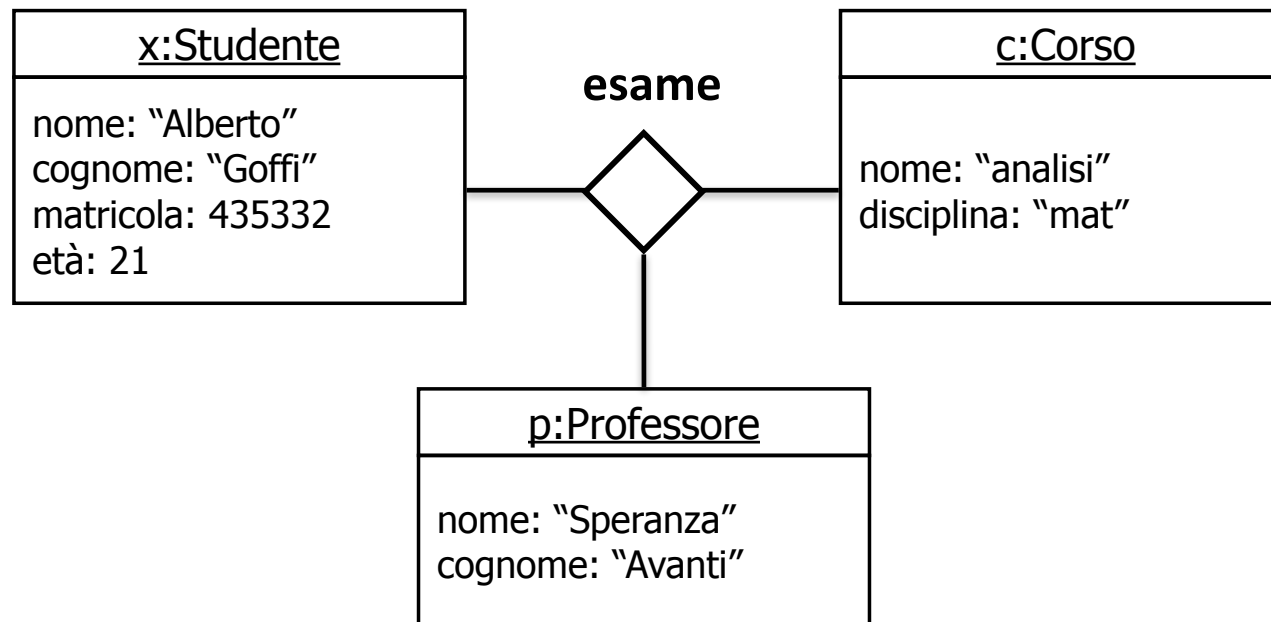
Associazioni n-arie

Una associazione può essere definita su tre o più classi. In tale caso l'associazione si dice **n-aria**, e modella una **relazione matematica tra n insiemi**



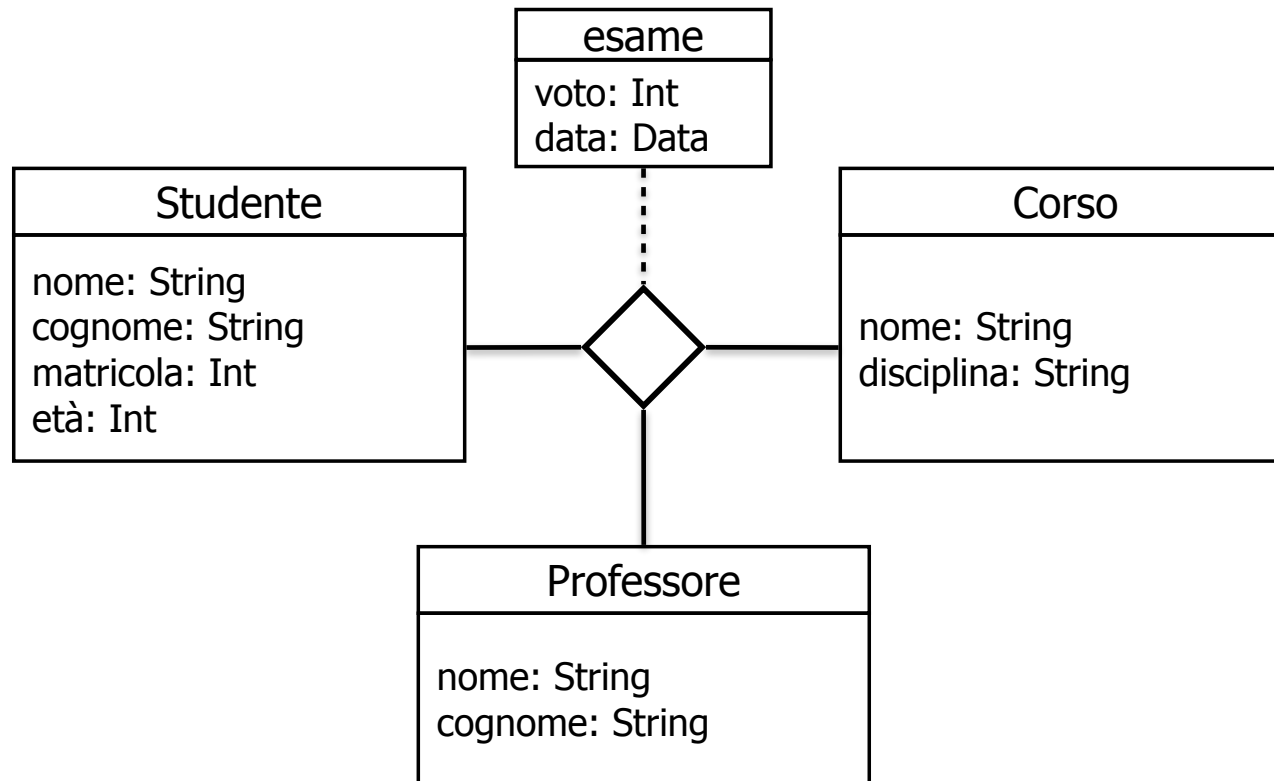
Istanze di associazioni n-arie: collegamenti n-ari

Ogni istanza di una associazione n-aria è un link n-ario, cioè che coinvolge n oggetti

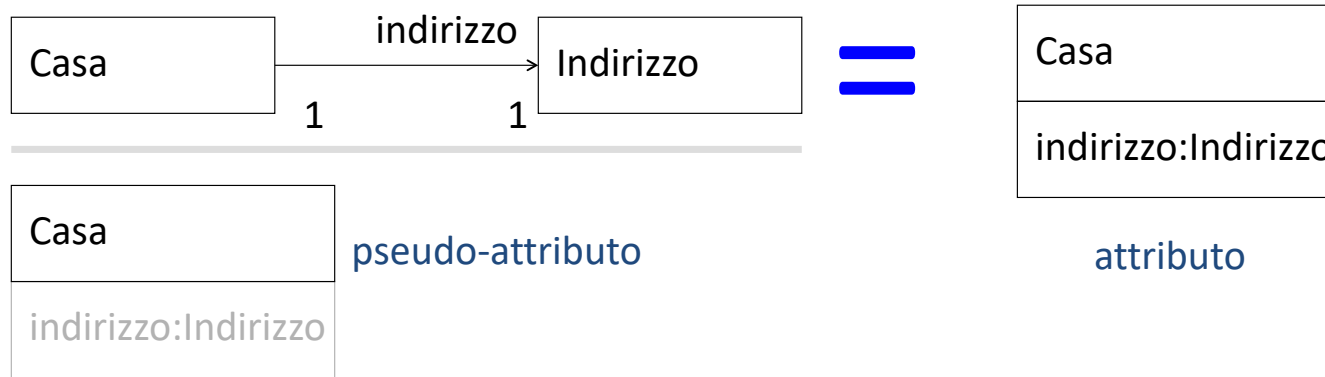


Associazioni n-arie con attributi

Ovviamente, anche le associazioni n-arie possono avere attributi



Associazioni e Attributi

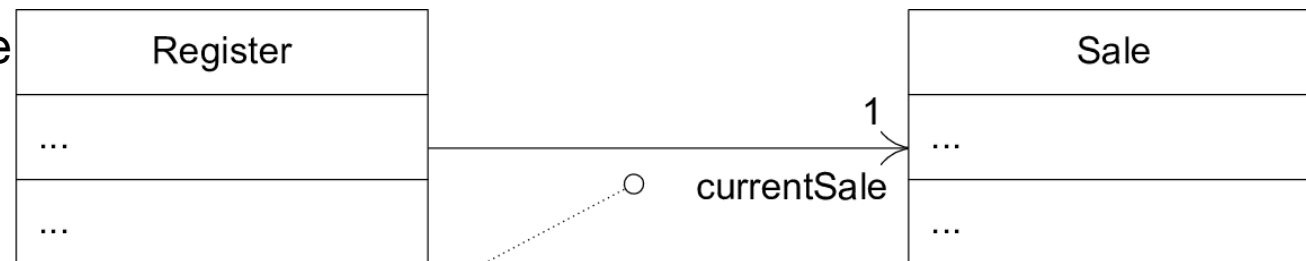


- Se un'associazione navigabile ha un nome di ruolo, è come se la classe di origine ha una pseudo-attributo il cui nome è il nome del ruolo e il cui tipo è la classe di destinazione
- Utilizzare le associazioni quando:
 - *La classe di destinazione è una parte importante del modello*
- Utilizzare gli attributi quando:
 - *La classe di destinazione non è una parte importante del modello ad esempio un tipo primitivo, come numero, stringa, ecc*
 - *La classe di destinazione è solo un dettaglio di implementazione, ad esempio una componente di libreria `Java.util.Vector` (dalle librerie standard di Java)*

Notazione per le estremità di associazioni

- freccia di navigabilità: dalla classe sorgente alla classe destinazione
- molteplicità: vicino alla destinazione
- nome di ruolo: solo all'estremità vicino alla destinazione

- Nessun nome per l'associazione

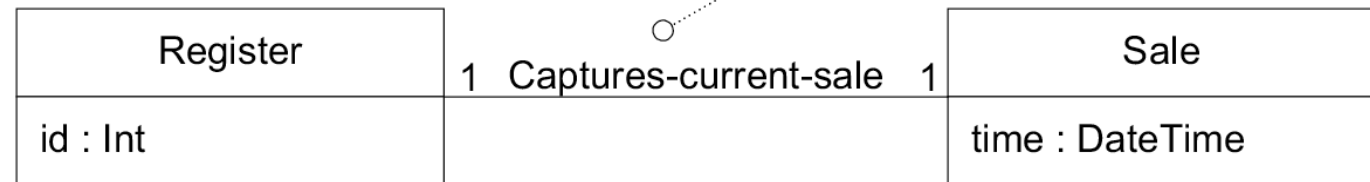


uso di un'associazione
(estremità di associazione) per
indicare che Register ha un
riferimento a un'istanza di Sale

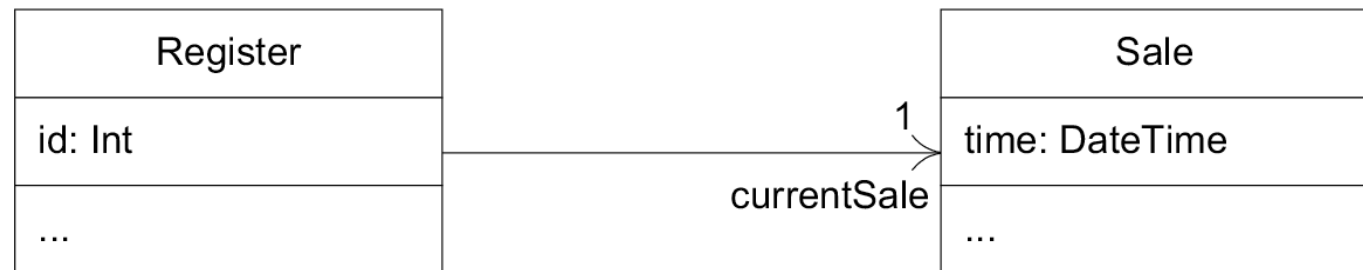
Idiomi nell'uso della notazione per le associazioni nei diversi punti di vista

il *nome* dell'associazione, normale quando si disegna un modello di dominio, è spesso escluso (anche se legale) quando si utilizzano i diagrammi delle classi da un punto di vista software in un DCD

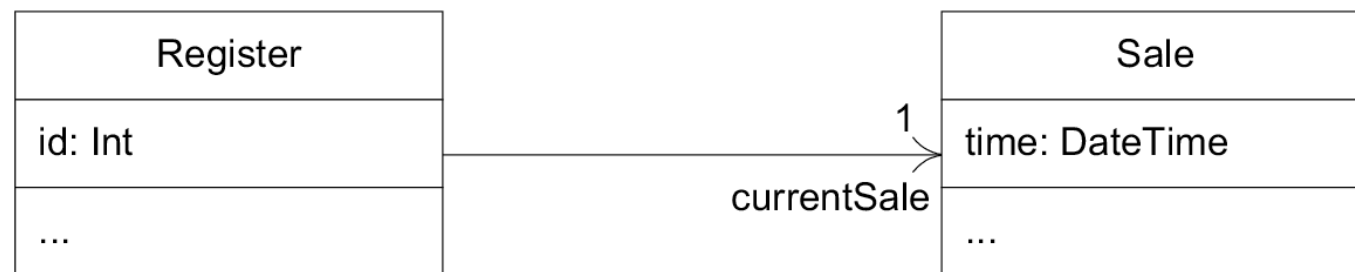
Punto di vista
concettuale
del Modello
di Dominio di UP



Punto di vista
software del
DCD del Modello
di Progetto di UP



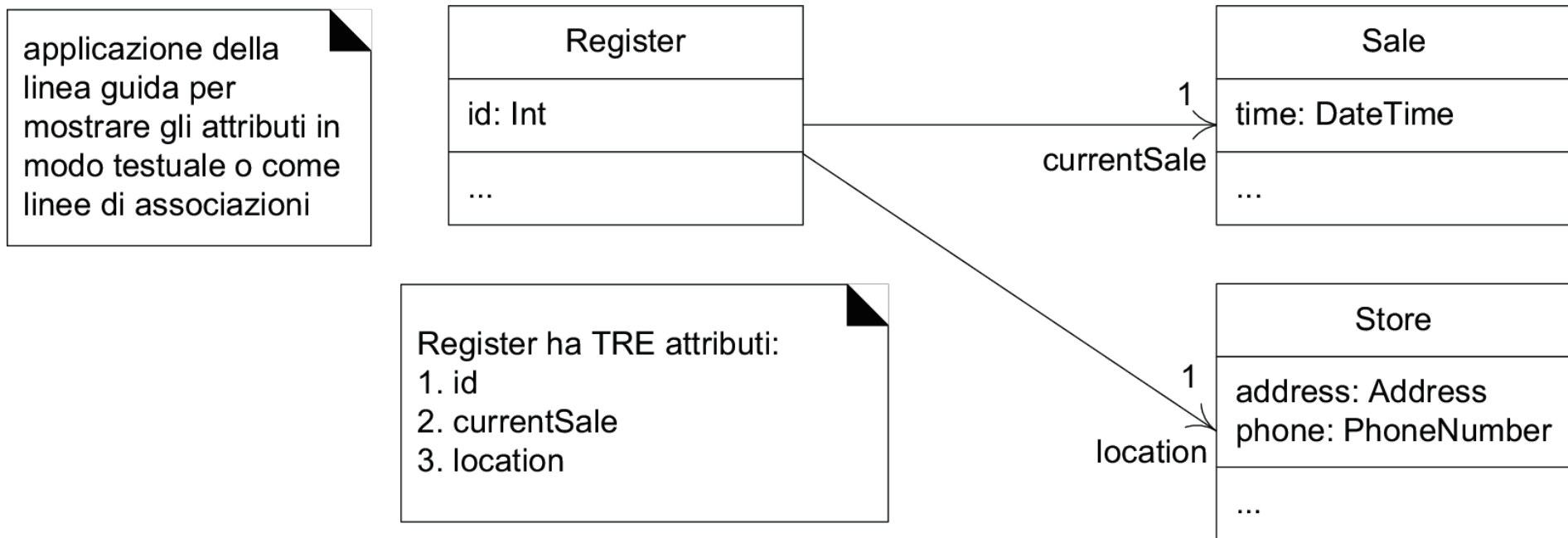
Idiomi nell'uso della notazione per le associazioni nei diversi punti di vista



```
public class Register {  
    private Sale currentSale;  
  
    ...  
}
```

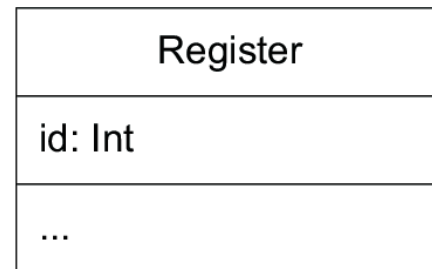
Quando usare per gli attributi la notazione testuale oppure le linee di associazione

- Si utilizzi la notazione testuale per gli attributi il cui tipo è un tipo di dato (insieme di valori in cui l'identità univoca non è significativa)
- Linee di associazioni per gli altri attributi

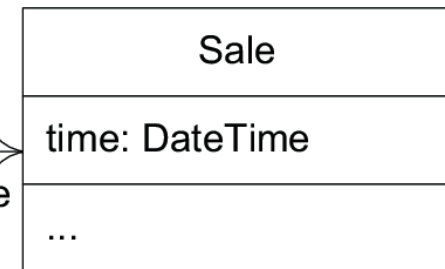


Quando usare per gli attributi la notazione testuale oppure le linee di associazione

applicazione della
linea guida per
mostrare gli attributi in
modo testuale o come
linee di associazioni

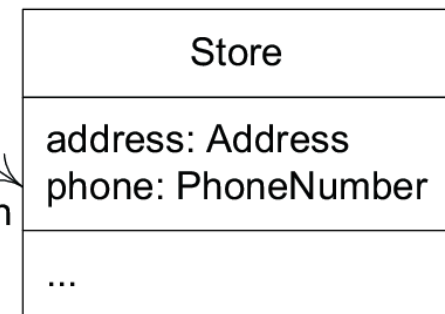


1
currentSale



Register ha TRE attributi:
1. id
2. currentSale
3. location

1
location



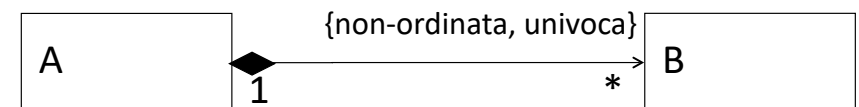
```
public class Register {  
    private int id;  
    private Sale currentSale;  
    private Store location;  
    ...  
}
```

Semantica della collezione: Stringhe di proprietà

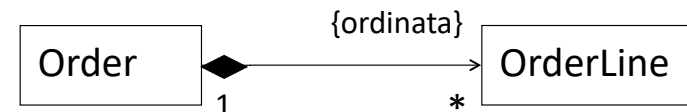
UML mette a disposizione delle proprietà standard che possono essere applicate alle molteplicità per indicare la semantica richiesta dalla collezione:

proprietà	Semantica
{ordered}	Gli elementi nella collezione vengono tenuti in ordine rigoroso
{unordered}	Non esiste alcun ordine degli elementi della collezione
{unique}	Gli elementi dell'insieme sono tutti univoci
{nonunique}	La collezione ammette duplicati

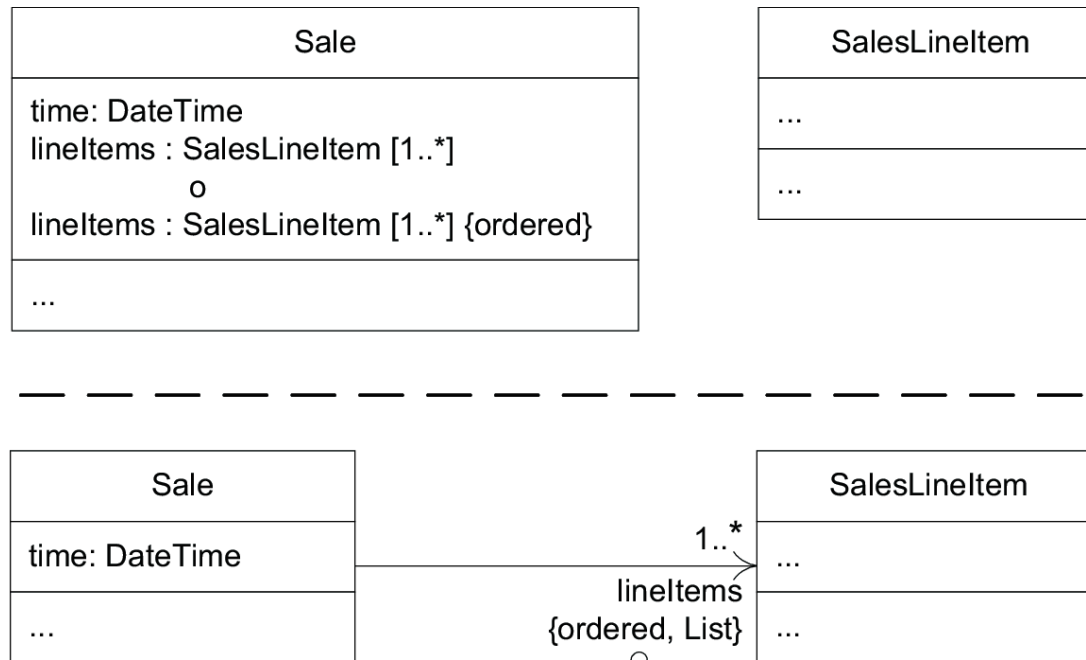
La semantica predefinita per una relazione è



Se servisse una classe collezione ordinata:



Semantica della collezione: Stringhe di proprietà



due modi per mostrare un attributo collezione

```
public class Sale {  
    private List<SalesLineItem> lineItems;  
    ...  
}
```

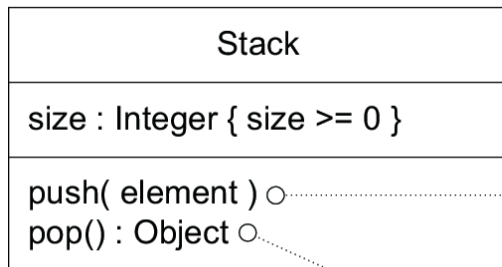
si noti che un'estremità di associazione può anche avere una stringa di proprietà come {ordered, List}

UML consente anche l'uso di parole chiavi definite all'utente

- È stato usato `{List}` per indicare che l'attributo collezione sarà implementato con un oggetto che implementa l'interfaccia *List*

Commenti, vincoli e corpo dei metodi

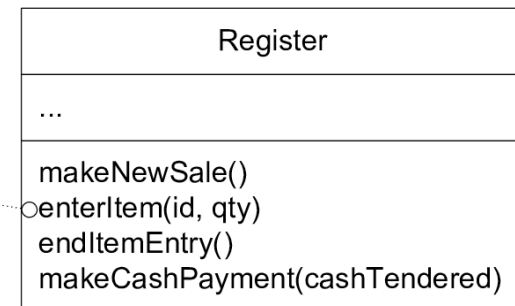
tre modi per mostrare i vincoli in UML



{ post-condizione: new-size = old-size + 1 }

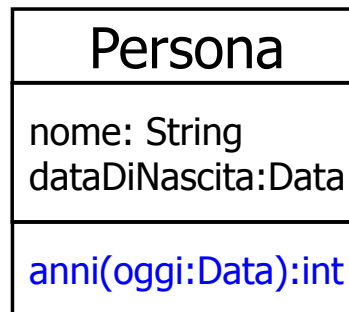
```
{  
  post-condizione: new-size = old-size - 1  
}
```

```
«method»  
// vanno bene sia un linguaggio specifico che pseudo-codice  
public void enterItem( id, qty ) {  
  ProductDescription desc = catalog.getProductDescription(id);  
  currentSale.makeLineItem(desc, qty);  
}
```



Operazioni

- Un'**operazione** di UML è una *dichiarazione*, con un nome, dei parametri, un tipo di ritorno, un elenco di eccezioni e magari un insieme di vincoli di precondizioni e post condizioni
- Una operazione associata ad una classe C indica che sugli oggetti della classe C si può eseguire una computazione
 - o per calcolare le proprietà
 - o per effettuare cambiamenti di stato (cioè per modificare le proprietà)



Definizione di un operazione

visibility name(parameter-list) : return-type {property-string}

- I nomi delle operazioni sono di solito in minuscolo
 - *Evitare simboli speciali e abbreviazioni*
- UML consente che le firme delle operazioni siano scritte in un qualunque linguaggio di programmazione, come Java, purché sia notificato al lettore o allo strumento
 - *Sono possibili entrambe l'espressioni:*

```
+ getPlayer( name : String ) : Player {exception IOException}  
public Player getPlayer( String name ) throws IOException
```

Osservazioni sulle operazioni

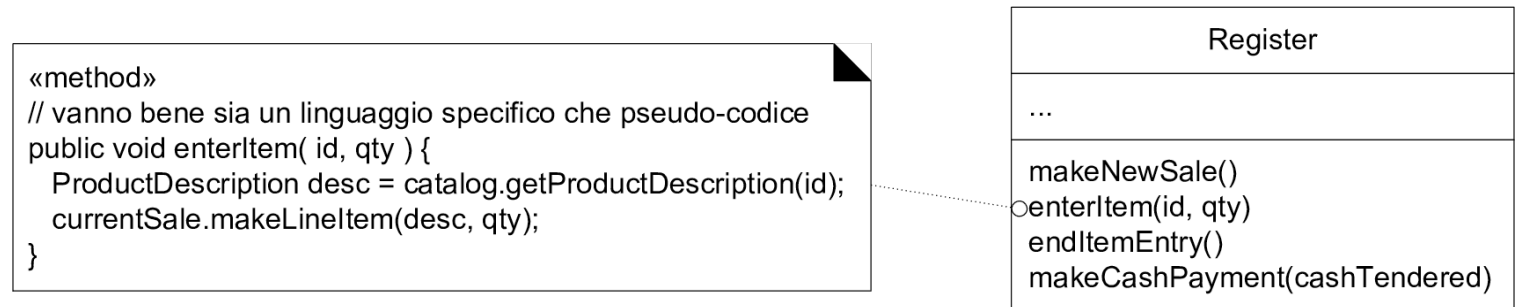
- Una operazione di una classe C è pensata per essere invocata facendo riferimento ad una istanza della classe C, chiamata oggetto di invocazione. Esempio di invocazione:

p.anni(oggi)

- (dove p è un oggetto della classe Persona). In altre parole, nell'attivazione di ogni operazione, oltre ai parametri c'è sempre in gioco implicitamente un oggetto (l'oggetto di invocazione) della classe in cui l'operazione è definita

Metodi

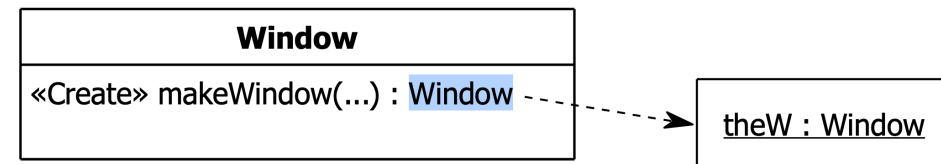
- In UML, un **metodo** è l'*implementazione* di un'operazione
- Un metodo può essere illustrato in vari modi, tra cui:
 - *Nei diagrammi di interazione, dai dettagli che dalla sequenze dei messaggi;*
 - *Nei diagrammi delle classi come una nota di UML stereotipata con <<method>>*



L'operazione “create” e operazioni per accedere agli attributi

- L'operazione create corrisponde a un costruttore

- stereotipo «Create» o «Constructor»



- Operazioni di accesso, come *getPrice* e *setPrice*, accedono e modificano gli attributi
 - *non vengono di solito mostrate nei diagrammi delle classi di progetto per l'elevato rapporto disturbo-valore che generano*

Parole chiave

- Una parola chiave in UML è un decoratore testuale per classificare un elemento di un modello
- Alcune tra le parole chiave predefinite in UML

Parola chiave	Significato	Esempio di uso
«actor»	il classificatore è un attore	nei diagrammi delle classi, sopra al nome di un classificatore
«interface»	il classificatore è un'interfaccia	nei diagrammi delle classi, sopra al nome di un classificatore
{abstract}	l'elemento è astratto; non può essere istanziato	nei diagrammi delle classi, dopo il nome di un classificatore o il nome di un'operazione
{ordered}	un insieme di oggetti ha un ordine predefinito	nei diagrammi delle classi, a un'estremità di associazione

Meccanismi di Estendibilità

- **Vincoli**

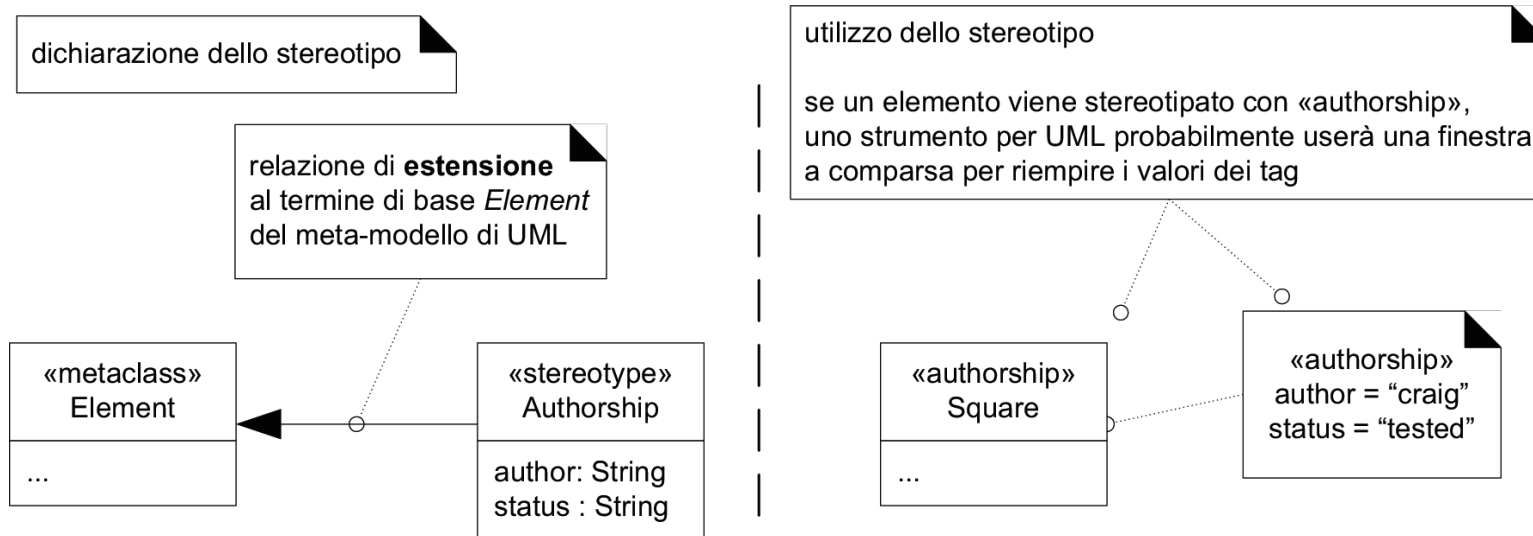
- Estendono la semantica di un elemento consentendo di aggiungere nuove regole

- **Stereotipi**

- Definiscono un nuovo elemento di modellazione UML basandosi su un esistente: è possibile definire la semantica degli stereotipi, che aggiungono nuovi elementi al meta-modello UML

- **tag**

- Permettono di estendere la definizione di un elemento tramite l'aggiunta di nuove informazioni specifiche



Profili

- I profili si usano per personalizzare UML per un uso specifico
- Un profilo UML è un insieme di stereotipi, tag e vincoli
 - *I valori etichettati e i vincoli sono associati agli stereotipi*
- Ogni stereotipo in un profilo estende uno degli elementi del meta-modello UML (es. Classe, Associazione)

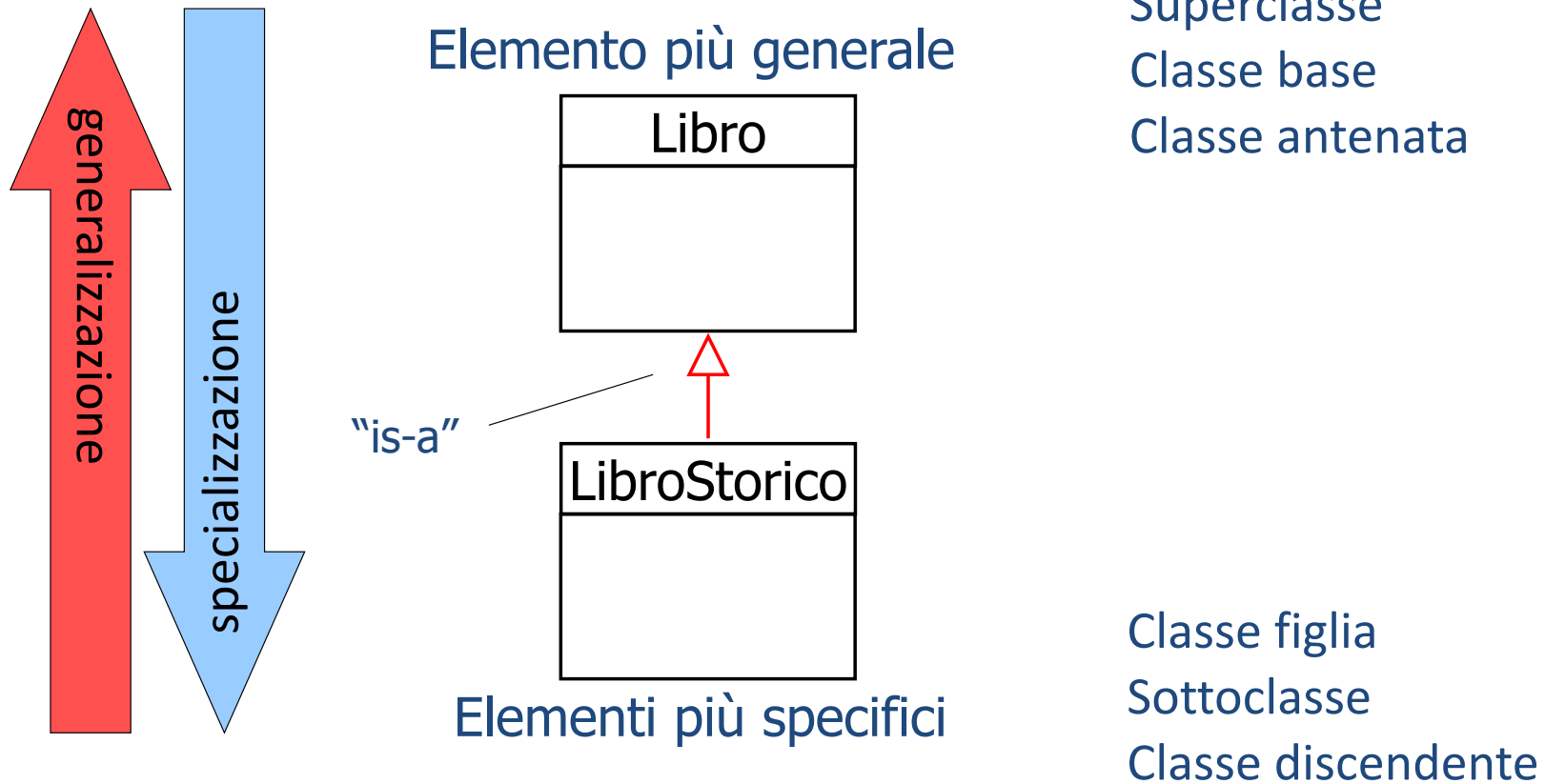
Proprietà e stringhe di proprietà

- In UML, una proprietà è un valore con un nome, che denota una caratteristica di un elemento
- alcune predefinite
 - *es. visibilità è una proprietà delle operazioni*
- possono essere descritte da stringhe proprietà
 - *es. {abstract, visibility=public}*

Generalizzazione

- Fino ad ora abbiamo assunto che due classi siano sempre disgiunte. In realtà può accadere che tra due classi sussista una relazione tale (chiamata relazione is-a) che ogni istanza di una sia anche istanza dell'altra
- In UML la relazione is-a si modella mediante la nozione di **generalizzazione**
- La generalizzazione coinvolge una superclasse ed una o più sottoclassi (dette anche **classi derivate**). Il significato della generalizzazione è il seguente: **ogni istanza di ciascuna sottoclasse è anche istanza della superclasse**
- Quando la sottoclasse è una, la generalizzazione modella appunto la **relazione is-a** tra la sottoclasse e la superclasse

Generalizzazione in UML



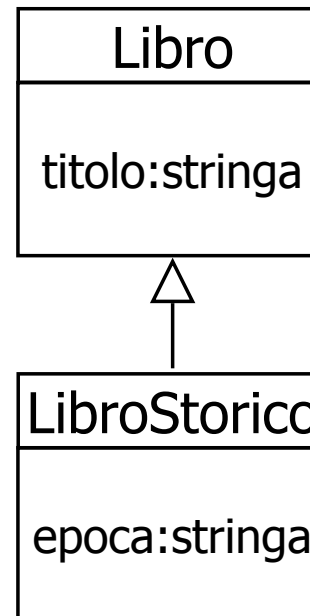
Ereditarietà

Principio di ereditarietà: ogni proprietà della superclasse è anche una proprietà della sottoclasse, e non si riporta esplicitamente nel diagramma

Dal fatto che

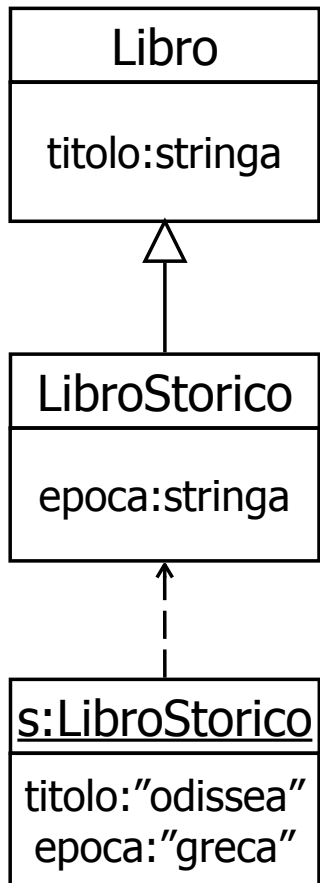
1. Ogni istanza di Libro ha un titolo
 2. Ogni istanza di LibroStorico è una istanza di Libro
- segue logicamente che
3. Ogni istanza di LibroStorico ha un titolo

Ragionamento sillogistico (Aristotele)



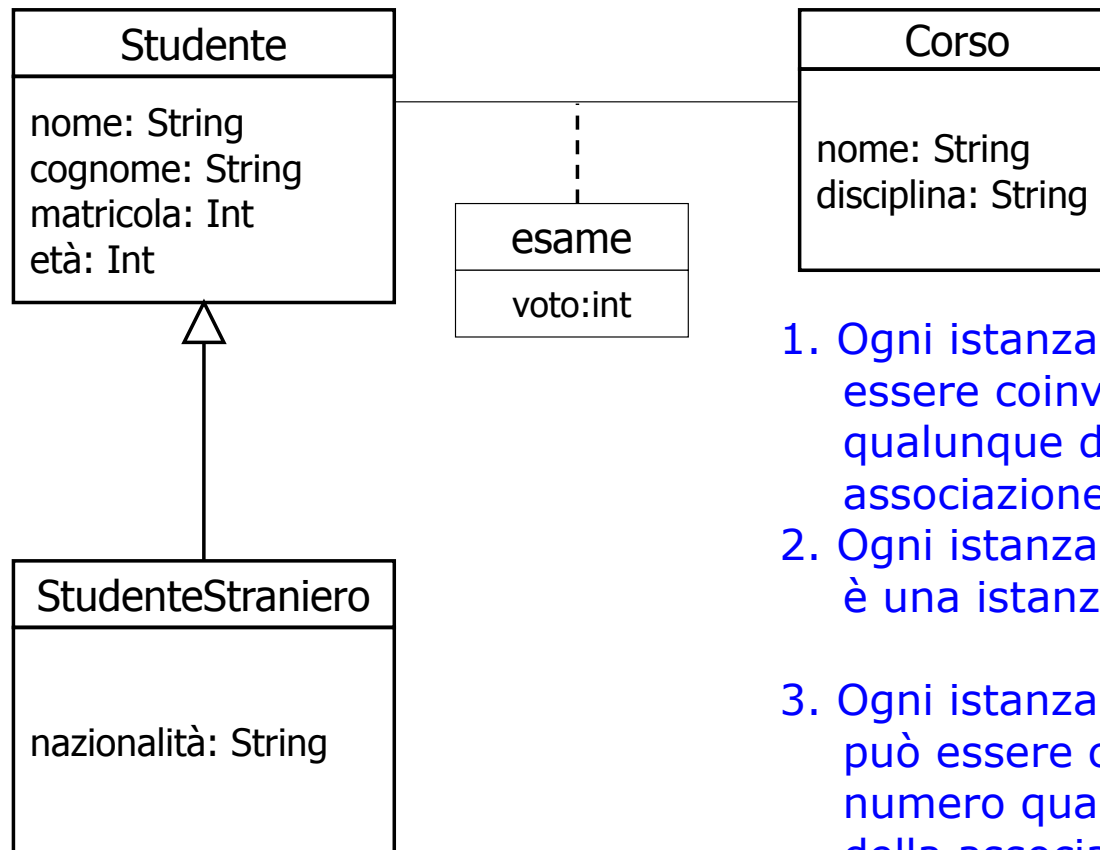
Titolo è ereditato da Libro. Epoca è un'ulteriore proprietà

Ereditarietà: Istanze



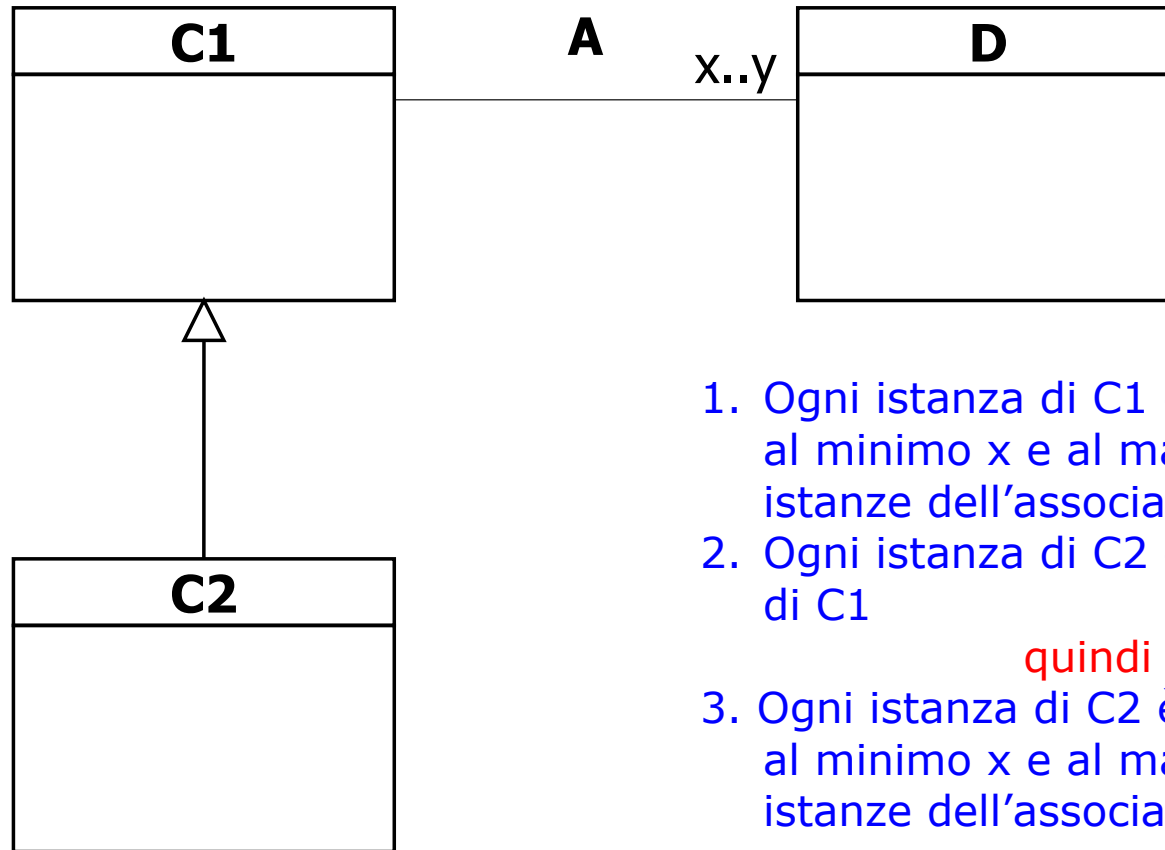
- **s** è istanza sia di **LibroStorico** (classe più specifica) sia di **Libro**
- non è più vero che due classi diverse sono disgiunte: **Libro** e **LibroStorico** non sono ovviamente disgiunte
- resta comunque vero che ogni istanza ha una ed una sola classe più specifica di cui è istanza; in questo caso la classe più specifica di **s** è **LibroStorico**
- **s** ha un valore per tutti gli attributi di **LibroStorico**, sia quelli propri, sia quelli ereditati dalla classe **Libro**

Ereditarietà sulle associazioni



1. Ogni istanza di **Studente** può essere coinvolta in un numero qualunque di istanze della associazione "esame"
2. Ogni istanza di **StudenteStraniero** è una istanza di **Studente**
quindi
3. Ogni istanza di **StudenteStraniero** può essere coinvolta in un numero qualunque di istanze della associazione "esame"

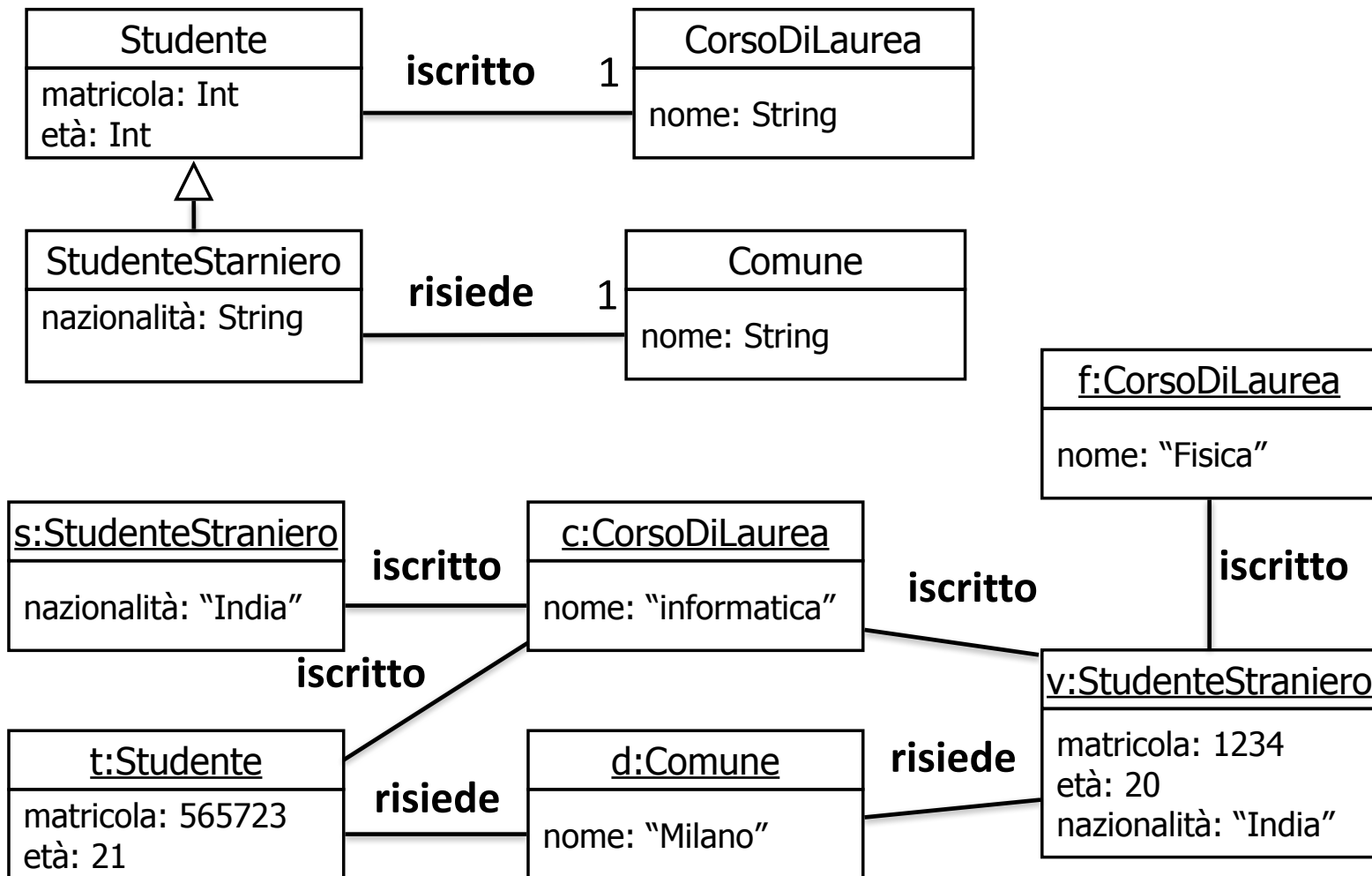
Ereditarietà sulle molteplicità



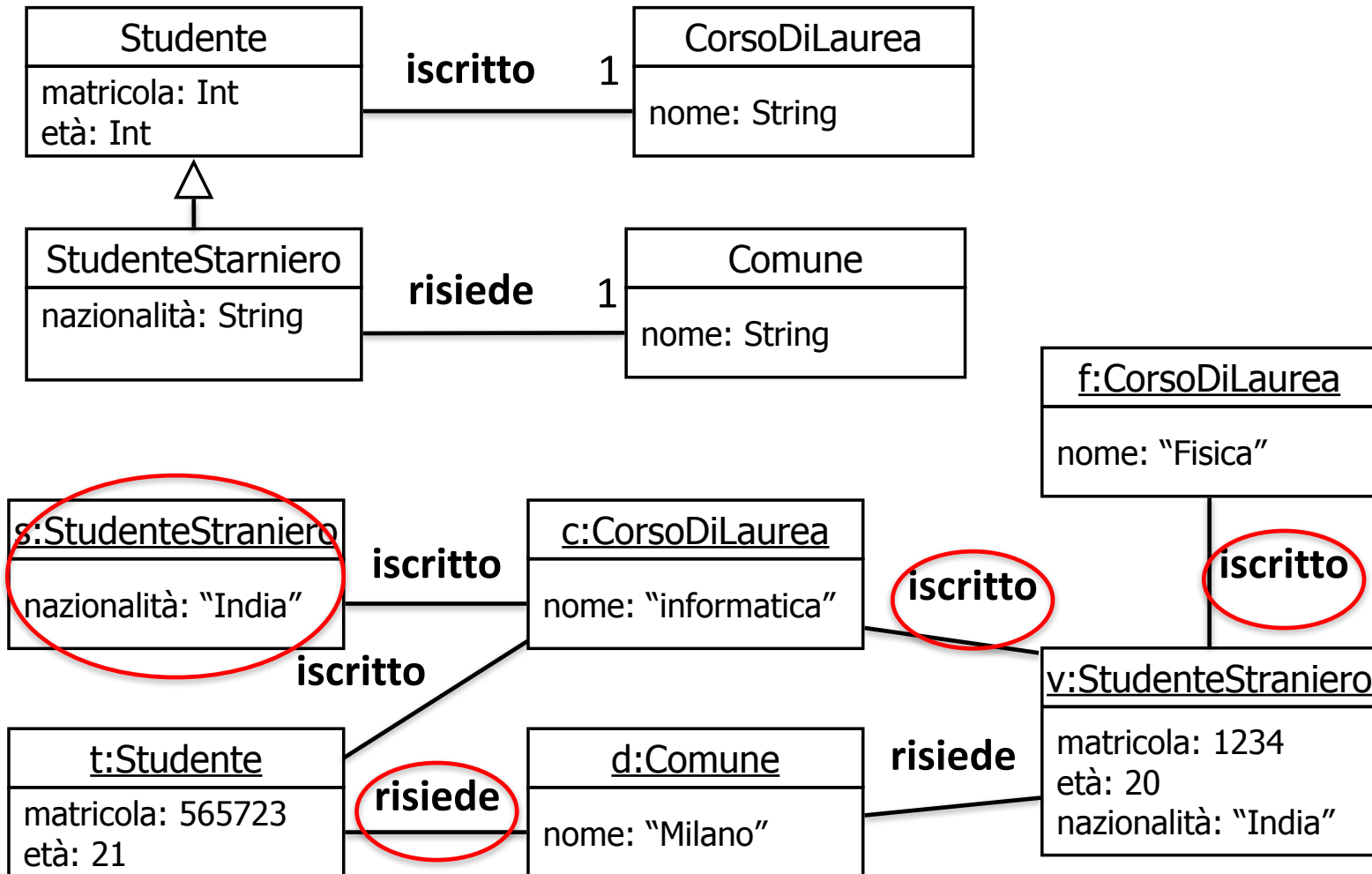
1. Ogni istanza di C1 è coinvolta in al minimo x e al massimo y istanze dell'associazione A
2. Ogni istanza di C2 è una istanza di C1
3. Ogni istanza di C2 è coinvolta in al minimo x e al massimo y istanze dell'associazione A

quindi

Individuare gli errori



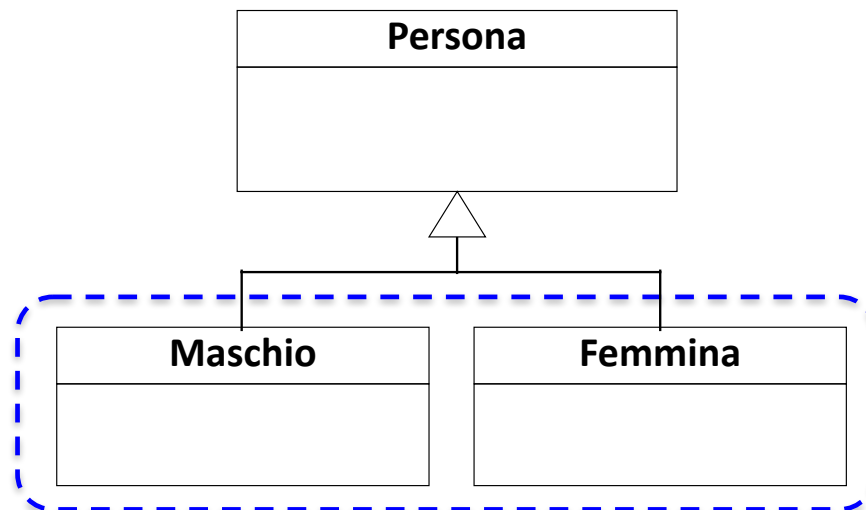
Individuare gli errori



Generalizzazione in UML

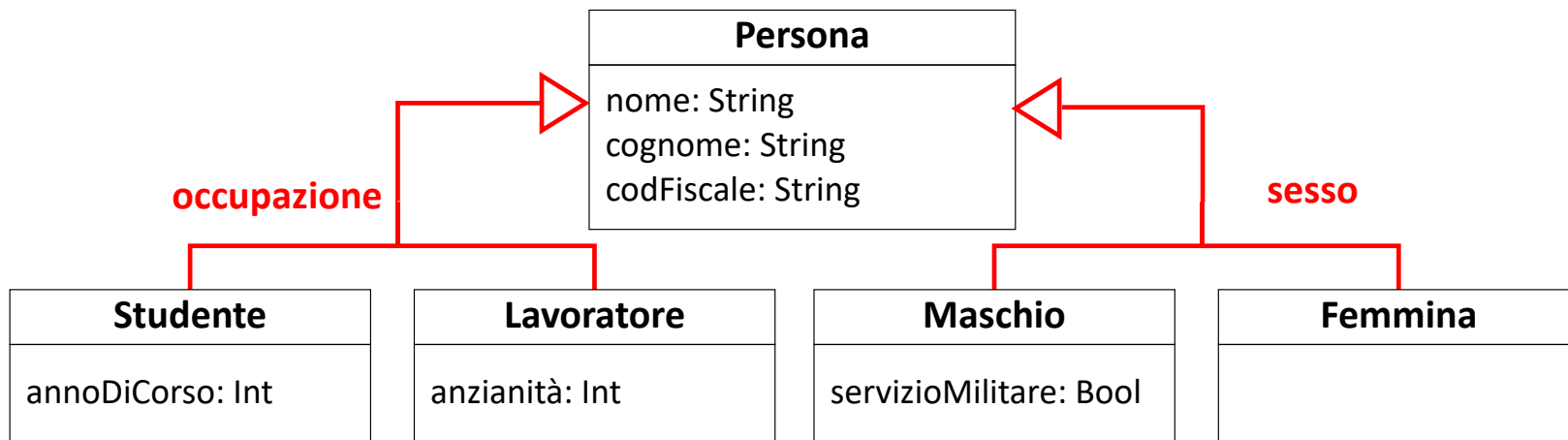
- Finora, abbiamo considerato la generalizzazione come mezzo per modellare la relazione is-a tra due classi. La superclasse però può anche generalizzare **diverse sottoclassi** rispetto ad un **unico criterio** (che si può indicare con un nome, che è il nome della generalizzazione)

Tra Maschio e Persona
e tra Femmina e
Persona sussiste la
relazione is-a



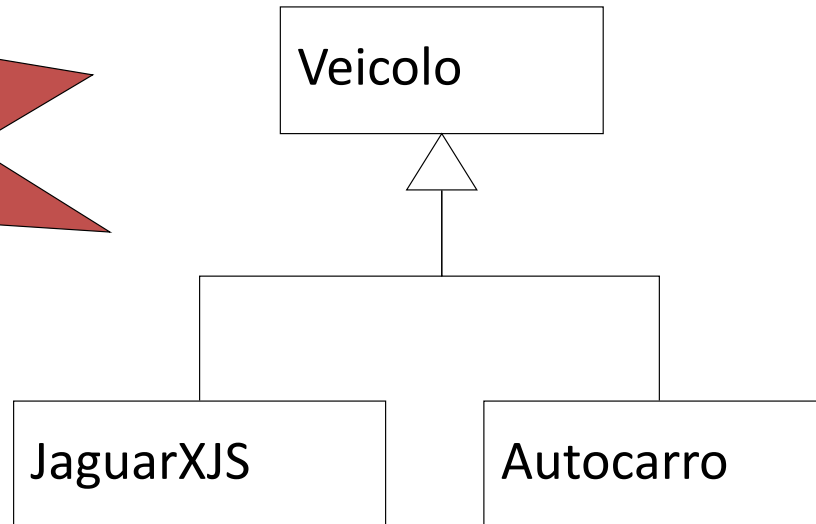
Più generalizzazioni della stessa classe

- Concettualmente, non c'è alcuna correlazione tra due generalizzazioni diverse, perché rispondono a due criteri diversi di classificare le istanze della superclasse

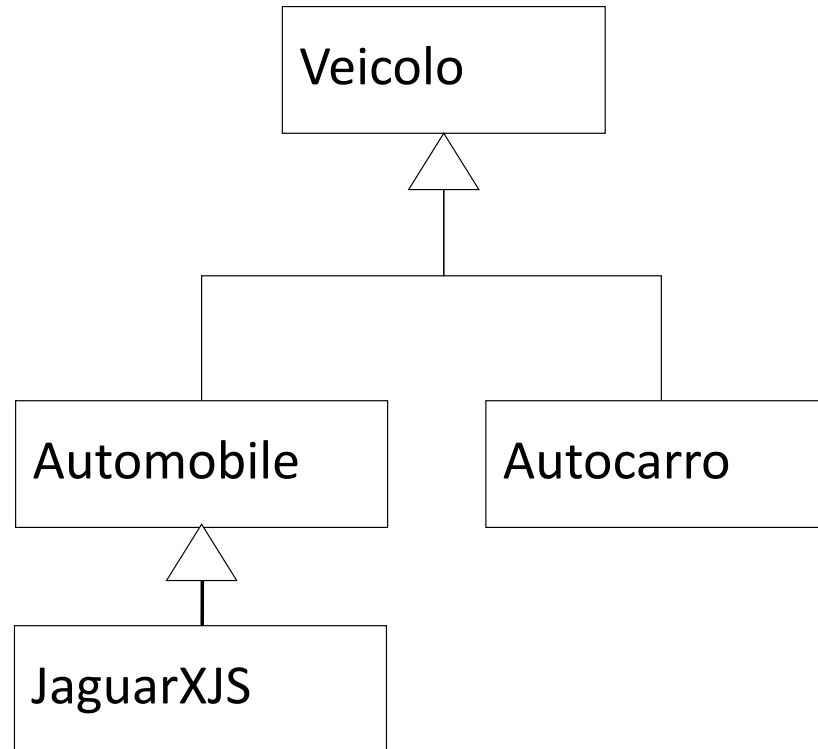


Trovare l'errore

cosa c'è di
sbagliato con
questo
modello?

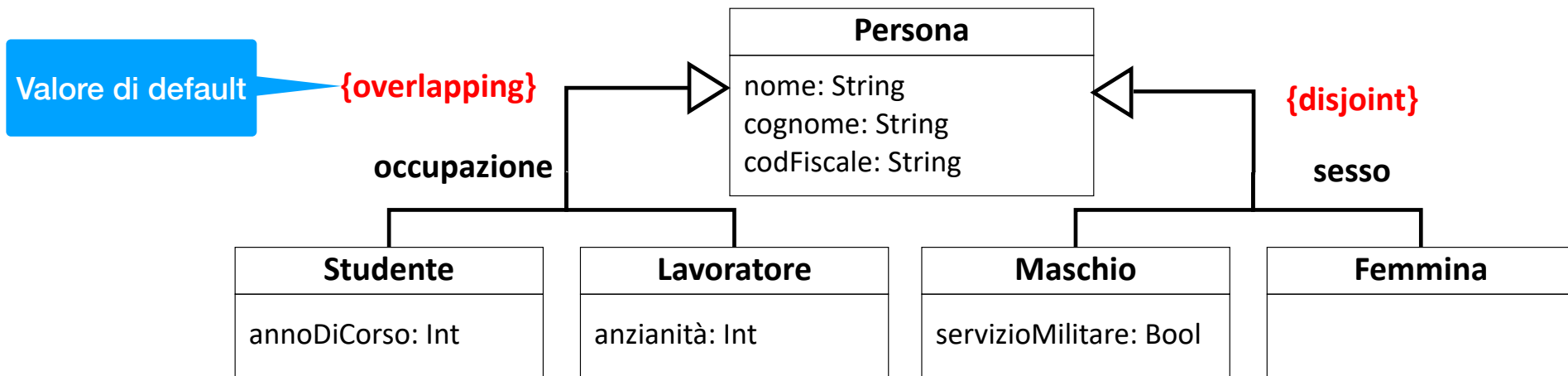


Soluzione



Generalizzazioni disgiunte

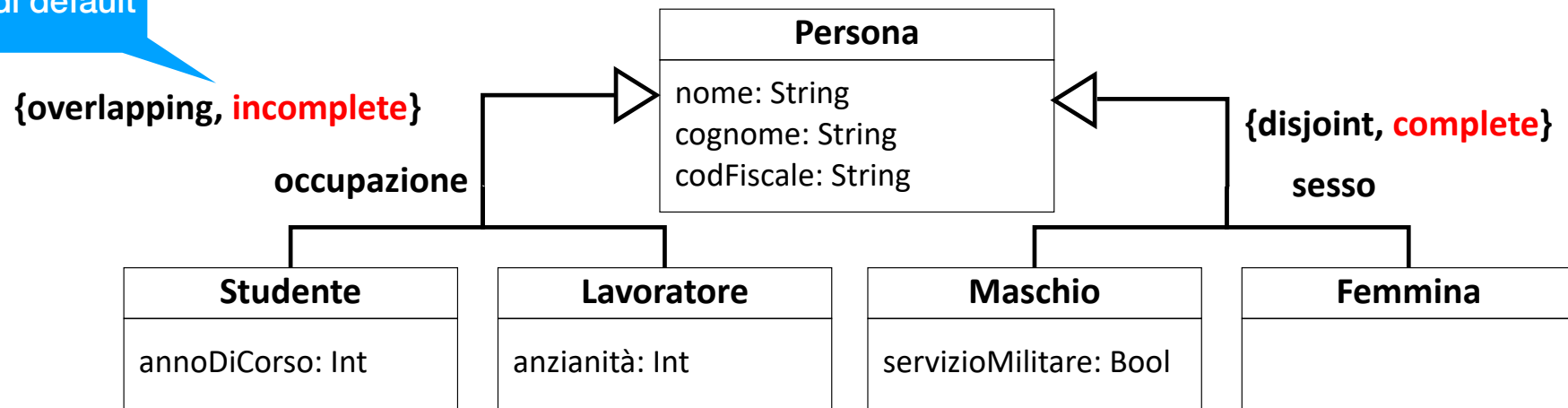
- Una generalizzazione può essere disgiunta (i classificatori specifici non hanno istanze comuni) oppure no



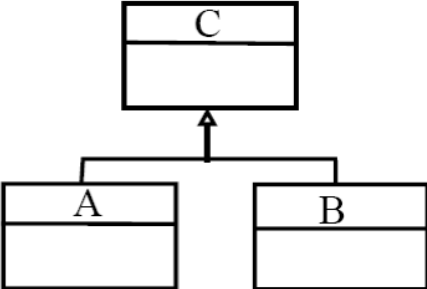
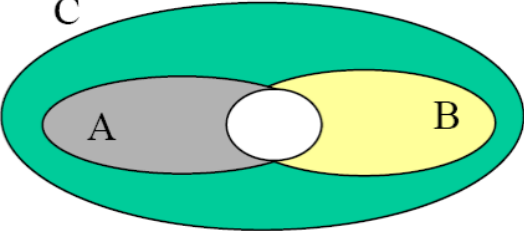
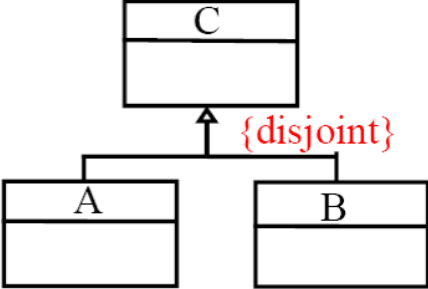
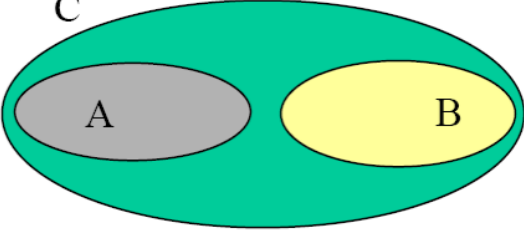
Generalizzazioni complete

- Una generalizzazione può essere completa (l'unione delle istanze delle sottoclassi è uguale all'insieme delle istanze della superclasse) o no

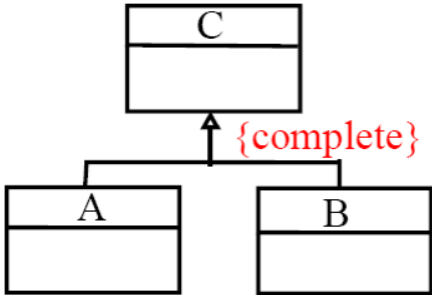
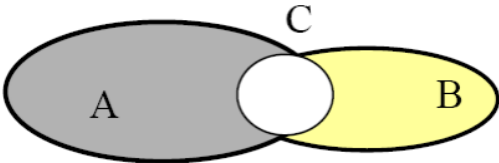
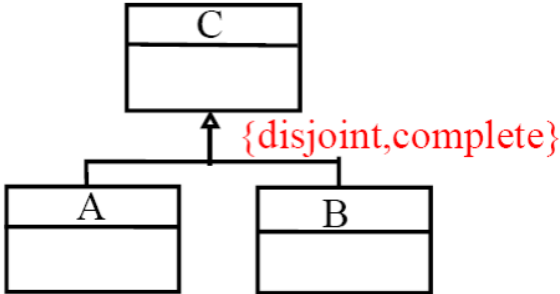
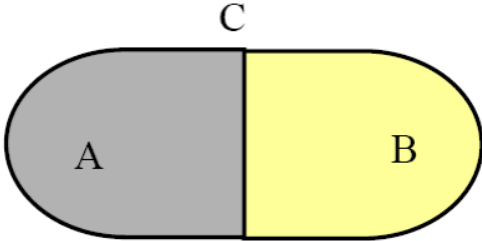
Valore di default



Tipi di generalizzazione

Livello intensionale	Livello estensionale
 A diagram showing a generalization hierarchy. At the top is a box labeled 'C'. Below it are two boxes labeled 'A' and 'B'. An upward-pointing arrow connects the space between 'A' and 'B' to the bottom of 'C', indicating that 'C' is the generalization of 'A' and 'B'.	 A Venn diagram illustrating the extensional level. A large green oval labeled 'C' contains two overlapping ovals: a gray one labeled 'A' and a yellow one labeled 'B'. The intersection of 'A' and 'B' is a white circle, representing the common elements between the two sets.
 A diagram showing a disjoint generalization hierarchy. At the top is a box labeled 'C'. Below it are two boxes labeled 'A' and 'B'. An upward-pointing arrow connects the space between 'A' and 'B' to the bottom of 'C'. The label '{disjoint}' is written in red next to the arrow, indicating that 'A' and 'B' have no common elements.	 A Venn diagram illustrating the extensional level for disjoint sets. A large green oval labeled 'C' contains two disjoint ovals: a gray one labeled 'A' and a yellow one labeled 'B'. There is no intersection between 'A' and 'B'.

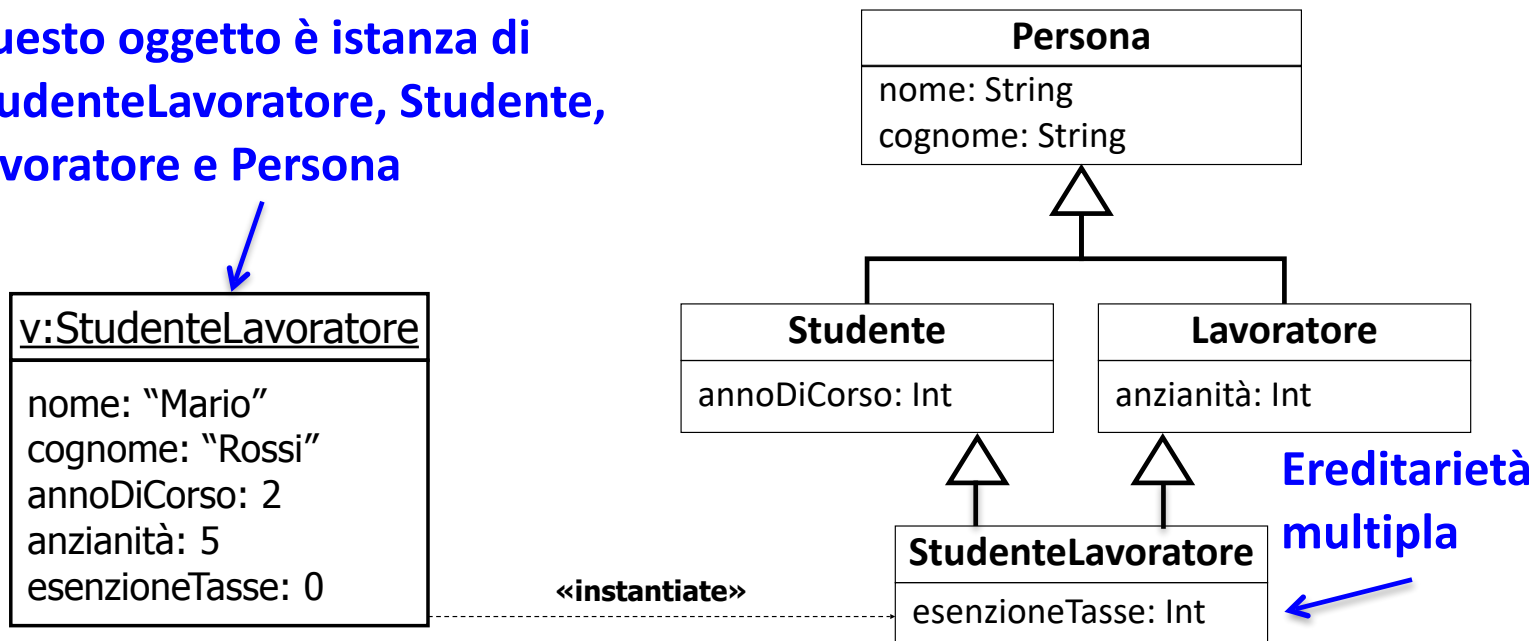
Tipi di generalizzazione

Livello intensionale	Livello estensionale
	
	

Ereditarietà multipla

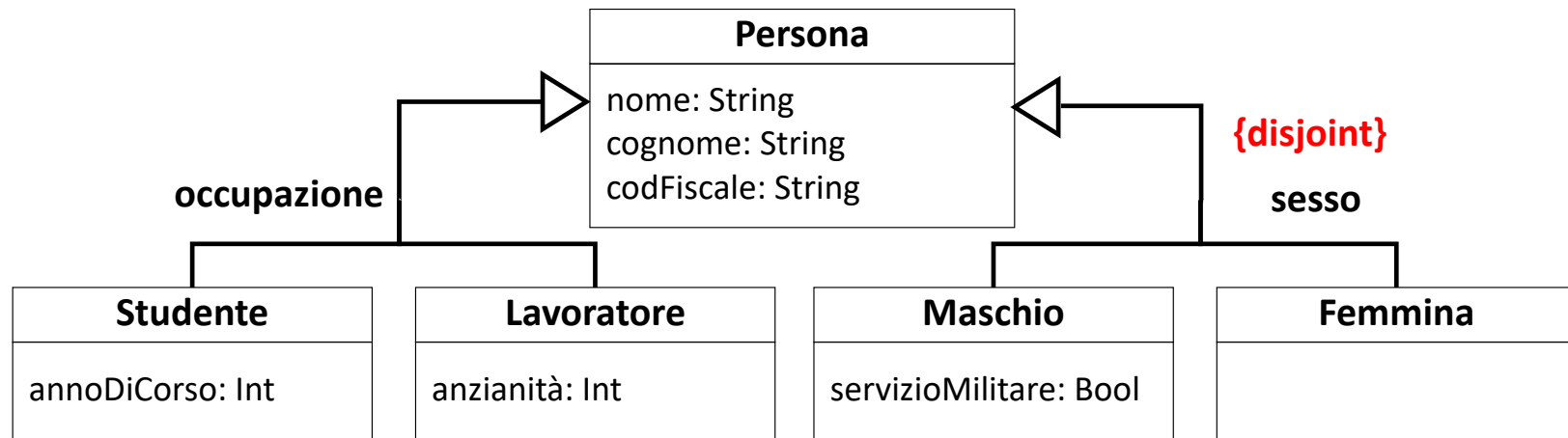
Attenzione: poichè un oggetto è istanza di una sola classe più specifica, due sottoclassi non disgiunte possono avere istanze comuni solo se hanno una sottoclasse comune (ereditarietà multipla)

Questo oggetto è istanza di
**StudenteLavoratore, Studente,
Lavoratore e Persona**



Differenza tra classi disgiunte e non

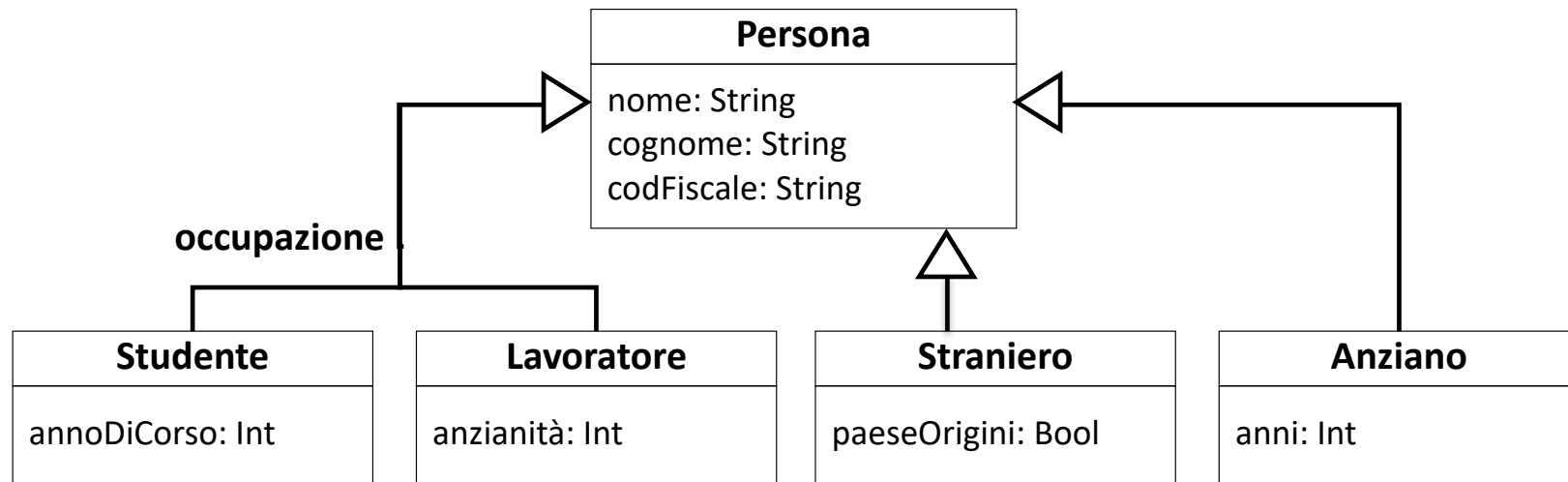
Due classi disgiunte non possono avere sottoclassi comuni, mentre è possibile definire una classe come sottoclasse di due classi non disgiunte



È possibile definire una classe che è sottoclasse sia di **Studente** che di **Lavoratore**

Non è possibile definire una classe che è sottoclasse sia di **Maschio** che di **Femmina**

Differenza tra due is-a e una generalizzazione



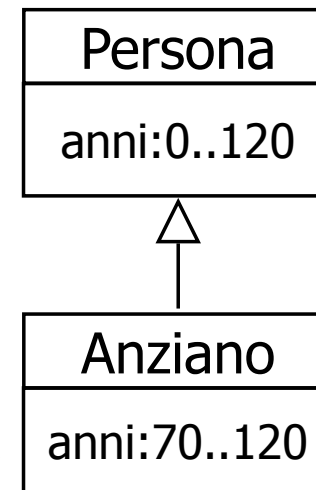
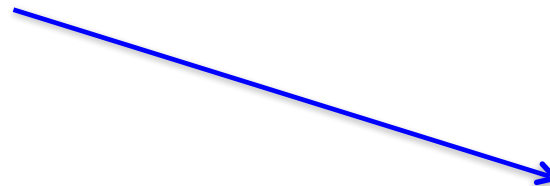
Le due sottoclassi derivano da uno stesso criterio di classificazione delle istanze della superclasse

Le due classi sono indipendenti, nel senso che il loro significato non deriva dallo stesso criterio di classificazione delle istanze della superclasse

Specializzazione di un attributo

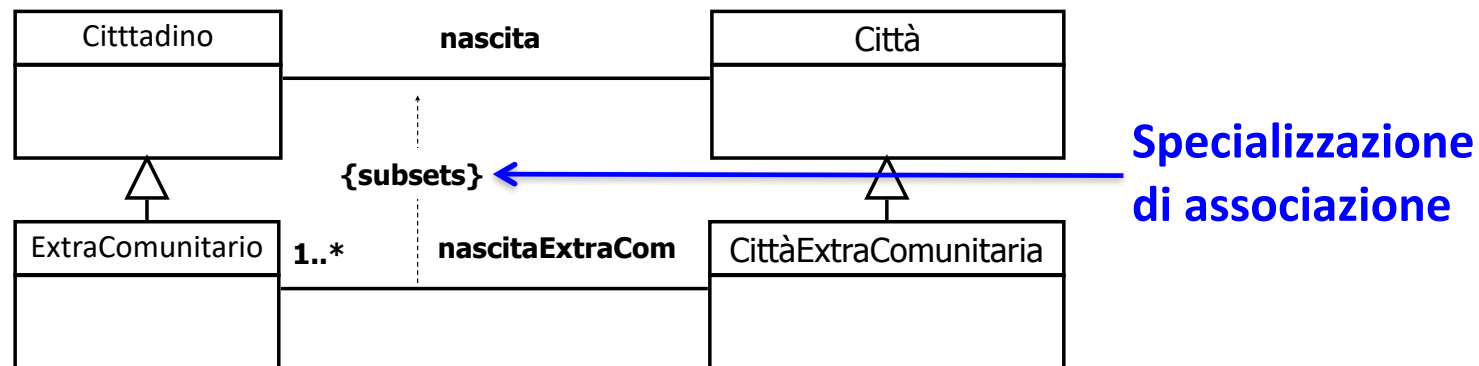
- In una generalizzazione la sottoclasse non solo può avere proprietà aggiuntive rispetto alla superclasse, ma può anche specializzare le proprietà ereditate dalla superclasse.
 - *Specializzazione di un attributo: Se una classe C1 ha un attributo A di tipo T1, e se C2 è una sottoclasse di C1, specializzare A in C2 significa definire A anche in C2 ed assegnargli un tipo T2 i cui valori sono un sottoinsieme dei valori di T1*

Specializzazione dell'attributo anni



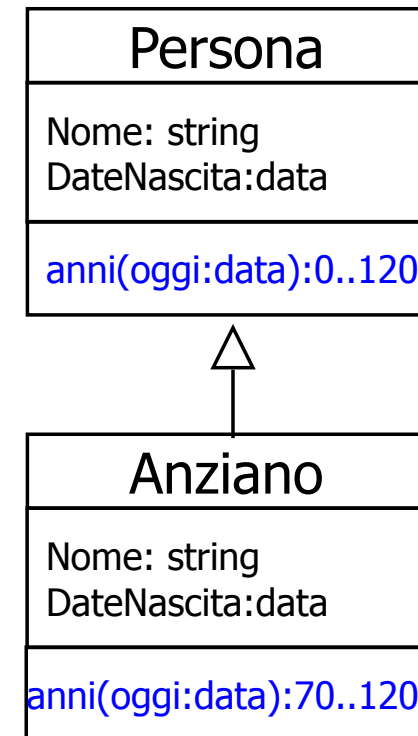
Specializzazione di un'associazione

- Se una classe C1 partecipa ad una associazione R con un'altra classe C3, e se C2 è una sottoclasse di C1, specializzare R in C2 significa:
 - Definire una nuova associazione R2 tra la classe C2 e una classe C4 che è sottoclasse di C3 (al limite C4 può essere la classe C3 stessa)
 - Stabilire una dipendenza di tipo {subsets} da R2 a R
 - Definire eventualmente molteplicità più specifiche (molteplicità minima non minore, e molteplicità massima non maggiore) su R2 rispetto alle corrispondenti molteplicità definite su R



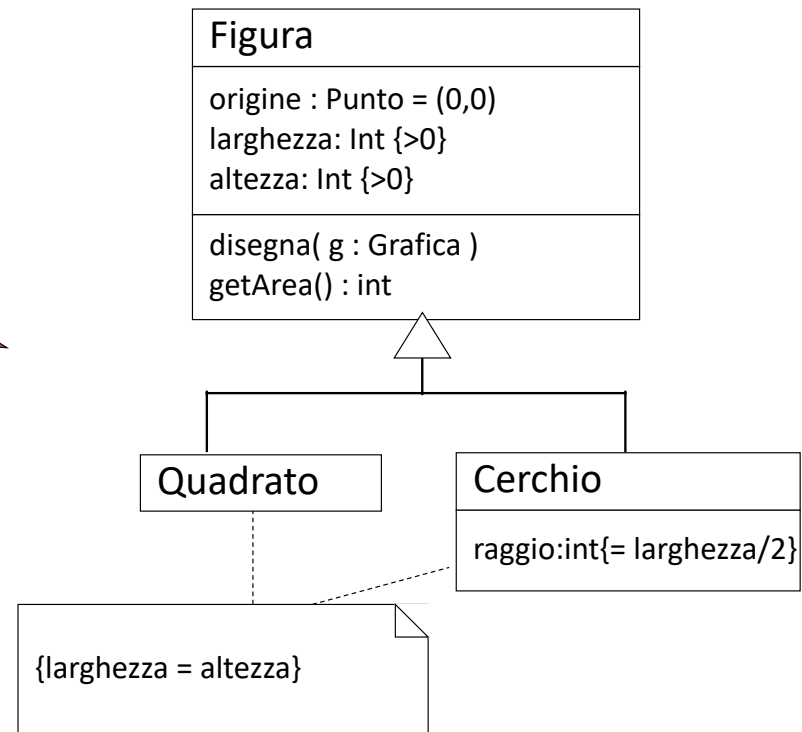
Specializzazione di operazioni

- Una operazione si specializza specializzando i parametri e/o il tipo di ritorno
- Si noti che il metodo associato ad una operazione specializzata in una sottoclasse è in genere diverso dal metodo associato alla stessa operazione nella superclasse

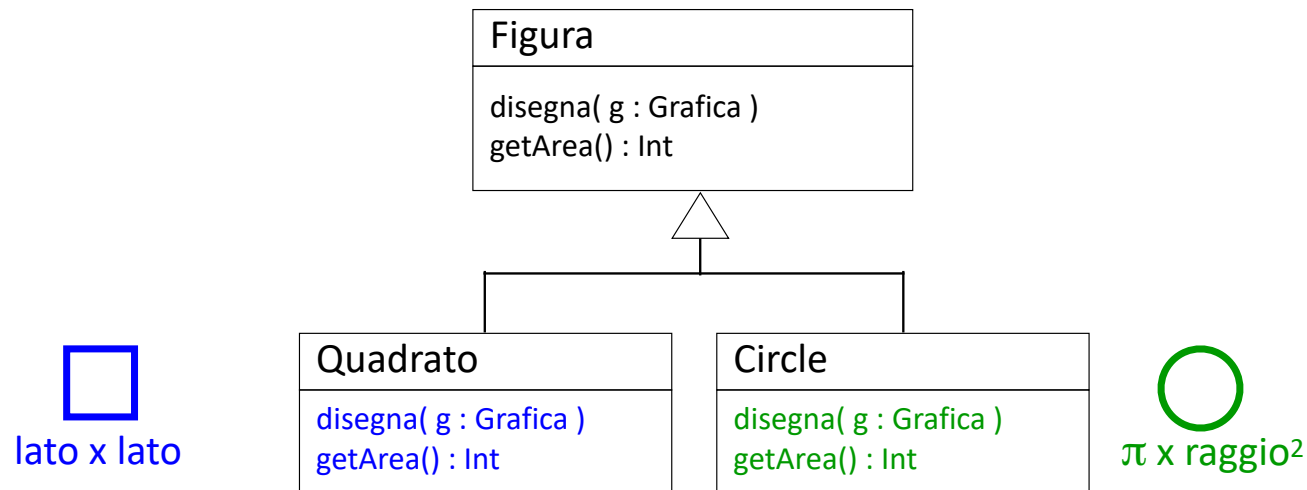


Cosa c'è di sbagliato?

cosa c'è di
sbagliato con
questo
modello?

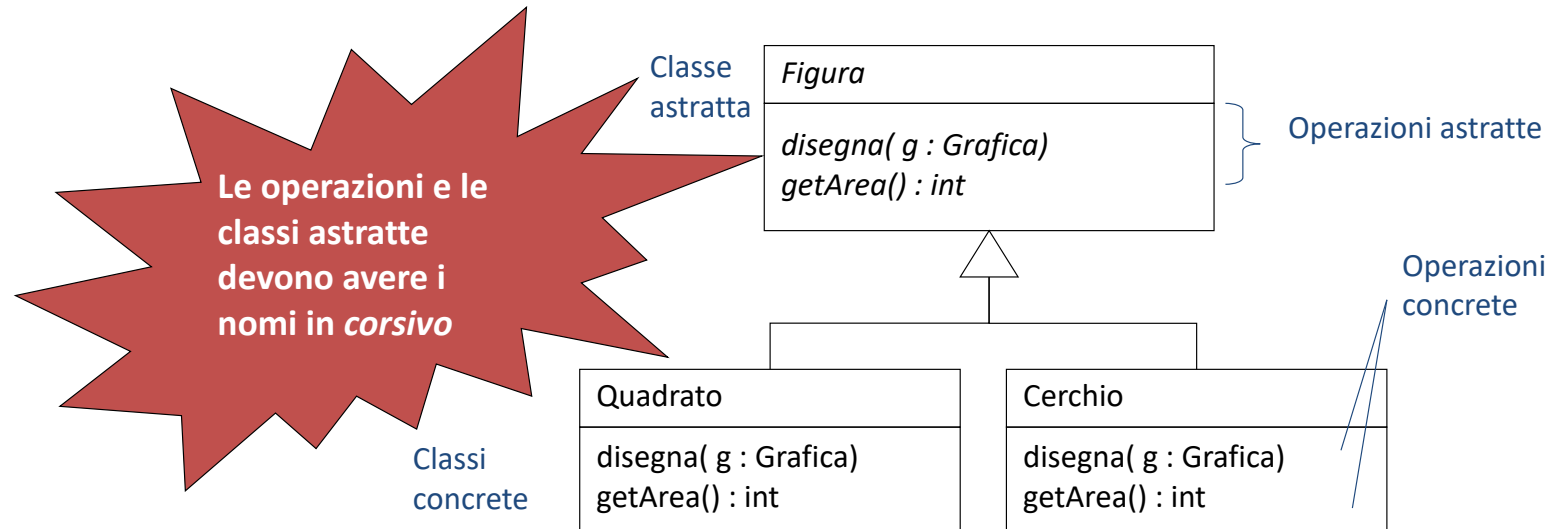


Overriding



- Le sottoclassi spesso necessitano di sovrascrivere il comportamento della superclasse
- Per sovrascrivere un'operazione della superclasse, una sottoclasse deve definire una operazione con la stessa firma
 - *La firma dell'operazione è data dal nome dell'operazione, il tipo di ritorno e i tipi di tutti i parametri*
 - *I nomi dei parametri non vengono considerati come parte della firma*

Operazioni e Classi Astratte

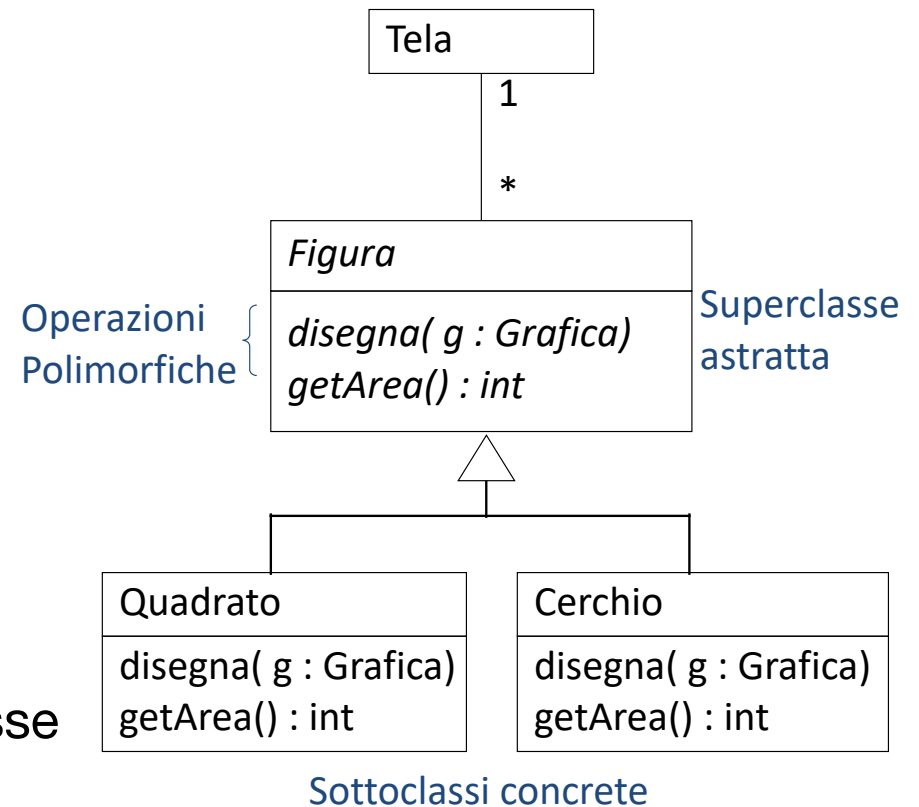


- Non possiamo fornire un implementazione per
Figura:: *disegna*(g : Grafica) o per
Figura:: *getArea*() : int
perché non sappiamo come disegnare o calcolare l'area per una "figura" generica!
- Le operazioni che non hanno un implementazione sono operazioni astratte
- Una classe con una o più operazioni astratte non può essere istanziata (classe astratta)

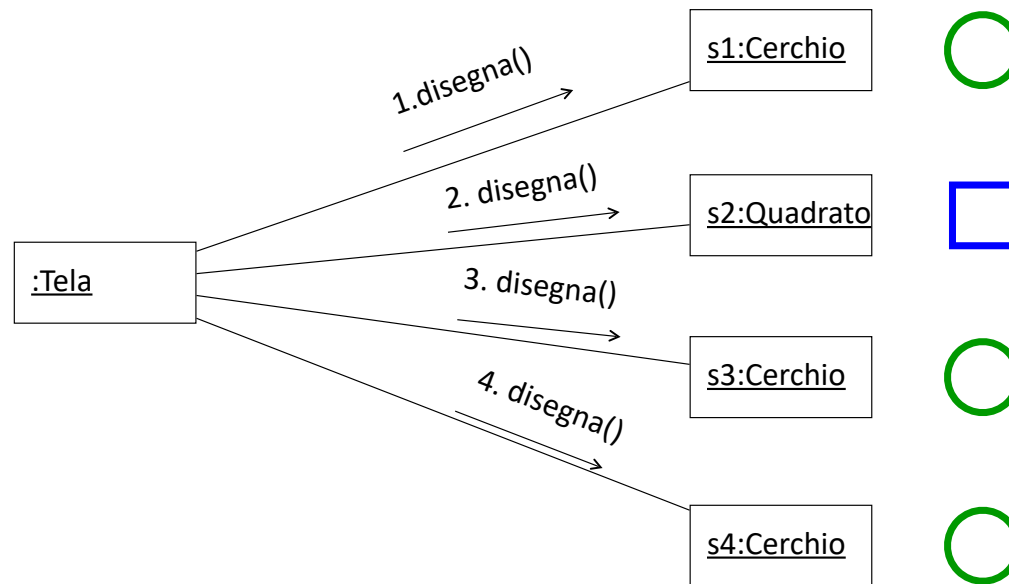
Polimorfismo

Un oggetto *Tela* contiene una insieme di oggetti *Figura* che possono essere istanze o di *Quadrato* o di *Cerchio*

- Polimorfismo = "molte forme"
 - *Un operazione polimorfica ha molte implementazioni*
 - *Quadrato e Cerchio forniscono implementazioni differenti per le operazioni polimorfiche **Figura::disegna()** e **Figura::getArea()***
- Tutte le sottoclassi concrete di Figura devo fornire le operazioni concrete disegna() e getArea() perché sono astratte nella superclasse



Che cosa succede?



Dipendenza

In UML, una dipendenza è una **relazione di dipendenza**

- un elemento cliente (classi, package, casi d'uso...) è a conoscenza di un elemento fornitore
 - *Un cambiamento ne fornitore potrebbe influire sul cliente*
- Un elemento è **accoppiato** con un altro elemento o dipende da un altro elemento

Alcune tipologie di dipendenze

- Avere un attributo del tipo del fornitore
- Inviare un messaggio a un fornitore
- Ricevere un parametro del tipo del fornitore
- Il fornitore è una super classe con un'interfaccia implementati



Quando mostrare una dipendenza?

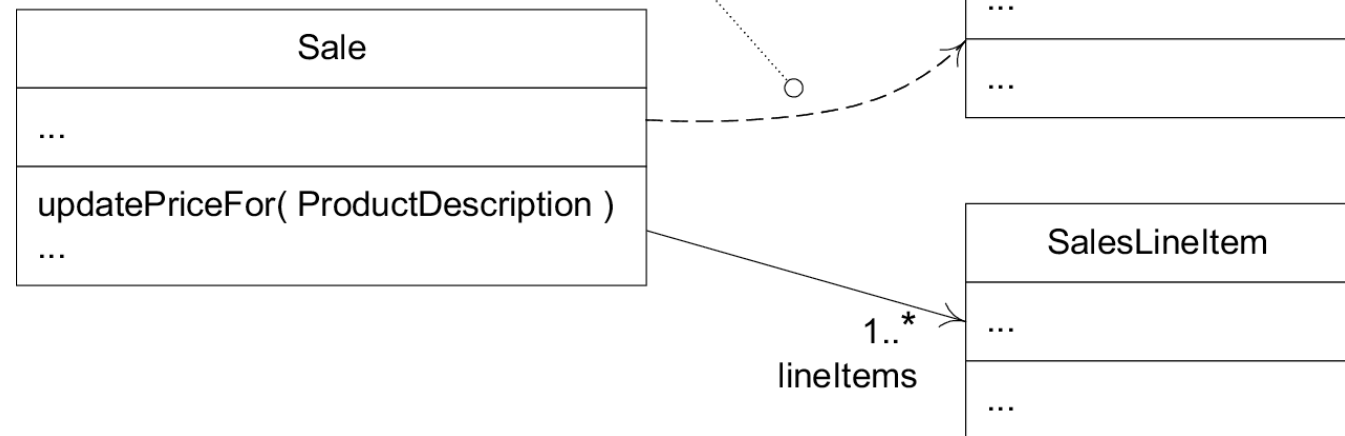
Nei diagrammi delle classi le linee delle dipendenze vanno usate per

- descrivere le dipendenze tra oggetti per visibilità globale,
- per variabile parametro,
- per variabile locale
- e per metodo statico
 - *quando viene effettuato una chiamata un metodo statico da un'altra classe*

Esempio

```
public class Sale {  
    ...  
    public void updatePriceFor( ProductDescription pd ) {  
        Money basePrice= pd.getPrice();  
        ...  
    }  
    ...  
}
```

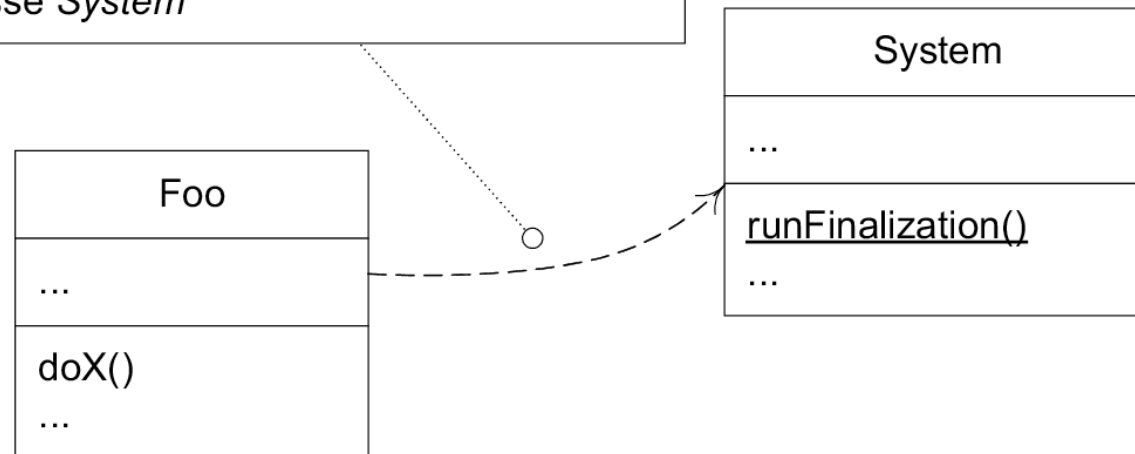
Sale ha una visibilità per parametro verso *ProductDescription*, quindi un qualche tipo di dipendenza



Un altro esempio

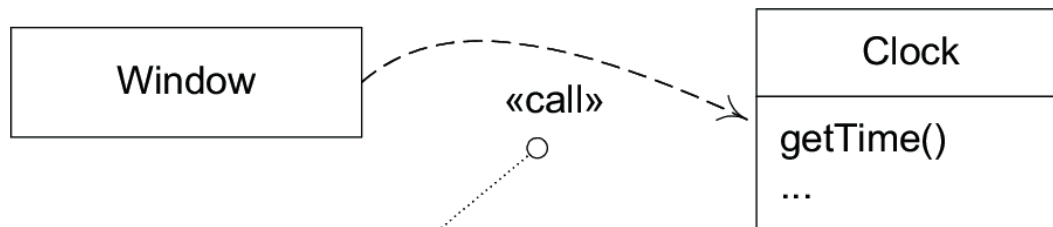
```
public class Foo {  
    ...  
    public void doX() {  
        System.runFinalization(  
        );  
    }  
    ...  
}
```

il metodo *doX* richiama il metodo statico *runFinalization*, quindi ha una dipendenza dalla classe *System*

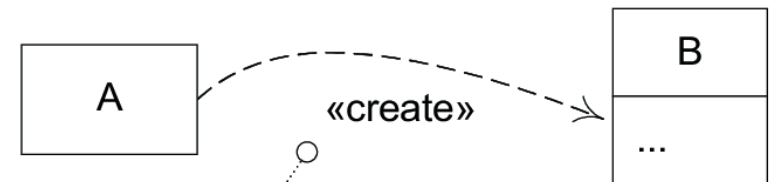


Etichette delle dipendenze

- Per mostrare il tipo di una dipendenza si possono utilizzare parole chiavi e stereotipi



una dipendenza di chiamata
alle operazioni di un *Clock*



una dipendenza secondo cui gli oggetti A
creano oggetti B

Interfaccia



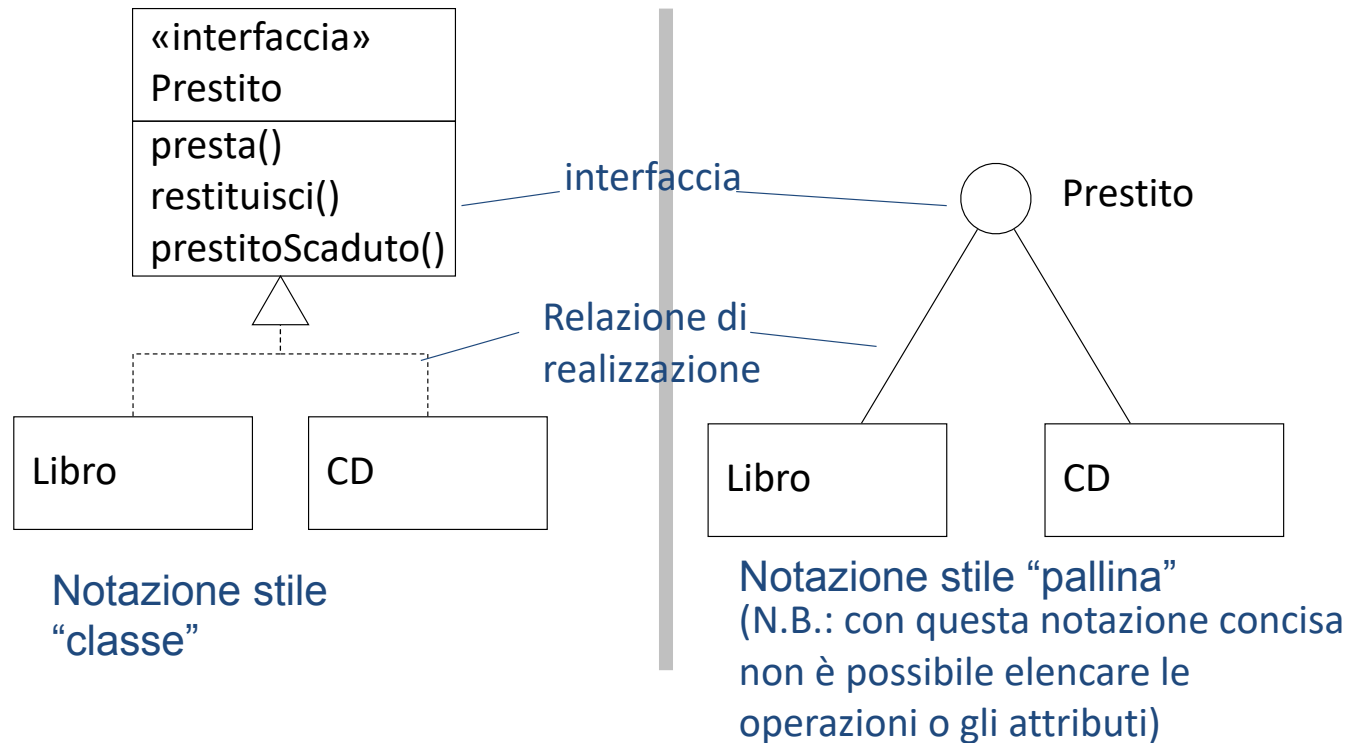
design by
contract

- Un interfaccia è un insieme di funzionalità pubbliche identificate da un nome
- Separa le specifiche di una funzionalità dall'implementazione
- Un interfaccia definisce un contratto e tutti i classificatori che la realizzano devono rispettare

L'interfaccia specifica	Classificatore di realizzazione
Operazione	Deve avere un'operazione con la stessa segnatura e semantica
Attributo	Deve avere operazioni pubbliche per leggere o cambiare il valore dell'attributo – al classificatore non è richiesto di possedere l'attributo specificato dall'interfaccia, ma deve comportarsi come se lo avesse
associazione	Deve avere un'associazione al classificatore destinazione – se un'interfaccia specifica un'associazione ad un'altra interfaccia, i classificatori di queste interfacce devono avere un'associazione tra di loro
vincolo	Deve supportare il vincolo
stereotipo	Deve avere lo stereotipo
Valore etichettato	Deve avere il valore etichettato
protocollo	Deve realizzare il protocollo

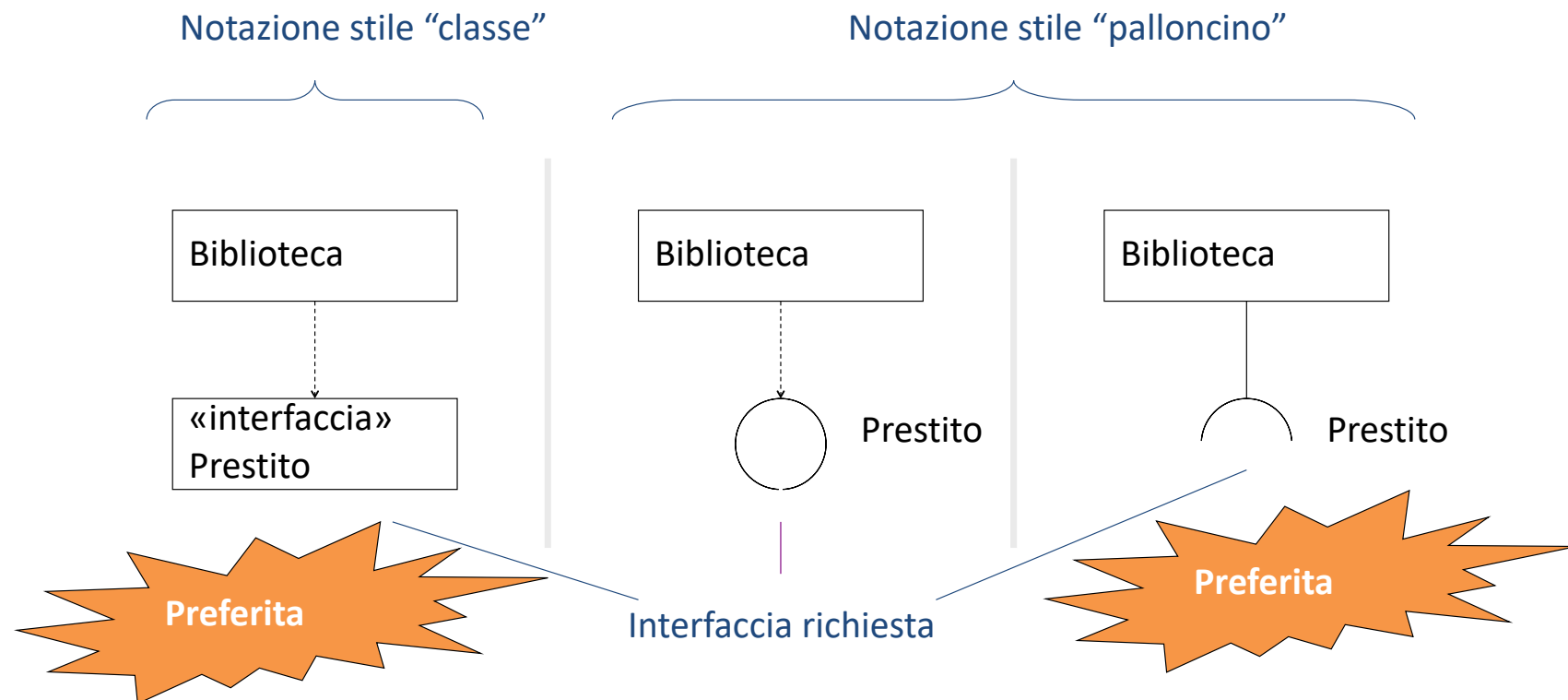
Sintassi interfacce fornite

- Un interfaccia fornita indica che un classificatore implementa il servizio definito in un'interfaccia



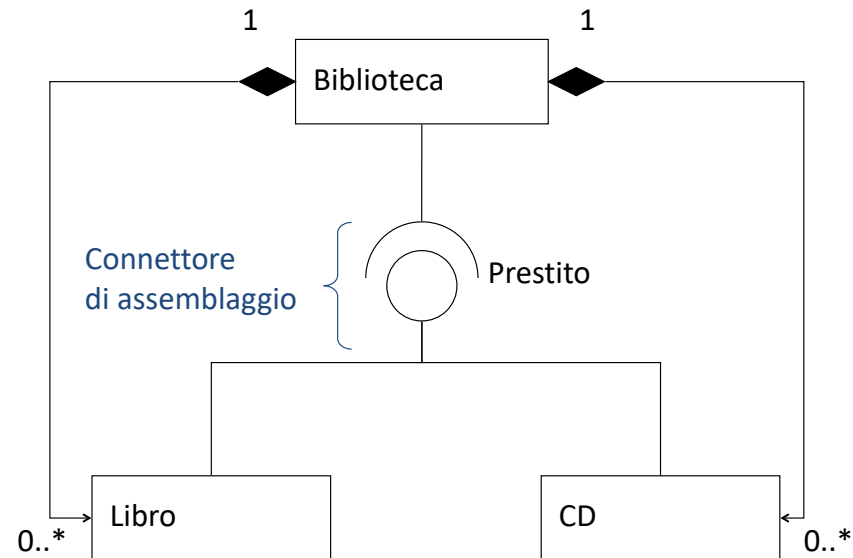
Sintassi interfacce richieste

- Un'interfaccia richiesta indica che un classificatore utilizza i servizi definiti dall'interfaccia



Connettore di assemblaggio

- Si possono connettere interfacce richieste e fornite utilizzando il connettore di assemblaggio



Individuare le interfacce

- Esamina ogni associazione:
 - *Dovrebbe questa associazione effettivamente essere associata ad una classe particolare o dovrebbe essere più flessibile?*
- Esamina ogni messaggio inviato:
 - *Dovrebbe questo messaggio essere inviato solo agli oggetti di una specifica classe o dovrebbe essere più flessibile?*
- Raggruppa operazioni che potrebbero essere riusabili altrove
- Raggruppa insiemi di attributi che si ripetono in più di una classe
- Esamina classi che svolgono lo stesso ruolo nel sistema – il ruolo può indicare un'interfaccia
- Esamina la dipendenza tra componenti – utilizza connettori di assemblaggio dove possibile
- Analizza la possibilità di espansioni future – Se si prevede che nel futuro altre classi verranno aggiunte al sistema, tenta di definire una o più interfacce il cui compito è quello di definire il protocollo per definire queste nuove classi

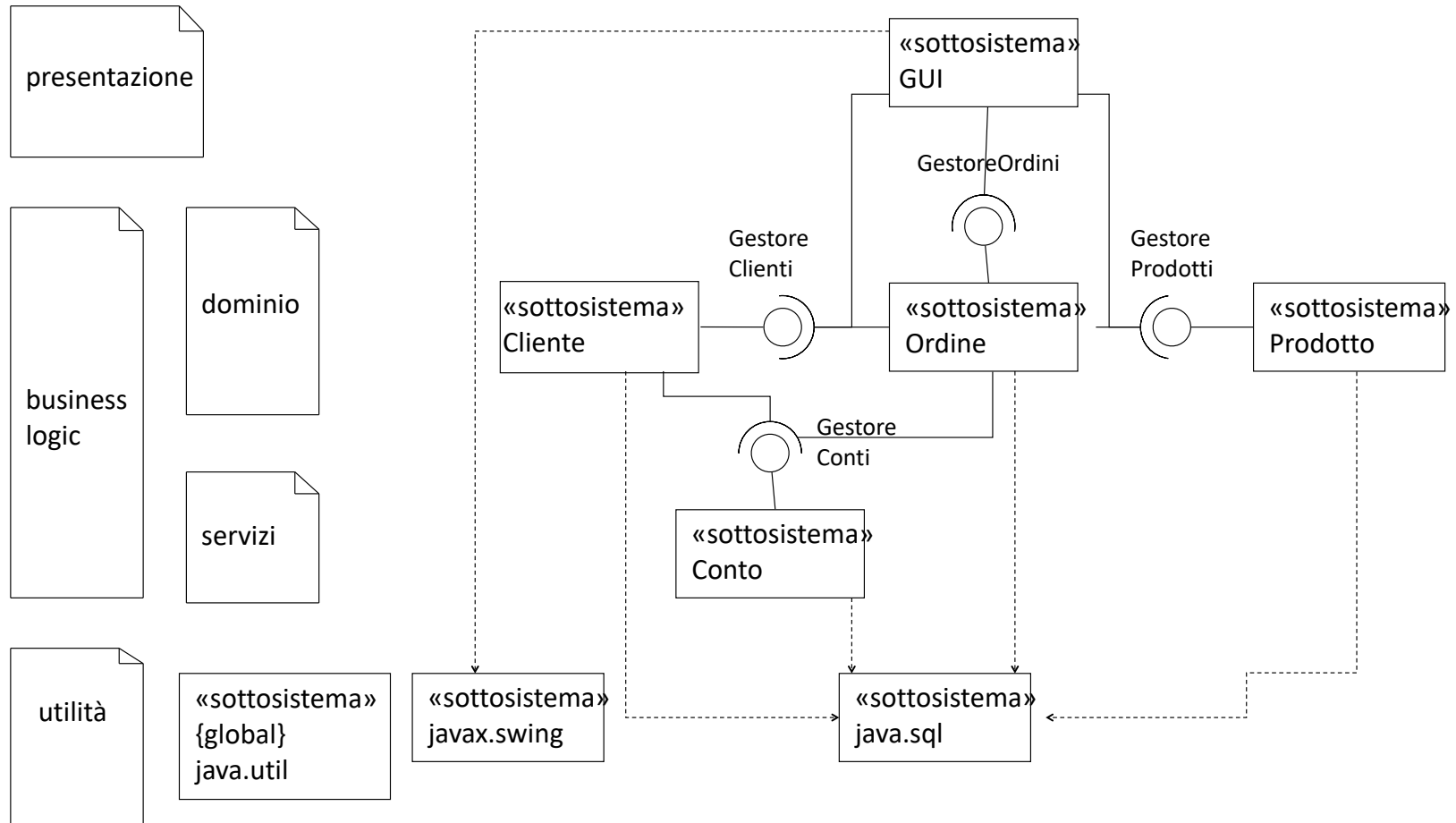
Progettare utilizzando le interfacce

- Usando le interfacce , si possono stabilire protocolli comuni che potrebbero essere realizzati da molte classi e molti componenti.
- Le interfacce quindi forniscono la capacità di inserire facilmente nuove classi nel sistema
 - *Se i sistemi sono progettati intorno alle interfacce, allora associazioni e invio di messaggi non sono più associati a specifici oggetti di classi particolari, ma piuttosto ad una particolare interfaccia*
- Nascondere i dettagli implementativi di sottosistemi complessi dietro un'interfaccia semplice e ben definita è conosciuto come il pattern di progetto (design pattern) Façade:
 - *Individuare parti del sistema tra loro coese*
 - *Raggruppare tali parti in package «sottosistema»*
 - *Definire un'interfaccia per quel sottosistema*

Architettura

- L'insieme dei sottosistemi e delle interfacce di progetto costituisce l'architettura di alto livello di un sistema.
- Per capire e mantenere questa architettura, l'insieme dovrebbe essere organizzato in modo coerente, applicando un architettura a strati
 - *i sottosistemi sono organizzati in strati*
 - *i sottosistemi in ogni strato sono semanticamente coesi es. Strato di Presentazione e Strato di Dominio*
 - *Le dipendenze tra strati devono essere gestite molto attentamente*
 - *Le dipendenze tra strati sono unidirezionali e tutte nello stesso senso*
 - *Tutte le dipendenze sono mediate da interfacce*

Esempio



Utilizzando le interfacce

- Vantaggi:

- *Quando progettiamo con le classi, stiamo progettando specifiche implementazioni*
- *Quando progettiamo con le interfacce, stiamo invece progettando dei contratti che possono essere realizzati da molte implementazioni diverse (classi)*
- *La progettazione per interfacce svincola il modello da dipendenze dell'implementazione e ne aumenta la flessibilità e la estendibilità*

- Svantaggi:

- *Maggiore flessibilità significa anche più complessità. In teoria, ogni operazione di ogni classe potrebbe essere un'interfaccia, ma in questo modo il sistema diventerebbe incomprensibile.*
- *Maggior costo a livello di prestazioni.*

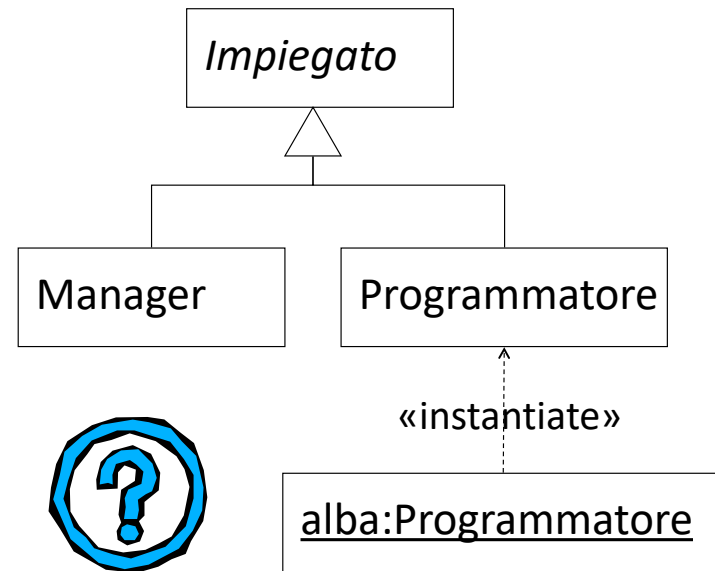
Aggregazione e composizione

In UML

- una aggregazione è una associazione che rappresenta una relazione intero-parte
- una composizione (aggregazione composta) è una forma forte di aggregazione in cui
 - *una parte appartiene a un composto alla volta*
 - *ciascuna parte appartiene sempre a un composto*
 - *il composto è responsabile della creazione e cancellazione delle sue parti*

Aggregazione vs. ereditarietà

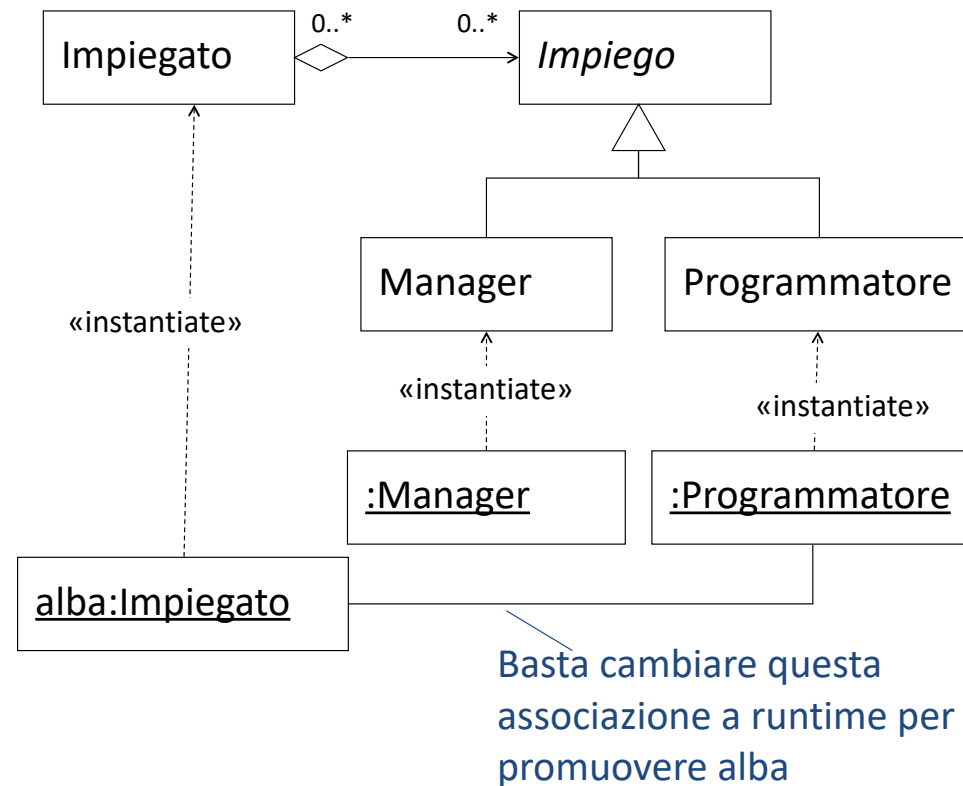
- E' in assoluto la forma di interdipendenza più forte tra classi
- Non è possibile cambiare la classe di un oggetto a tempo di esecuzione
- C'è un errore semantico fondamentale: E' più corretto dire che impiegato è un impiego o che impiegato ha un impiego?



1. Come possiamo promuovere Alba?
2. Può Alba avere più di un impiego?

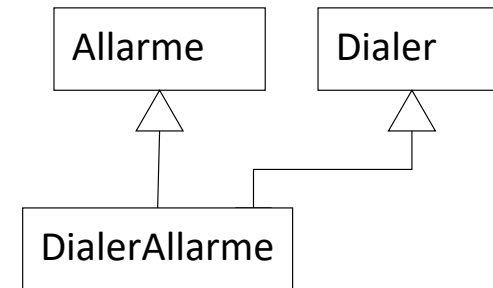
Una soluzione migliore...

- Usando l'aggregazione otteniamo una semantica corretta:
 - *Un Impiegato ha un impiego*
- Con questo modello più flessibile, gli Impiegati possono avere più di un Impiego

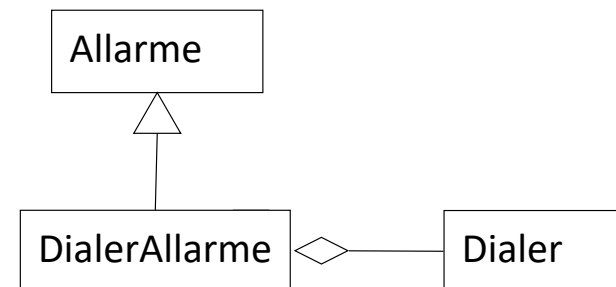


Ereditarietà Multipla

- A volte una classe può avere più di una classe genitore
- Il principio di sostituibilità è la relazione “is-a” devono essere applicabili tra la sottoclasse e ciascuna delle superclassi interessate
- L’ereditarietà multipla a volte offre la soluzione più elegante. Tuttavia:
 - *Non tutti i linguaggi la supportano (es. Java)*
 - *Può essere sempre sostituita con ereditarietà singola e delegazione*



In questo esempio **DialerAllarme** suona l’allarme e chiama la polizia – logicamente è sia un **Allarme** che un **Dialer**



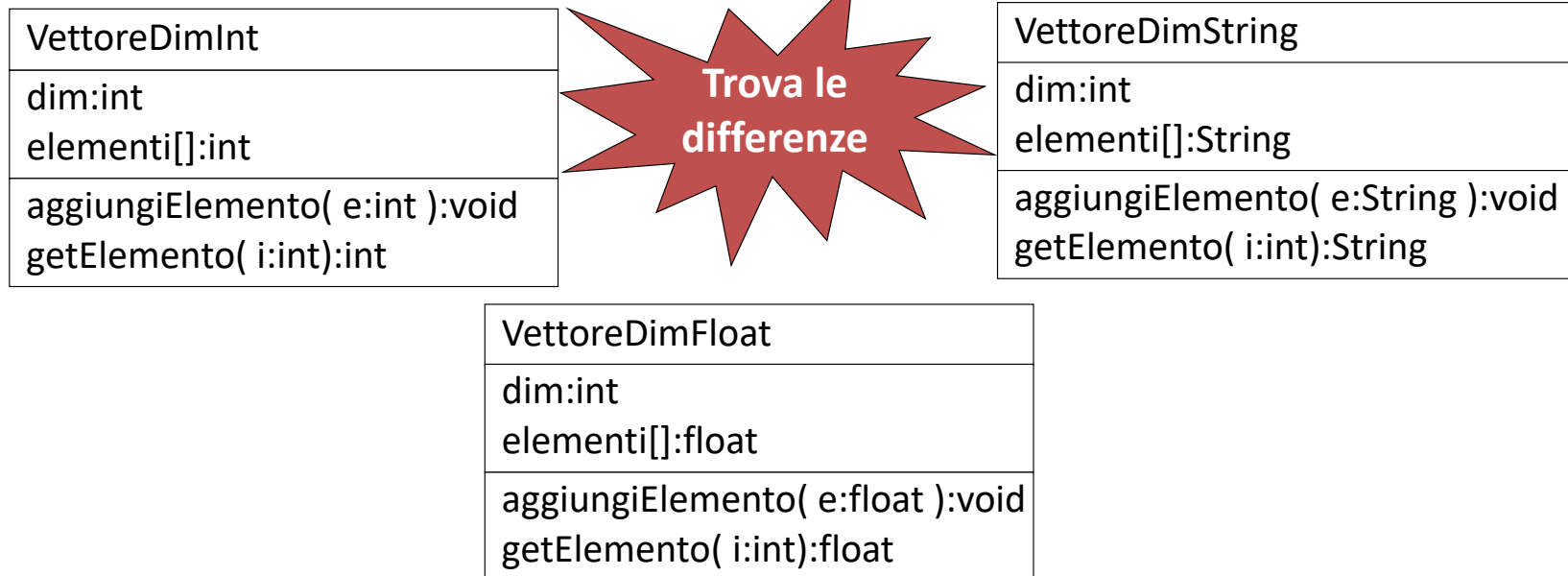
Ereditarietà vs. Realizzazione di Interfaccia

- Con l'ereditarietà si definiscono:
 - *L'interfaccia– le operazioni pubbliche della classe base;*
 - *L'implementazione: gli attributi, le relazioni e i metodi della classe base*
- Con la definizione di un interfaccia si definisce:
 - *L'interfaccia – un insieme di operazioni pubbliche, attributi e relazioni non implementati*

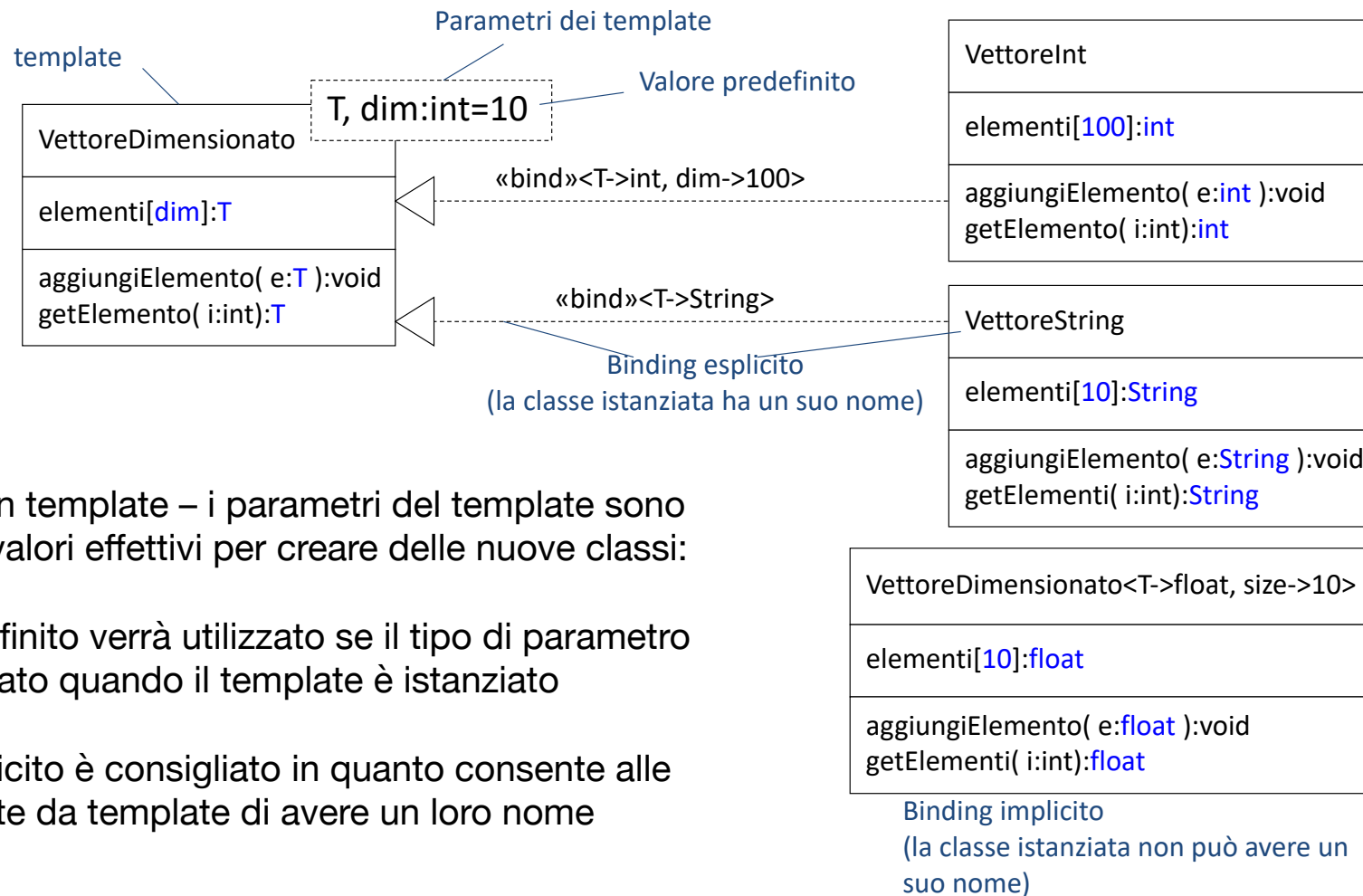
Utilizza l'ereditarietà quando vuoi ereditare l'implementazione
Utilizza la realizzazione di Interfaccia quando vuoi definire un contratto

Template di Classe

- Fin ora, per definire una classe di progettazione si è detto che è necessario specificare esplicitamente i tipi dei suoi attributi e i tipo di parametri ed il valore restituito dalle sue operazioni. Tuttavia, può a volte limitare il riuso



Sintassi dei template di classe



- Istanziare di un template – i parametri del template sono sostituiti con valori effettivi per creare delle nuove classi:
- Il valore predefinito verrà utilizzato se il tipo di parametro non è specificato quando il template è istanziato
- Il binding esplicito è consigliato in quanto consente alle classi istanziate da template di avere un loro nome

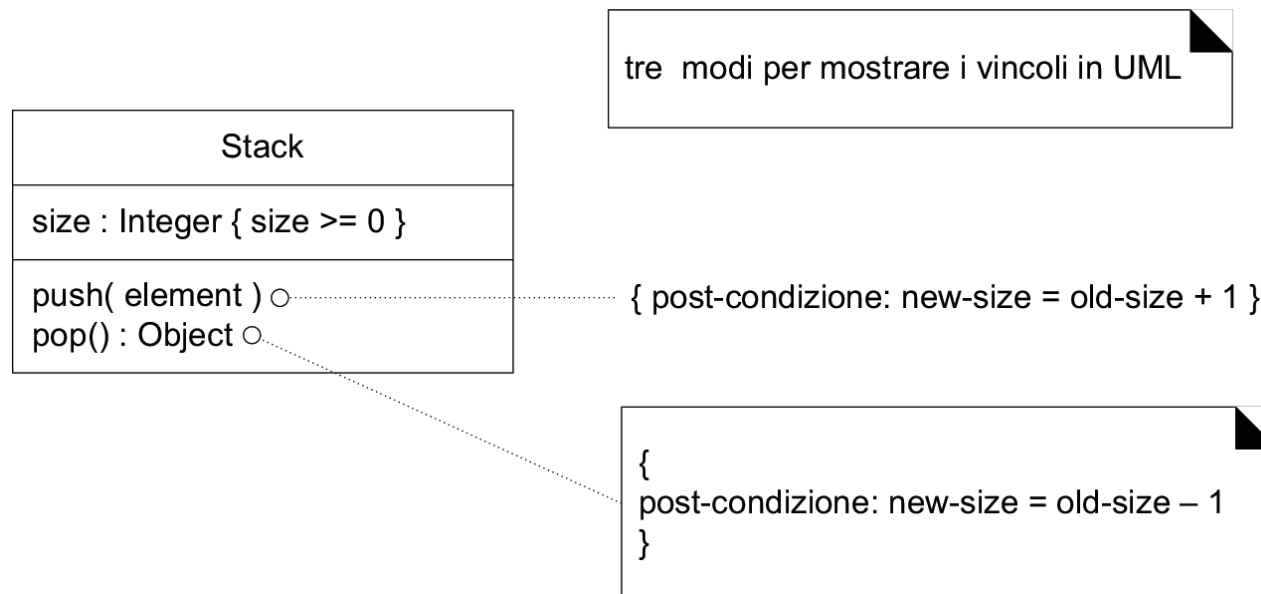
Template e ereditarietà multipla

- Si consiglia di utilizzare i template e l'ereditarietà multipla durante la progettazione solo quando il linguaggio di sviluppo li supporta

linguaggio	template	ereditarietà multipla
C#	No	No
Java	No	No
C++	Sì	Sì
Smalltalk	No	No
Visual Basic	No	No
Python	No	Sì

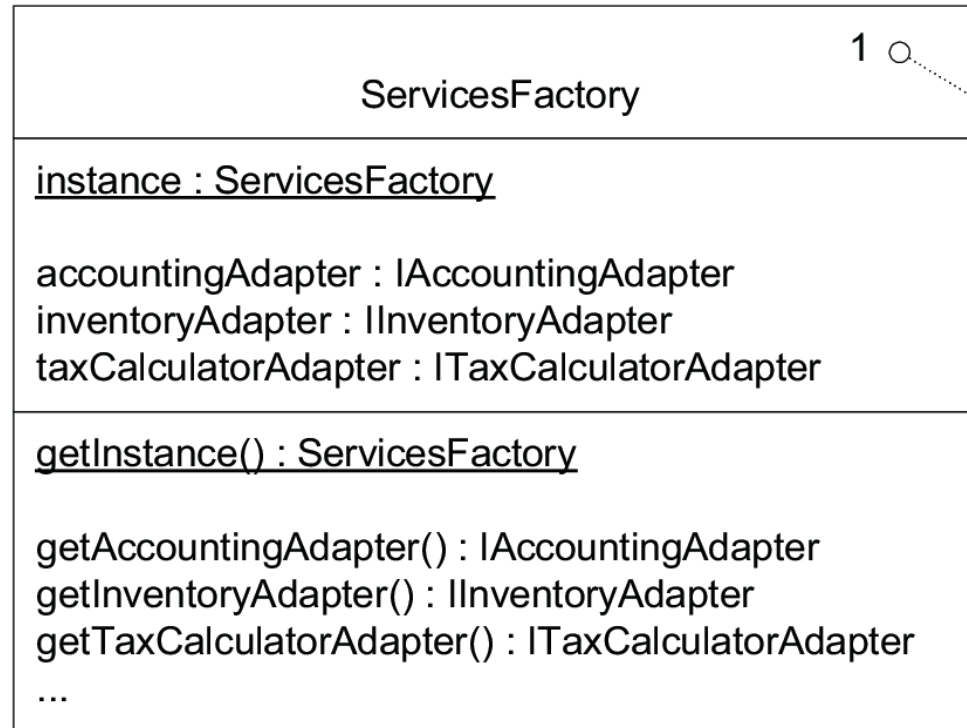
Vincoli

- Un vincolo indica una restrizione o una condizione su un elemento di un diagramma
- Il testo può essere in linguaggio naturale o altro, per esempio nel linguaggio di specifica formale per UML ovvero OCL



Classi Singleton

notazione UML: nel rettangolo di una classe, un attributo o un metodo sottolineato indicano un membro statico (a livello di classe), anziché un membro d'istanza

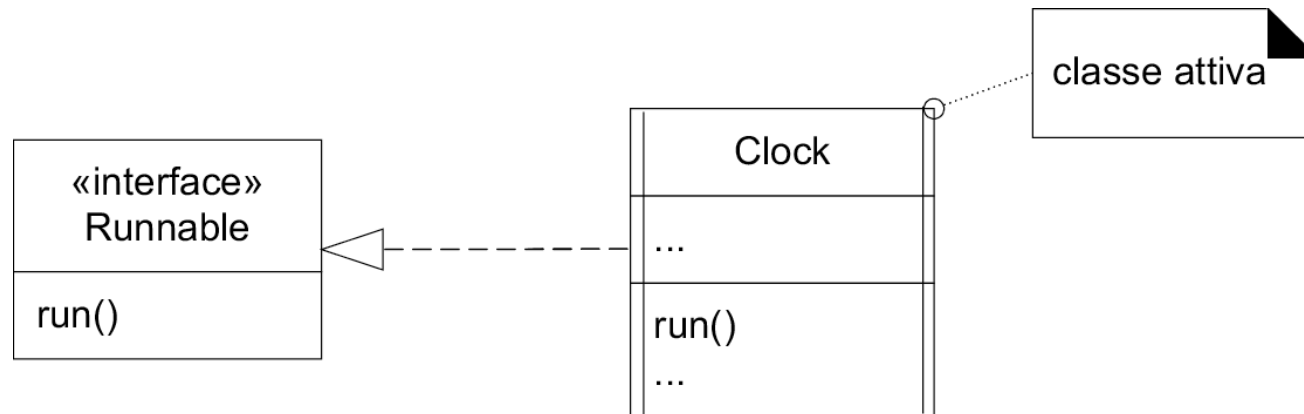


notazione UML: questo '1' può essere usato, come opzione, per indicare che verrà creata una sola istanza (un singleton)

Sezioni definite dall'utente

.DataAccessObject
id : Int ...
doX() ...
eccezioni sollevate DatabaseException IOException
responsabilità serializzare e scrivere oggetti leggere e deserializzare oggetti ...

Classi attive



Relazione tra diagrammi di interazione e diagramma delle classi

